

水泥烧成系统电除尘器的协同优化控制

摘 要

本文从附件精确辨识出总功率生成律 $P = 73.1 + 0.0850 \sum_i U_i^2 + 5.39 \times 10^4 \sum_i T_i^{-1}$ (回顾性测试 $R^2 = 0.993$ 、RMSE=5.98 kW、偏差 -0.02 kW)，并据此给出四问的直接答案。该式的第一项是电场功率 ($\propto$ 电压平方)，第二项是振打功率 ($\propto$ 振打频率 $60/T_i$)；由此发现**振打环节在中位工况下占总功率 38.8%**，与电压同为一级节能杠杆，而入口温度、浓度、流量对总功率的增量解释力低于 10^{-3} 。

问题一：可识别层给出上述功率律、历史操作规律（入口浓度升高时缩短四场振打周期， $r = -0.21 \sim -0.28$ ，四个时窗一致）与两段持续异常工况的控制响应；不可识别层为出口浓度——10030 条有效记录全部落在 $48.74 \sim 50.00 \text{ mg/Nm}^3$ ，5491 条恰为 50，与全部 12 个变量的相关系数绝对值均小于 0.017，故排放层以 Deutsch-Anderson 灰箱推演并全程标注。振打峰值不再取假定指数：由“每周期积灰量 $\propto L \cdot T$ ”推得单次峰值随周期近似线性放大，而对小时均值的贡献 $\propto L \cdot T \cdot (1/T) = L$ 与周期无关。

问题二：以入口三变量聚类得 4 类工况（轮廓系数 0.4084）。以历史记录值 50 mg/Nm^3 为锚点、不作小数点缩放，在历史运行包络内用多起点 SLSQP 求解，四工况满足 10 mg/Nm^3 的最低功率为 $1851.1 \sim 2104.3 \text{ kW}$ ，加权 1973.0 kW ；相对历史实际加权功率 1783.2 kW 上升 **10.6%**（日电耗 $42.8 \rightarrow 47.4 \text{ MWh}$ ）。收紧排放必然增电耗，不存在“等排放节能”。

问题三：单位功率去除收益 $\eta_i = K_0 \alpha_i / (\sum \alpha \cdot b_i U_i^{ref2})$ 给出解析优先级——因第三电场参考电压最低，其性价比在四类工况中均居首，“前级优先”实由排序约束 $U_1 \geq U_3 + 3$ 导出，而非权重 α_i 本身。

问题四：闭式解 $\Delta P = P_{field} \ln[(10 - B)/(5 - B)]/K_{10}$ 给出 $123.1 \sim 142.4 \text{ kW}$ ，与数值解相差不足 1%；加权功率由 1973.0 升至 2105.5 kW ，**增幅 6.72%**（分工况 6.59%~6.83%），日电耗增 3.2 MWh 。增幅严格正比于 $2/p$ ($K \propto U^p$)， p 是唯一需现场标定的标量，一次电压阶跃试验即可确定。关键约束不在电耗而在**容量**： 10 mg/Nm^3 仍处历史电压包络内， 5 mg/Nm^3 需各场电压超出历史最大值最多 5%。八组最小功率解的马氏距离平方为历史经验 97.5% 外壳的 2.4~7.0 倍；若要求严格留在该外壳内，八组中七组仍可行，代价为多耗 1.5%~6.1% 电力。

关键词：电除尘器；可识别性审计；Deutsch-Anderson 灰箱；振打再飞扬；约束优化

1 问题重述与分析

1.1 问题背景

电除尘器通过高压电场使粉尘荷电并向收尘极迁移。提高电场电压通常能够增强荷电与迁移,但会提高运行功率并受到火花、绝缘和联锁条件限制;振打过于频繁会增加再飞扬次数,周期过长又可能导致积灰加重和单次脱落量增大。水泥工业颗粒物排放受到国家标准约束 [1], 题目进一步给出 10 mg/Nm^3 和 5 mg/Nm^3 两个控制目标, 要求同时考虑排放、总功率和振打峰值。

本文研究对象的四电场电除尘器结构及其可测量与可调量如图 1 所示: 含尘烟气依次流经四个电场, 每个电场由独立高压电源施加电压 U_i 并由独立振打装置按周期 T_i 清灰, 入口侧可测入口温度 H 、入口粉尘浓度 C_{in} 与烟气流量 Q , 出口侧可测出口粉尘浓度 C_{out} 与总功率 P 。

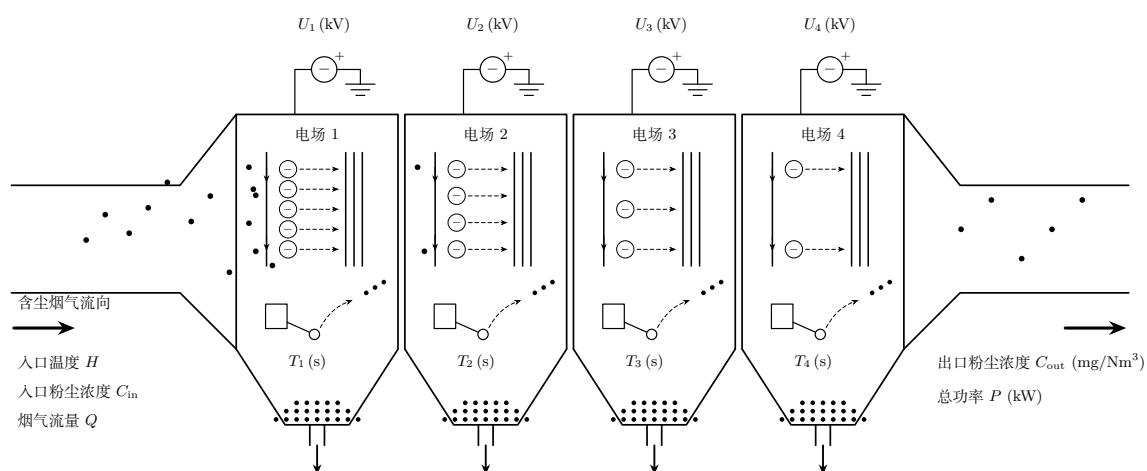


图 1 四电场电除尘器结构与变量定义

题目包含四项相互依赖的任务: 第一, 解释入口温度、入口浓度、烟气流量、四电场电压和振打周期与出口浓度之间的关系, 并分析振打峰值; 第二, 划分典型工况, 在 10 mg/Nm^3 约束下寻找最低功率控制参数; 第三, 选取差异显著的工况解释控制优先级; 第四, 将约束收紧至 5 mg/Nm^3 并估计功率代价。

1.2 总体思路

附件的两列结果变量信息量极不对称: 总功率几乎是控制量的确定性函数, 而出口浓度被压缩在量程上限附近。若不区分地把两者都当作监督目标, 模型只会出口浓度上复现传感器封顶值。本文因此采用“先分层、再优化”的路线:

1. 审计每一列的可识别性, 把“可由附件验证”与“只能由先验推演”分开;
2. 对可识别的总功率, 按物理形式 (电场功率 $\propto U^2$ 、振打功率 $\propto 1/T$) 建立并检验模型;
3. 对不可识别的排放层, 以记录值本身为锚点建立 Deutsch-Anderson 灰箱, 不作任何未经证实的量纲改写;
4. 在历史运行包络内求解约束优化, 并把“解在何处越出历史经验”作为结论的一部分显式报告。

第 3 点是本文与常见做法的关键差别。附件出口浓度约 50 mg/Nm^3 , 而题目要求 10 与 5 mg/Nm^3 , 两者相差 $5 \sim 10$ 倍。承认这一差距, 问题就自然变成“由现状到超低排放的控制路径与代价”; 反之, 若为迁就目标区间而假设记录值存在小数点错误, 第二问就会退化为“把排放放宽到 10 mg/Nm^3 能省多少电”, 与题意相反。

2 数据审计与可识别性门禁

2.1 数据完整性与派生变量

附件覆盖 2024 年 5 月 1 日 0 时至 5 月 7 日 23 时 59 分，共 10080 条记录，采样间隔 1 min，时间戳无重复、无缺口。原始字段包括入口温度 H_t 、入口浓度 $C_{in,t}$ 、烟气流量 Q_t 、四电场电压 $U_{i,t}$ 、四电场振打周期 $T_{i,t}$ 、出口浓度 $C_{out,t}$ 和总功率 P_t 。出口浓度缺失 50 条 (0.50%)，其余字段完整。为表达过程负荷和振打风险，构造粉尘质量负荷

$$L_t = C_{in,t}Q_t/1000 \tag{1}$$

其中 C_{in} 单位为 g/Nm^3 , Q 单位为 Nm^3/h , 故 L 单位为 kg/h 。另构造电压平方和、振打频率 $60/T_i$ 与每周期积尘代理 $L_iT_i/3600$ 。所有聚类与回归参数仅在训练期 (5 月 1—5 日) 估计，验证期 (5 月 6 日) 与回顾性测试期 (5 月 7 日) 不参与拟合，划分按时间顺序而非随机打乱。

2.2 逐列可识别性结论

对 14 列逐一审计的结论见表 1：只有出口浓度一列不可用，其余各列信息量充足。

表 1 附件逐列可识别性审计结论

数据块	审计证据	可用性
C_{out} 出口浓度	有效值 10030 条全部位于 48.74~50.00 mg/Nm^3 ，5491 条恰等于 50；与全部 12 个变量的同期相关系数绝对值均 < 0.017 ，1~120 min 滞后同样无稳定关系；封顶分类 AUC=0.5098，未封顶子样本回归验证 $R^2 = -0.019$	不可识别，不可修复
P 总功率	由控制量近乎确定，残差标准差约 6 kW (见 §5.2)	精确可辨识
H, C_{in}, Q 工况变量	连续波动并含两段持续异常事件 (§4.3)	工况分层与扰动分析
U_i, T_i 控制设定	与工况变量存在稳定相关结构 (§4.2)	可提取历史操作规律

出口浓度的分布与截顶情况如图 2 所示。**必须强调：**以上结果只能说明“当前记录无法提供可学习信号”，不能反推物理变量对真实排放没有影响。已有数据驱动研究使用宽负荷下的机组负荷与各电场二次电流训练出口浓度模型 [8]；对照来看，本附件既缺少二次电流，出口记录也没有足够变化，故不具备相同的辨识条件。附录 B 给出以 C_{out} 为直接目标建模的对照结果，可供与其他方案交叉比较。

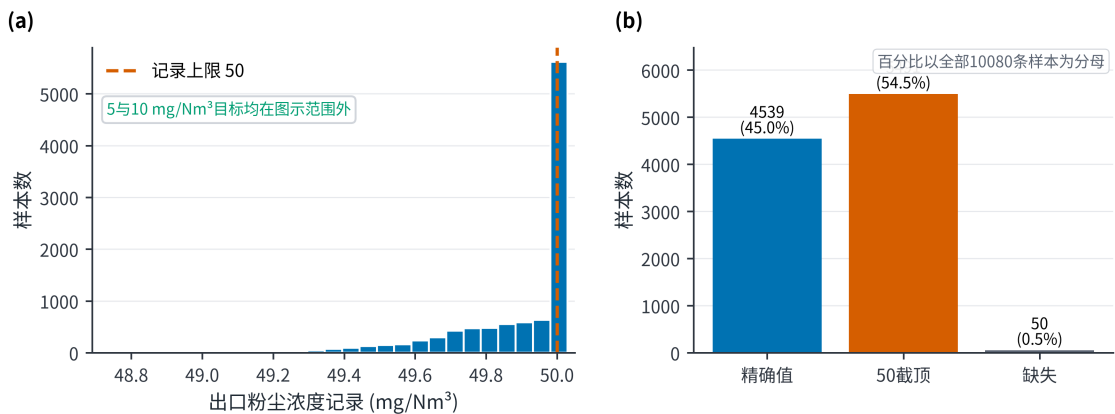


图 2 出口浓度数据诊断

首 24 h 过程量的变化如图 3 所示。入口变量和功率具有连续变化和工况迁移特征，而出口浓度记录几乎被压缩在单点附近。

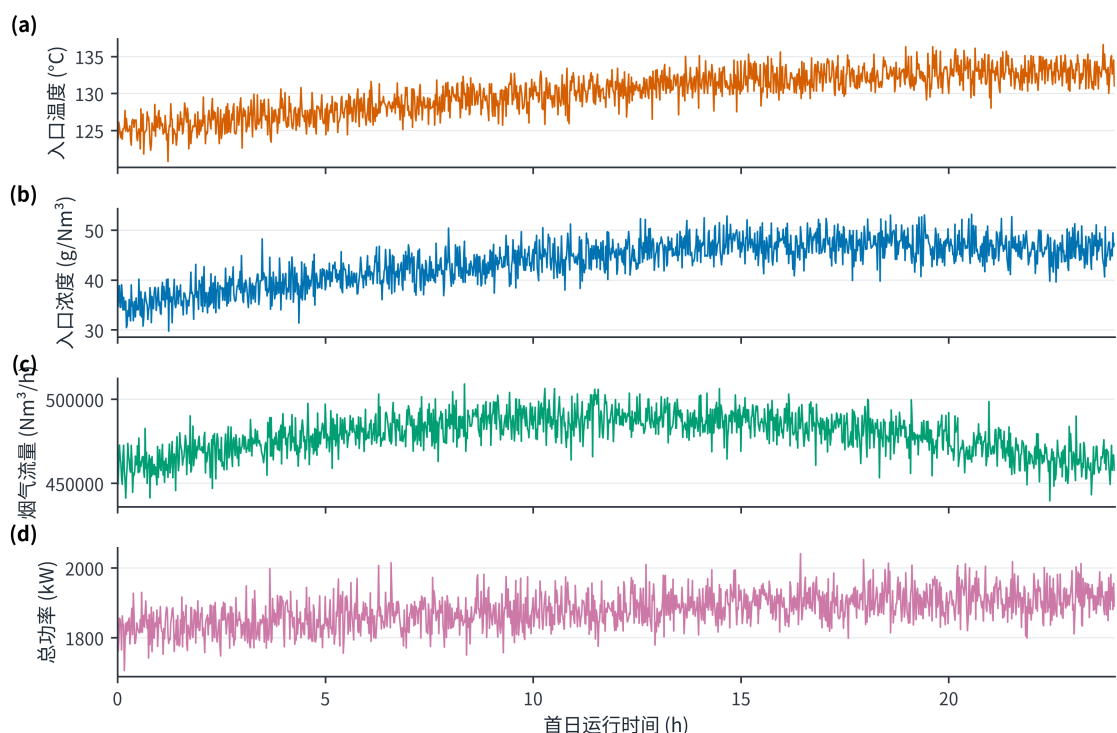


图3 首24 h过程量时序

2.3 排放锚点的取法

本文不对出口记录作任何缩放,直接以各工况出口浓度记录中位数(均为 50 mg/Nm^3)为 Deutsch-Anderson 锚点。由此确立的问题定位是:设备当前远未达到超低排放水平,本文求解的是“由现状降至 10 与 5 mg/Nm^3 ”的控制路径与代价。这一取法有三个后果,全部在后文显式报告:目标点必然落在历史电压包络的上端 (§5.5); $10 \rightarrow 5 \text{ mg/Nm}^3$ 的功率增幅显著小于由更低锚点出发时的估计 (§7);排放层全部数值仍为条件式情景推演,须在补采未封顶排放数据后重新标定(附录 C)。

3 模型假设、符号与总体框架

3.1 模型假设

本文将假设分为数据事实、情景假设、优化边界与运行约束四类,并逐条标注其证据状态,如表 2 所示。

表2 模型假设及证据状态

类别	假设	证据状态
数据事实	7 天记录按分钟连续,除 50 条出口浓度缺失外不填造目标值	附件可核验
数据事实	总功率按 U_i^2 与 $1/T_i$ 的物理形式建模,按时间滚动验证	可由验证与测试结果检查
数据事实	排放锚点取出口浓度记录中位数 50 mg/Nm^3 ,不作缩放	附件记录值
情景假设	去除指数按 $K = K_0 S(U)$ 随电压幂次 p 缩放,中心取 $p = 2$	Deutsch 迁移速度形式,未经现场标定
情景假设	电场贡献权重 α_i 取等权、弱前级与中心前级三种形状	工程先验与消融
情景假设	振打峰值指数 $\gamma = 1$ 由积灰量正比关系推导,扫描 1.00/1.30/1.60	本文推导+结构情景
优化边界	控制量位于各工况训练期实测最小—最大范围内	历史运行包络,不等同设备安全边界
优化边界	5 mg/Nm^3 档允许电压上界外推至历史最大值的 1.05 倍	显式声明的外推,须经容量核算
运行约束	前级电压高于后级至少 3 kV,后级振打周期长于前级至少 100 s	工程排序约束

3.2 主要符号

表 3 主要符号与单位

符号	含义	单位
U_i	第 i 电场电压	kV
T_i	第 i 电场振打周期	s
C_{in}, C_{out}	入口、出口粉尘浓度	g/Nm ³ 、mg/Nm ³
Q	烟气流量	Nm ³ /h
L	粉尘质量负荷	kg/h
$P, \hat{P}$	实际、预测总功率	kW
b_i, c_i	电场功率系数、振打功率系数	kW/kV ² 、kW·s
K, K_0	去除指数及其历史锚点值	—
$S(U)$	电压强度因子, $U = U^{ref}$ 时为 1	—
p	去除指数对电压的幂次, $K \propto U^p$	—
$B(T)$	振打引起的峰值增量代理	mg/Nm ³
ρ, γ	振打相位叠加尺度、峰值—周期指数	—

框架上, 工况与总功率由实测数据辨识, 排放约束由显式先验建立, 两者仅在支持域优化阶段合并; 本文对每一个数值结论都标注其属于“实测辨识”还是“条件式推演”。

4 问题一：可识别的影响关系与振打峰值

第一问要求解释各变量与出口浓度的关系并分析振打峰值。按 §2 的审计结论, 本节分两层作答: 先给出附件能够支持的三类实证结论, 再给出排放层只能由机理推演的部分。

4.1 总功率的生成律 (实证)

在训练期以物理形式拟合, 得到

$$P = 73.09 + \sum_{i=1}^4 b_i U_i^2 + \sum_{i=1}^4 \frac{c_i}{T_i} \quad (2)$$

其中 $b = (0.0849, 0.0851, 0.0847, 0.0853)$ kW/kV²、 $c = (5.43, 5.37, 5.49, 5.26) \times 10^4$ kW·s。四个电场的 b_i 相互差异不足 0.8%, 说明四场高压电源具有相同的电气特性; 聚合形式为 $P \approx 73.0 + 0.0850 \sum U_i^2 + 5.39 \times 10^4 \sum T_i^{-1}$ 。该式的物理含义直白: **电场功率正比于电压平方, 振打功率正比于振打频率** $60/T_i$ 。模型的检验结果见 §5.2。

由此可直接读出一个此前被忽略的结构性事实。在训练期中位控制设定下, 电场功率 1020.3 kW、振打功率 693.7 kW, 即**振打环节占总功率的 38.8%**。振打周期在题面中形似配角, 其对电耗的影响却与电压同量级。相应地, 把四场周期由中位延长至历史 P95 可回收 56.1 kW, 延长至历史最大值可回收 109.1 kW, 代价分别是单次积灰量增加 8.4% 与 17.9% (图 5b)。§7.3 将说明, 这一可回收量几乎恰好抵消排放限值收紧一档的功率代价, 这正是“不能为省电无限延长振打周期”的定量依据。

另需指出: 把入口温度、浓度、流量加入该式后, 验证与测试 R^2 的变化均小于 10^{-3} (表 7 末行)。**总功率在物理上只由控制量决定, 与工况变量无关**; 后文“分工况优化”得到的功率差异, 全部来自各工况不同的排放锚点与支持域, 而非工况本身直接影响功率。

4.2 历史操作规律 (实证)

出口浓度不可识别, 并不意味着附件没有关于“操作策略”的信息。图 4(c) 给出全记录期工况变量与控制设定的相关矩阵, 其中两类规律值得报告:

规律一（稳定）：入口浓度升高时，四个电场的振打周期一律缩短（ $r = -0.208, -0.211, -0.277, -0.278$ ）。该方向在训练期、验证期与测试期三个时窗内符号一致，是附件中可复现的操作规律，与“高负荷下不宜延长周期”的工程直觉相符。

规律二（不平稳）：入口浓度升高时，前两电场电压上升（全记录 $r = +0.172, +0.166$ ）而后两电场几乎不变（ $r = +0.019$ ）——但这一规律并不贯穿全周。 $r(C_{in}, U_1)$ 在前 5 天为 -0.052 ，在第 6、7 天分别为 $+0.507$ 与 $+0.492$ 。也就是说，“高浓度时抬升前级电压”是后两天才出现的操作模式。本文如实报告这一非平稳性：前后级响应的不对称在全记录与后两天成立，但不能当作贯穿全周的稳定规律；这也提示历史数据中至少存在一次操作策略调整，第三问的经验对照因此只作为参照，而不作为定论。

表 4 工况变量与控制设定的历史相关系数（全记录期）

控制量	与入口温度 H	与入口浓度 C_{in}	与烟气流量 Q
U_1	-0.130	+0.172	+0.074
U_2	-0.127	+0.166	+0.070
U_3	-0.112	+0.019	-0.068
U_4	-0.113	+0.019	-0.063
T_1	-0.075	-0.208	-0.180
T_2	-0.064	-0.211	-0.175
T_3	-0.159	-0.277	-0.130
T_4	-0.151	-0.278	-0.135

4.3 两段持续异常工况（实证）

附件含两段持续数十分钟以上的异常工况，其控制端响应方向相反，构成理解设备行为的两个直接入口（图 4a、4b）。

表 5 两段持续异常工况及控制端响应

事件	时间窗	特征	控制端响应
入口浓度尖峰	05-02 14:02—15:49，其中 41 min 超过 60 g/Nm ³	入口浓度峰值 72.47 g/Nm ³ （基线中位 37.41）	U_1 均值 63.7 kV（峰值 67.8），振打周期同步缩短，总功率均值 1924 kW、峰值 2020 kW
入口温度尖峰	05-06 13:00—14:59，持续 120 min	入口温度峰值 158.2 °C（基线中位 126.5），同期入口浓度降至均值 25.9 g/Nm ³	U_1 均值降至 55.5 kV，总功率均值降至 1668 kW

第二段事件在工程上更值得注意：**140~160 °C 恰是水泥窑尾粉尘比电阻的高值区**，比电阻过高会引发反电晕，使有效驱进速度下降、火花率上升，是电除尘最难运行的工况带 [3-4]。附件在该窗口的记录显示控制端选择了降压而非升压，这与“高比电阻区应限制电流密度以抑制反电晕”的工程处置一致，但附件无二次电流与火花率记录，无法据此判定其是否为最优处置。本文由此得出两点：其一，第二问的工况聚类以温度为主轴之一是必要的；其二，本文四类工况的温度中心分别约 120.7 与 130.4 °C，**均未覆盖该高比电阻带**，故所有结论对该带外推无效，须单独补采标定 (§9.3)。

4.4 排放响应的灰箱骨架（条件式推演）

Deutsch-Anderson 关系以指数形式描述电除尘效率，其理想化形式忽略再飞扬、气流分布和粒径变化，因此适合作为灰箱骨架而不是无条件精确模型 [2-4]。White 的经典专著 [12] 与 Parker 的电除尘器电气运行专著 [13] 均强调电气运行、粉尘性质与振打之间的系统耦合。定义去除指数

$$K = \ln \frac{1000C_{in}}{C_{out}}, \quad C_{out} = 1000C_{in}e^{-K} \quad (3)$$

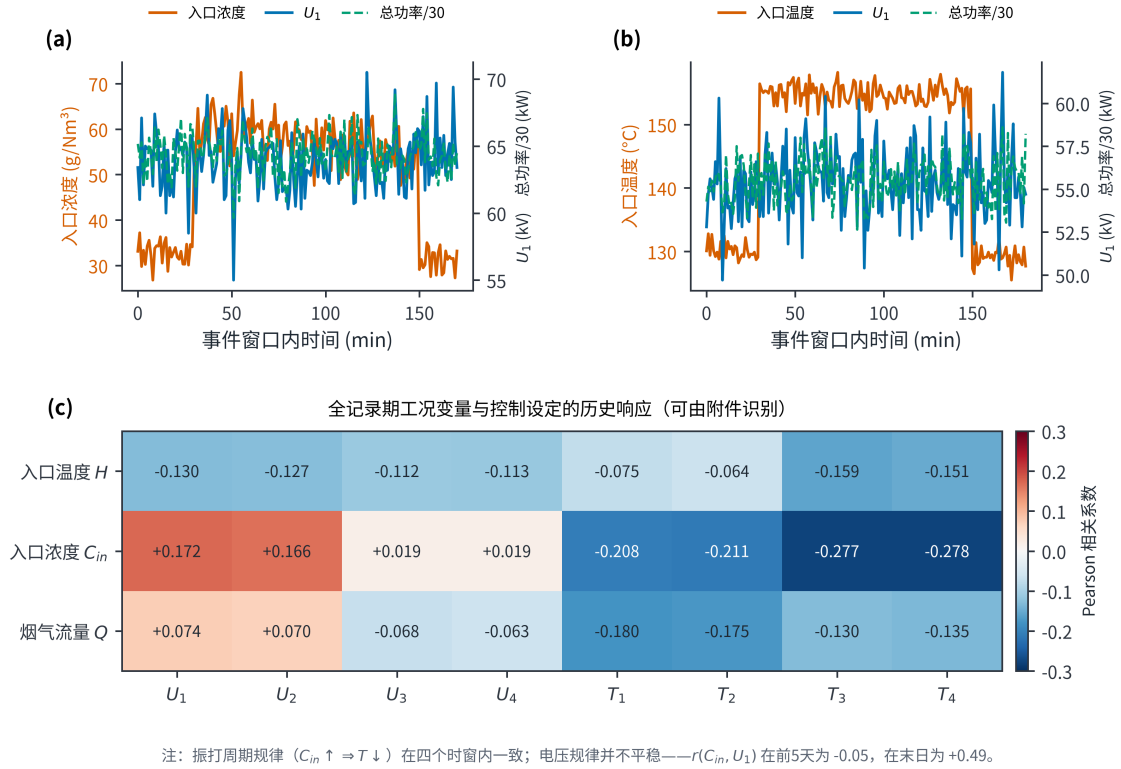


图 4 附件中可识别的运行证据

对工况 k ，以历史中位电压 $U_{i,k}^{ref}$ 和出口浓度记录中位数 $C_{0,k} = 50 \text{ mg/Nm}^3$ 为锚点， $K_{0,k} = \ln(1000C_{in,k}/C_{0,k})$ ，四工况依次为 6.257、6.514、6.702、6.797。驱进速度与电场强度的平方近似成正比，故取

$$K_k(U, T) = K_{0,k} S_k(U) - \{G_k(T) - G_k(T^{ref})\}, \quad S_k(U) = \frac{\sum_i \alpha_i \left(U_i / U_{i,k}^{ref} \right)^p}{\sum_i \alpha_i} \quad (4)$$

$$G_k(T) = \sum_{i=1}^4 \beta_{i,k} \left(\frac{T_i}{T_{i,k}^*} + \frac{T_{i,k}^*}{T_i} - 2 \right), \quad \beta_{i,k} = \beta_i \left(\frac{L_k}{L_{med}} \right)^{1/2} \quad (5)$$

该式在 $U = U^{ref}, T = T^{ref}$ 处严格还原 $K = K_{0,k}$ ，即模型按构造复现历史运行点，不再需要额外的整体尺度参数 η ；全部电压敏感性收敛到单一幂次 p ，中心取 $p = 2$ 。 $G_k(T)$ 在 $T_i = T_{i,k}^*$ 处取最小值，表达周期过短与过长均不利：过短导致频繁再飞扬，过长导致积灰和单次脱落量上升。试验研究确实观察到振打再飞扬和最大允许周期之间的权衡 [6]。加入安全裕度 s 后，连续排放基值为 $C_{base,k} = 1000C_{in,k} \exp[-K_k(U, T) + s]$ 。

本节全部排放系数均为工程先验，不是由附件出口浓度估计得到的参数。

4.5 振打峰值：由积灰量推导而非假定

附件只有振打周期设定，没有实际振打时刻、相位和秒级浓度，因此不能识别逐次峰值。但峰值随周期的**标度关系**可以由质量守恒直接推出，无须假定指数：设单位时间的积灰速率正比于粉尘负荷 L ，则一个振打周期内积累的灰饼质量正比于 $L \cdot T$ ；单次振打抛出的再飞扬量正比于灰饼质量，故

表 6 灰箱排放模型参数来源与证据状态

参数	中心取值	含义	来源	是否由附件识别
p	2.0	去除指数对电压的幂次	Deutsch 迁移速度形式	否
α_i	(1.15,1.10,0.95,0.80)	电场去除贡献权重	工程先验与消融	否
β_i	(0.30,0.27,0.20,0.17)	周期偏离损失	工程情景	否
b_0	1.2	峰值尺度	工程情景	否
γ	1.0	周期—峰值指数	本文由积灰量推导	否
ρ	1.0	相位叠加尺度	同相保守情景	否
s	0.08	安全裕度	工程情景	否

$$B_k(T; \rho, \gamma) = b_0 \rho \left(\frac{L_k}{L_{med}} \right) \sum_{i=1}^4 w_i \left(\frac{T_i}{T_{i,k}^*} \right)^\gamma, \quad \gamma = 1 \quad (6)$$

由此得到两条互补的结论，也正是第一问关于振打峰值的完整回答：

- **单次瞬时峰值随周期近似线性放大：**周期延长 10% 与 20% 时，单次峰值代理相应放大 10% 与 20%；
- **对小时均值的贡献与周期无关：**单位时间的总再飞扬量正比于 $L \cdot T \times (1/T) = L$ ，只取决于粉尘负荷。

即延长振打周期并不改变再飞扬对小时均值的贡献，只是把同样多的粉尘集中到更少、更高的尖峰中释放。这一结论有两个直接用途：其一，它解释了为什么“延长周期省电”必须与瞬时峰值约束同时考虑 (§7.3)；其二，由于现行标准以小时均值考核，本文把瞬时峰值直接约束到限值的口径是**保守的** (§7.4)。中心情景取 $\rho = 1$ (四场同相叠加的保守上界)，错相情景取 $\rho = 0.80$ 或 0.65；敏感性分析仍扫描 $\gamma \in \{1.00, 1.30, 1.60\}$ ，以检验推导值附近的稳健性。

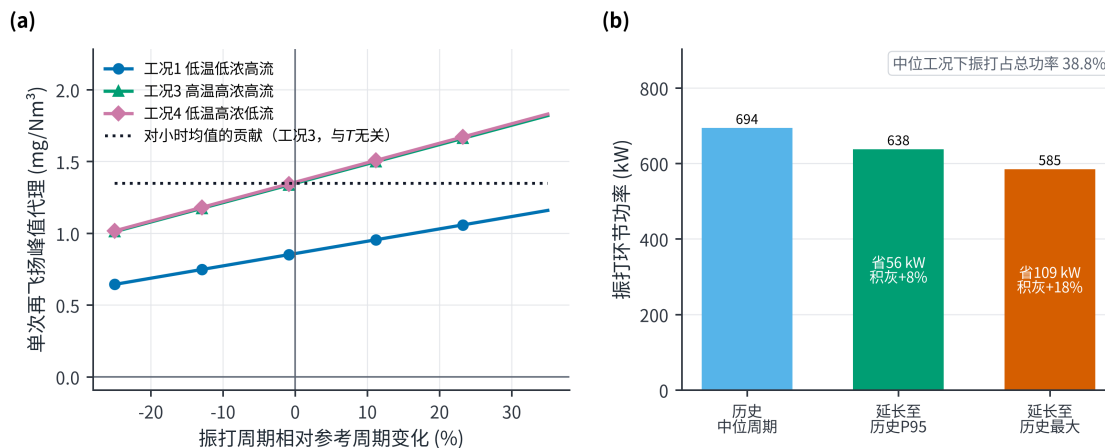


图 5 振打峰值机理与振打功率权衡

4.6 各因素方向性小结

在灰箱骨架中，固定其他条件时：提高 U_i 增加 K 并降低连续排放，但总功率按 U^2 上升；固定 K 时，入口浓度提高使出口质量浓度同比提高；流量增加会缩短有效停留时间并提高质量负荷，通常要求更高捕集强度；温度通过气体性质、粉尘比电阻和放电状态影响除尘性能，其中 140~160 °C 的高比电阻带最为不利 (§4.3)。这些方向来自机理与文献 [2-4]，而非附件回归。

5 问题二：典型工况划分与 10 mg/Nm³ 最低功率

5.1 工况聚类

取训练期的入口温度、入口浓度和烟气流量三个外生变量，标准化后执行 K-means[9]，避免用结果变量定义工况。比较 $k = 2, 3, 4, 5, 6$ ，轮廓系数 [10] 依次为 0.2963、0.3682、0.4084、0.3958、0.3934，因此取 $k = 4$ ；同一网格上的 Calinski-Harabasz 指标依次为 3204.2、4399.0、4721.2、4835.5、4712.0，最大值出现在 $k = 5$ ，即两项判据不一致。本文仍取 $k = 4$ ，理由是轮廓系数在 $k = 4$ 最高，且四类簇心恰好构成温度两水平（约 120.7 与 130.4 °C）与流量两水平（约 44.0 与 48.1 万 Nm³/h）的 2×2 因子结构，物理含义清晰。但须说明 CH 指标支持 $k = 5$ ，轮廓系数 0.4084 也只表明分离度中等，四类工况是统计分层而非唯一的物理状态划分；如 §4.3 所述，两段异常工况均落在四类簇心之外。

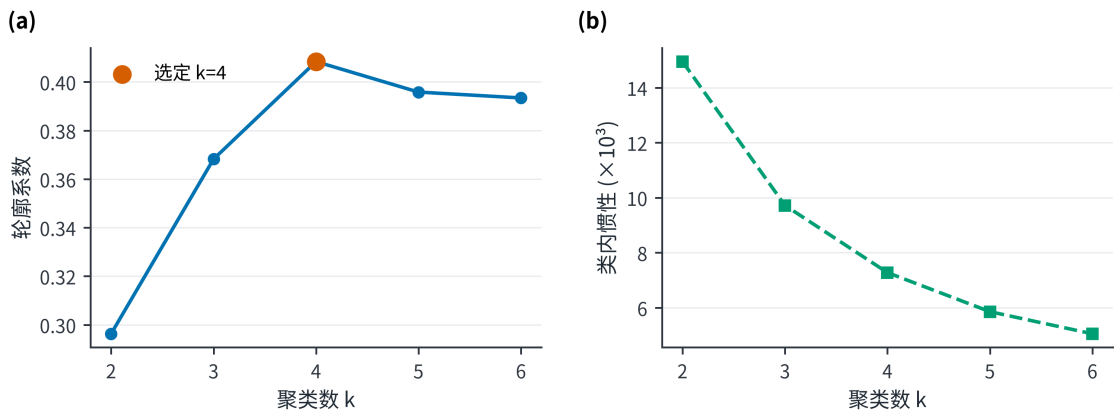


图 6 聚类数选择结果

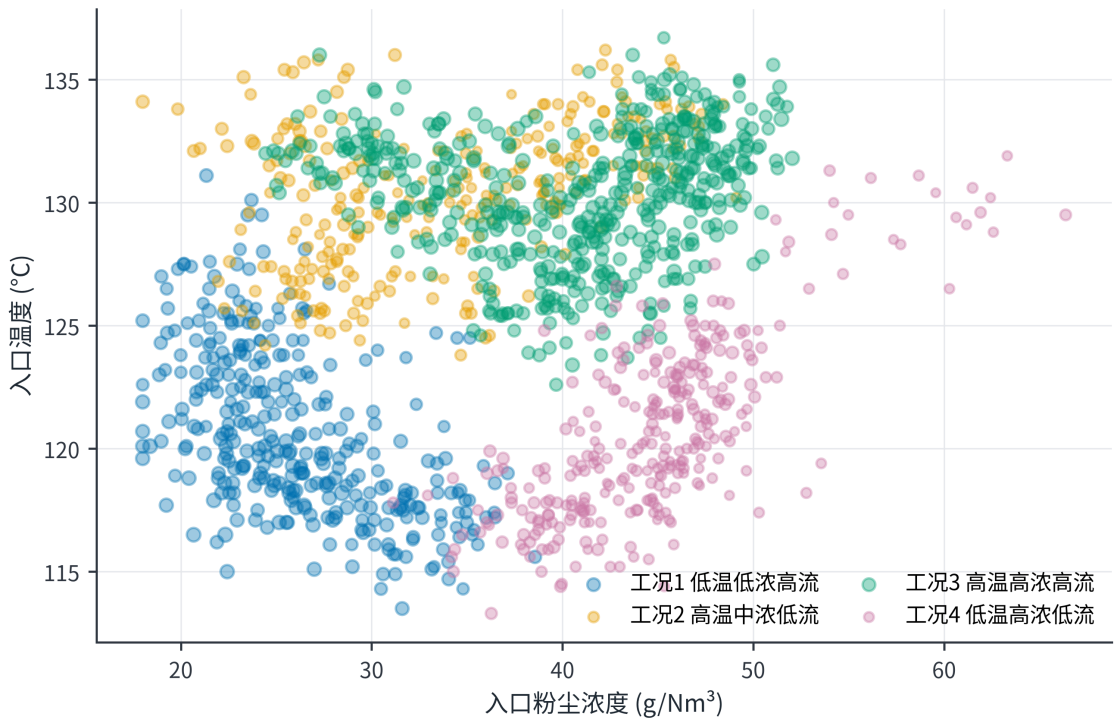


图 7 训练期典型工况分布

表 7 四类典型工况画像

工况	占比	温度/ $^{\circ}\text{C}$	入口浓度/(g/Nm ³)	流量/(Nm ³ /h)	粉尘负荷/(kg/h)	历史总功率/kW	锚点 K_0
1 低温 低浓度 高流量	23.35%	120.65	26.08	478257	12460	1818.35	6.257
2 高温 中浓度 低流量	20.06%	130.32	33.72	443591	14922	1870.89	6.514
3 高温 高浓度 高流量	34.93%	130.41	40.71	480851	19560	1763.08	6.702
4 低温 高浓度 低流量	21.67%	120.69	44.75	439990	19673	1696.55	6.797

5.2 功率模型的设定与检验

§4.1 的物理设定并非唯一可选。为说明设定本身的重要性, 本文在同一时间划分下比较四种设定: 均值基线、电压一次+周期一次的岭回归 [11]、电压二次+周期一次的岭回归, 以及本文的 $U_i^2 + 1/T_i$ 设定。岭回归的正则强度按训练期 3 个扩展窗口滚动验证选择, 物理设定无可调超参数。结果如表 8。

表 8 功率模型设定比较与时间外推结果

设定	验证 R^2	验证 RMSE/kW	测试 R^2	测试 RMSE/kW	测试平均偏差/kW
均值基线	-5.3259	140.18	-0.3733	83.51	-43.54
电压一次+周期一次 (岭)	0.9798	7.93	0.9180	20.41	-16.65
电压二次+周期一次 (岭)	0.9837	7.12	0.9571	14.76	-10.78
本文 $U_i^2 + 1/T_i$	0.9885	5.97	0.9930	5.98	-0.02
本文设定+工况变量	0.9885	5.97	0.9929	5.98	+0.08

三点结论值得强调。第一, **把振打周期作为线性项是设定误差而非数据漂移**: 线性 T 设定在回顾性测试段留下 -10.78 kW 的平均偏差, 且偏差在高功率区扩大; 改为 $1/T$ 后偏差降至 -0.02 kW, 测试 RMSE 由 14.76 kW 降至 5.98 kW, R^2 由 0.957 升至 0.993 (图 9)。若沿用线性设定, 这一偏差会被误当作“需要额外误差预算”的数据问题, 而实际上它是模型自身制造的。第二, T 是优化的决策变量, 线性设定会让模型认为延长周期的边际节电恒定, 而实际边际节电按 c_i/T_i^2 衰减, 会系统性扭曲最优周期。第三, 加入工况变量后指标不再改善, 印证 §4.1 的判断。

验证集绝对误差的 90% 分位数为 9.78 kW, 作为功率预算的误差附加量; 该附加量不改变候选排序, 也不保证在后续日期仍具有 90% 覆盖率。

5.3 优化模型

设控制向量 $z = (U_1, \dots, U_4, T_1, \dots, T_4)$ 。对每类工况 k 与限值 C_{lim} 求解

$$\min_z \hat{P}(z) = a_0 + \sum_i b_i U_i^2 + \sum_i \frac{c_i}{T_i} \quad (7)$$

$$\text{s.t. } C_{base,k}(z) + B_k(z) \leq C_{lim}, \quad U_1 \geq U_3 + 3, U_2 \geq U_4 + 3, T_3 \geq T_1 + 100, T_4 \geq T_2 + 100, \quad z \in \Omega_k \quad (8)$$

支持域 Ω_k 取该工况训练期各控制量的实测最小—最大范围; 5 mg/Nm³ 档另允许电压上界外推至历史最大值的 1.05 倍, 该外推量在表 11 中逐条报告。相关多电场节能研究同样采用排放约束下的

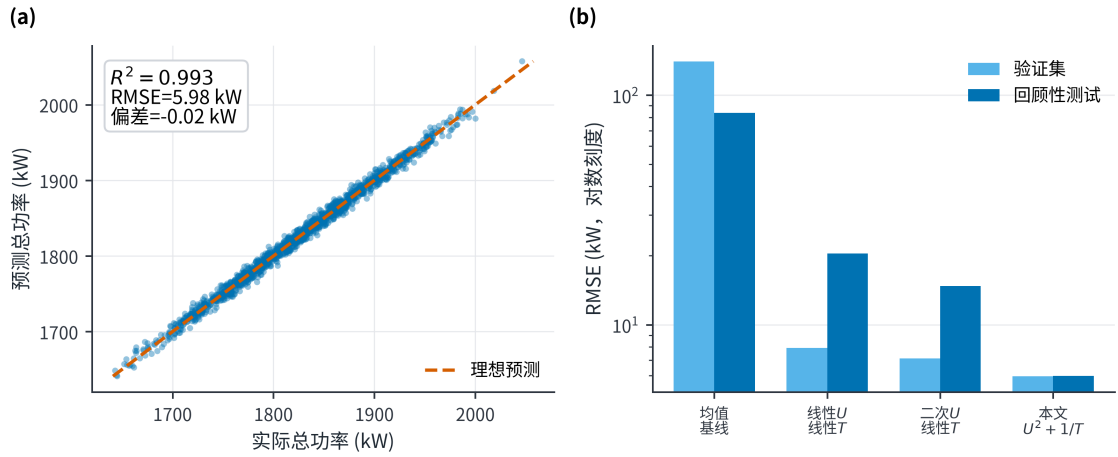


图8 功率模型预测结果

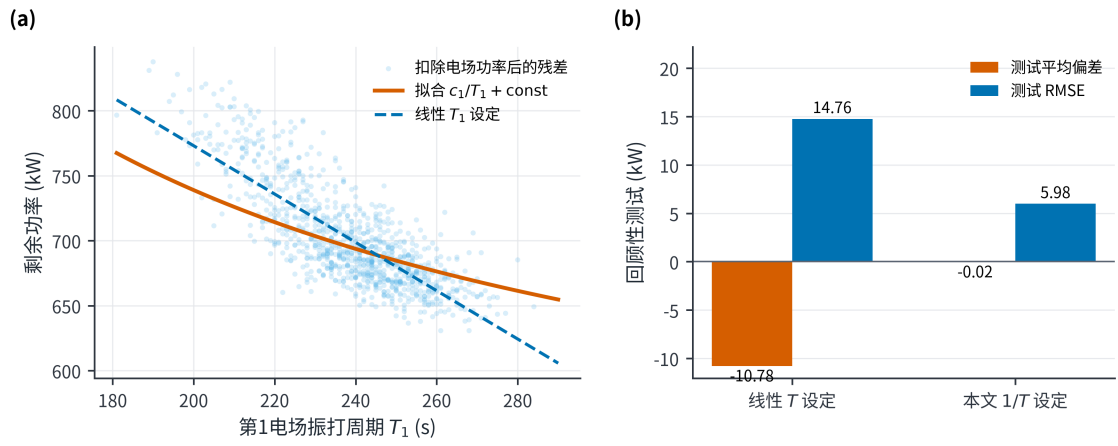


图9 振打周期的设定形式对比

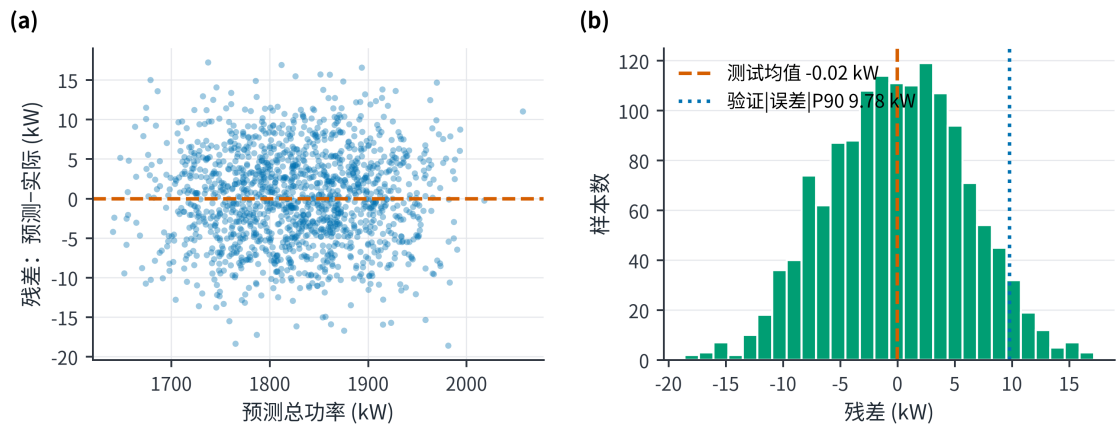


图10 回顾性测试残差诊断

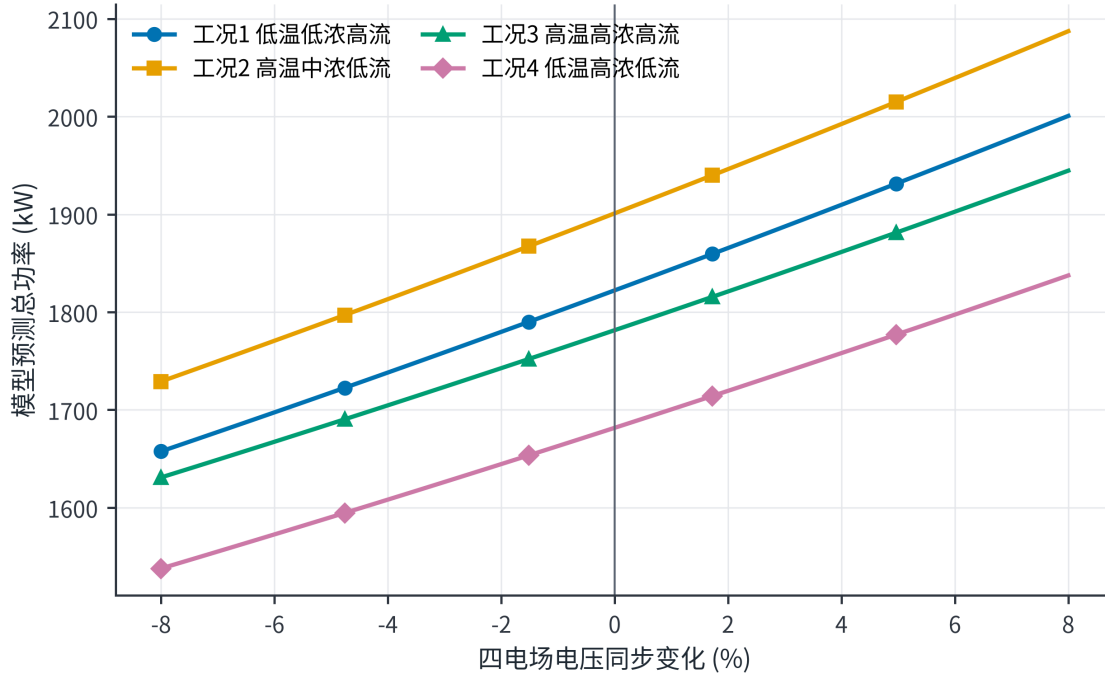


图 11 局部电压—功率响应

分场优化思想 [5,7], 但其排放模型均由现场实测标定, 与本文条件不同, 方法对照见附录 A。

关于求解方法：本问题在 U 空间中目标函数 $\sum b_i U_i^2$ 为凸, 而排放约束要求一个凸函数足够大, 属反凸约束, 故整体并非凸规划; 若改以 $v_i = U_i^2$ 为变量, 排放约束与目标转为线性, 但排序约束 $U_1 \geq U_3 + 3$ 与马氏椭圆在 v 空间不再是凸集。因此本文不宣称凸性, 而采用**多起点 SLSQP 确定性求解**: 每个工况—限值组合取 5 个结构化起点 (历史中位、上界、下界、中点、 T^* 点) 与 48 个准均匀随机起点, 并以同一可行集上 4×10^5 点的随机搜索作为独立对照 (§8.1)。这一做法取代了早期版本的百万级随机采样: 结构清楚之后, 确定性求解既更快也更可复算。

5.4 电场优先级的解析判据

在给出数值解之前, 先由模型结构导出可解释的优先级。对 $K_k = K_{0,k} \sum_i \alpha_i (U_i / U_{i,k}^{ref})^2 / \sum \alpha$ 与 $P = \sum_i b_i U_i^2$, 同时对 U_i^2 求导得**单位电功率换来的去除指数增量**

$$\left. \frac{dK}{dP} \right|_i = \frac{K_{0,k} \alpha_i}{\left(\sum_j \alpha_j \right) b_i \left(U_{i,k}^{ref} \right)^2} \quad (9)$$

由于四场的 b_i 几乎相同, 排序完全由 $\alpha_i / (U_{i,k}^{ref})^2$ 决定, 而不是由 α_i 决定。代入中心权重与各工况历史中位电压, 四类工况的判据值如表 9: 尽管第三电场的去除权重 $\alpha_3 = 0.95$ 低于前两级, 但其参考电压低约 10 kV, 平方之后使其单位功率收益在四类工况中**均居首位**。

表 9 电场优先级解析判据 ($\times 10^{-3} \text{ kW}^{-1}$)

工况	电场 1	电场 2	电场 3	电场 4	优先次序
1 低温低浓度高流量	5.768	5.504	6.668	5.573	E3 > E1 > E4 > E2
2 高温中浓度低流量	5.556	5.269	7.023	5.870	E3 > E4 > E1 > E2
3 高温高浓度高流量	6.630	6.327	8.024	6.706	E3 > E4 > E1 > E2
4 低温高浓度低流量	7.551	7.207	7.717	6.502	E3 > E1 > E2 > E4

这解释了本文早期版本“前级优先”结论的真实来源: **它由排序约束 $U_1 \geq U_3 + 3$ 导出, 而非由权重 α_i 本身**。抬升 U_3 必须同时抬升 U_1 , 两者被绑定, 故最优解中 U_1 与 U_3 同步走高; 一旦排除该

约束，性价比最高的仍是第三电场。

振打侧同理： $\partial P/\partial T_i = -c_i/T_i^2$ ，前级周期短（中位 240 s）而后级长（中位 441 s），故**延长前级周期的边际节电约为后级的 3.4 倍**。这与电压侧“前级承担主要捕集”的分工并不冲突：电压提捕集、周期省电耗，两者在不同电场上各有最优着力点。

5.5 10 mg/Nm³ 下的最优控制参数

四类工况的中心情景解如表 10（5 mg/Nm³ 行为第四问提供参数）。

表 10 四工况两档目标的最优控制参数

工况	目标/(mg/Nm³)	U_1/kV	U_2/kV	U_3/kV	U_4/kV	T_1/s	T_2/s	T_3/s	T_4/s	峰值排放/(mg/Nm³)	预测总功率/kW
1	10	70.2	68.6	57.6	57.4	290	289	492	493	9.948	2045.55
2	10	70.9	69.8	59.2	58.2	280	278	494	498	9.962	2104.30
3	10	68.0	60.3	56.3	57.3	275	280	506	510	9.971	1924.67
4	10	70.5	61.8	57.2	42.9	285	283	495	498	9.991	1851.15
1	5	73.7	71.6	60.5	60.3	290	289	492	493	4.993	2182.25
2	5	74.4	72.9	62.2	61.1	280	278	494	498	4.998	2245.51
3	5	71.4	63.4	59.1	60.4	275	280	506	510	4.991	2056.06
4	5	74.0	66.4	60.1	42.9	285	283	495	498	4.988	1973.10

第二问的答案：10 mg/Nm³ 下四工况最低总功率为 1851.15~2104.30 kW，按训练期工况占比加权为 **1973.0 kW**，折合日电耗 47.35 MWh/d。作为对照，历史实际加权功率为 1783.2 kW（日电耗 42.80 MWh/d），故满足 10 mg/Nm³ 意味着**功率上升 10.6%、日电耗增加 4.55 MWh**。这一方向是必然的：历史排放约 50 mg/Nm³，把出口浓度压到五分之一必须提高捕集强度，而捕集强度的唯一在线杠杆是电压，电压又按平方计入功率。“**节能**”只能在给定排放目标下相对其他达标方案而言，而不可能**相对历史运行而言**。

解的结构与表 9 的判据一致： T 在两档下都被推向历史区间上端（延长周期直接降低振打功率），而电压分配则按单位功率收益排序——工况 3 与 4 中 U_2 明显低于 U_1 ，工况 4 的 U_4 甚至降到历史下界 42.9 kV，因为该场在这一工况下性价比最低（表 9 末行）。可见“四场同步抬升”并非最优。

表中数值为点预测，预算时可另加 9.78 kW 验证期误差附加量。电压精度不应按 0.1 kV 执行：多起点求解在收敛起点间的功率差为 0，但控制量本身存在近似等价的方向（§8.1）。

5.6 目标点已越出历史运行经验

上表的解能否直接下发？答案是明确的“否”，理由可由数据本身量化。以各工况训练期控制量的均值 μ_k 与协方差 Σ_k 计算马氏距离平方 $d_M^2 = (z - \mu_k)^T \Sigma_k^{-1} (z - \mu_k)$ ，并与该工况历史距离平方的经验 97.5% 分位数 $q_{k,0.975}$ （依次为 20.55、18.72、16.85、19.09）比较，结果见表 11。

表 11 推荐解相对历史运行经验的外推审计

工况与限值	d_M^2	$q_{0.975}$	倍数	最高电压超出历史最大值	判定
工况 1, 10 mg	49.39	20.55	2.40×	0.0%	外推
工况 2, 10 mg	46.46	18.72	2.48×	0.0%	外推
工况 3, 10 mg	77.69	16.85	4.61×	0.0%	外推
工况 4, 10 mg	102.79	19.09	5.38×	0.0%	外推
工况 1, 5 mg	65.84	20.55	3.20×	+5.05%	外推
工况 2, 5 mg	60.31	18.72	3.22×	+5.07%	外推
工况 3, 5 mg	106.26	16.85	6.31×	+5.00%	外推
工况 4, 5 mg	133.43	19.09	6.99×	+5.07%	外推

结论有三层。

其一，最小功率解全部落在历史联合运行经验之外（2.40~6.99 倍），因此本文把 d_M^2 作为**外推风险指标报告**而不作为硬约束——否则求得的将不是“最低功率”，而是“最低功率且最像历史操作”，

二者不是同一个问题。

其二，把 $d_M^2 \leq q_{k,0.975}$ 加为硬约束时，八个工况—限值组合中有七个仍然可行 (results/mahalanobis_constrained_feasibility.csv)，代价是功率上升：10 mg/Nm³ 档四工况分别多耗 1.46%~4.38% (加权 2026.5 kW，比最小功率解高 2.7%)，5 mg/Nm³ 档三个可行工况多耗 2.22%~6.07%，仅工况 1 在 5 mg/Nm³ 下无解。这提供了一个可选的**保守变体**：若现场不接受偏离历史操作经验，可用数个百分点的额外电耗换取“每一步都落在历史做过的组合里”。本文以最小功率解为主报告结果，同时保留该变体供现场按风险偏好选择。

其三，从容量看，10 mg/Nm³ 的四个解**仍位于历史电压包络之内**（各场电压均不超过该工况历史最大值），说明该目标在既有高压电源容量下原则上可达；而 5 mg/Nm³ 需要各场电压超出历史最大值最多 5.07%，逐工况所需的最小电压裕度为 5%、4%、1%、0%。**这意味着 5 mg/Nm³ 首先是一个容量问题而非电耗问题**：若现场高压电源没有这一裕度，无论如何调度都无法达标，须通过增设电场、改造电源或前端减尘解决。

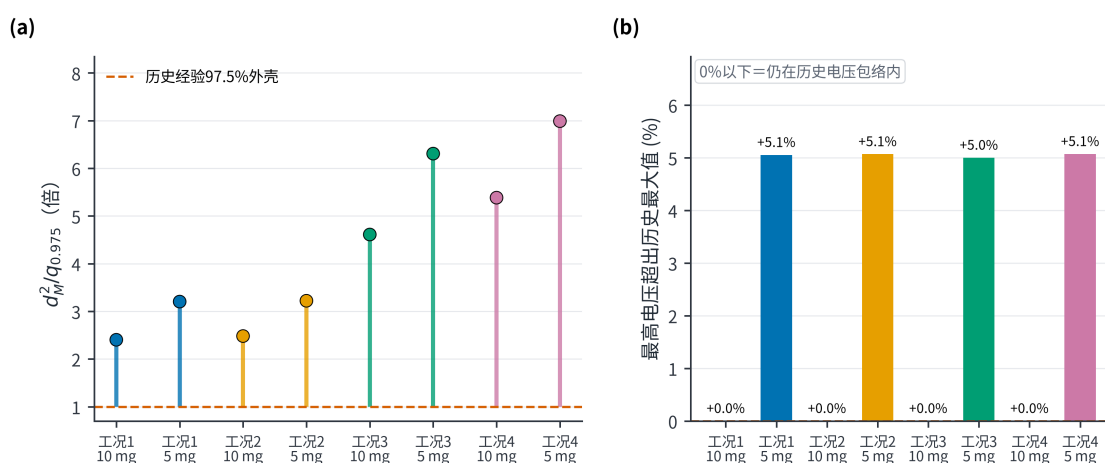


图 12 推荐解相对历史运行经验的外推审计

5.7 排放—功率权衡前沿

把情景排放限值由 10 逐档收紧至 5 mg/Nm³ 并重复求解，得到排放—功率权衡前沿（图 13）。四类工况的最优功率随限值收紧单调上升，工况加权功率由 1973.0 kW（10 mg/Nm³）升至 2105.5 kW（5 mg/Nm³），中间各档依次为 1991.3、2012.7、2046.3、2067.7 kW；所需电压裕度同步由 0% 增至 5%（每收紧 1 mg/Nm³ 约增 1 个百分点）。全部档位均高于历史实际加权功率 1783.2 kW。该图的作用是把“限值—功率”关系整体呈现：**沿前沿向左（更严）必然付出电耗，向右（更松）才降低电耗**；历史运行点位于该图右侧远端之外。

6 问题三：代表工况的控制策略比较

选取工况 1 和工况 3。二者流量都较高，但入口浓度和粉尘负荷差异明显：工况 1 平均负荷 12460 kg/h，工况 3 为 19560 kg/h，可用于比较低负荷与高负荷策略。

低负荷工况 1 采取“四场均衡抬升”：10 mg/Nm³ 下 $U = (70.2, 68.6, 57.6, 57.4)$ kV，四场电压相对各自参考值的抬升幅度接近。原因是该工况锚点 $K_0 = 6.257$ 最低，所需强度因子 $S = 1.288$ 最大，四场都必须出力，无从取舍。

高负荷工况 3 采取“择优抬升”： $U = (68.0, 60.3, 56.3, 57.3)$ kV， U_1 、 U_3 、 U_4 均接近历史最大，而 U_2 停在 60.3 kV（其历史最大为 69.3 kV）。这正是表 9 判据的直接体现——工况 3 的优先次序为

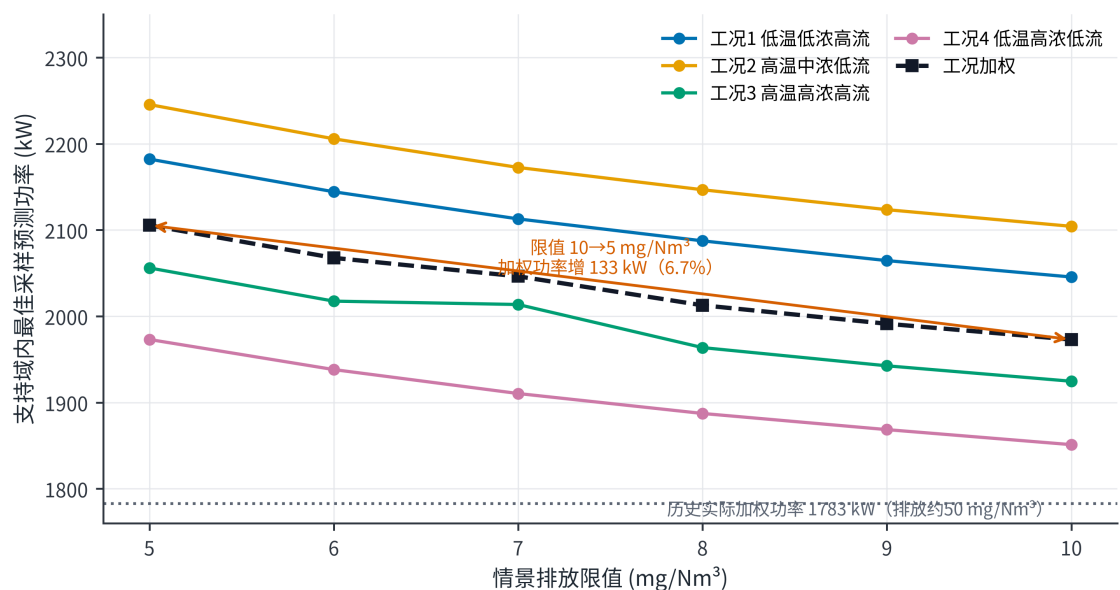


图 13 排放—功率权衡前沿

E3 > E4 > E1 > E2，第二电场性价比最低，故被留在低位。由 10 收紧至 5 mg/Nm³ 时，各场增量为 $\Delta U = (+3.4, +3.1, +2.8, +3.1)$ kV，仍保持这一格局。

因此第三问的答案不是“前级优先”这一口号，而是一条可计算的判据：

- **电压侧**：按 $\alpha_i/(b_i U_i^{ref2})$ 降序抬升，受排序约束 $U_1 \geq U_3 + 3$ 牵引时同步抬升 U_1 ；
- **振打侧**：按 c_i/T_i^2 降序延长周期，即优先延长周期较短的前级；
- **两侧协同**：电压提捕集、周期省电耗，二者在不同电场上着力，不应混为一谈。

历史数据对该判据提供了部分而非完全的支持：§4.2 的规律一（高浓度缩短各场周期）在全部时窗一致，与“高负荷下不宜为省电延长周期”相符；规律二（高浓度抬升前两级电压）只在后两天成立，故本文只把它列为参照。

为把上述比较转化为可执行判据，按排放构成与容量状态给出控制优先级决策规则，如表 12。

表 12 控制优先级决策规则

状态特征	优先动作	原因
连续排放基值接近限值，振打峰值占比较低	按 $\alpha_i/(b_i U_i^{ref2})$ 降序小步提高电压	主要矛盾是连续捕集强度不足，应优先动用性价比最高的电场
峰值增量占比较高，周期明显长于参考周期	优先缩短过长周期并实施错相振打	主要矛盾是单次积灰脱落和峰值叠加
周期过短、振打频繁，且电耗压力大	优先延长前级周期	前级 c_i/T_i^2 最大，单位延长的节电最多
高粉尘负荷且基值和峰值均高	先错相、再调整周期，最后小步提升电压	先控制峰值，再补充连续捕集能力
某电场电压已达历史最大值	转向次优电场；若各场均达上限则停止优化	数学可行不等于电源容量可行
入口温度进入 140~160 °C 高比电阻带	退出优化，按现场反电晕处置预案运行	该带未被本文工况覆盖，模型外推无效
超出历史支持域或出现火花、联锁风险	停止优化，回退现场安全设定	数学可行不等于设备安全

在线实施采用 **TARGET-10**、**TARGET-5** 与 **FALLBACK** 三态。排放目标由上级指令或管理规定选择，不设自动切换阈值——本文灰箱模型未经现场标定，不具备在线自动判定排放状态的精度。TARGET-10 与 TARGET-5 分别读取表 10 中对应工况、对应排放目标的控制参数。控制量按斜坡接近目标，试运行情景暂取电压不超过 0.5 kV/min、振打周期不超过 5 s/min，但现场 DCS、联锁或设

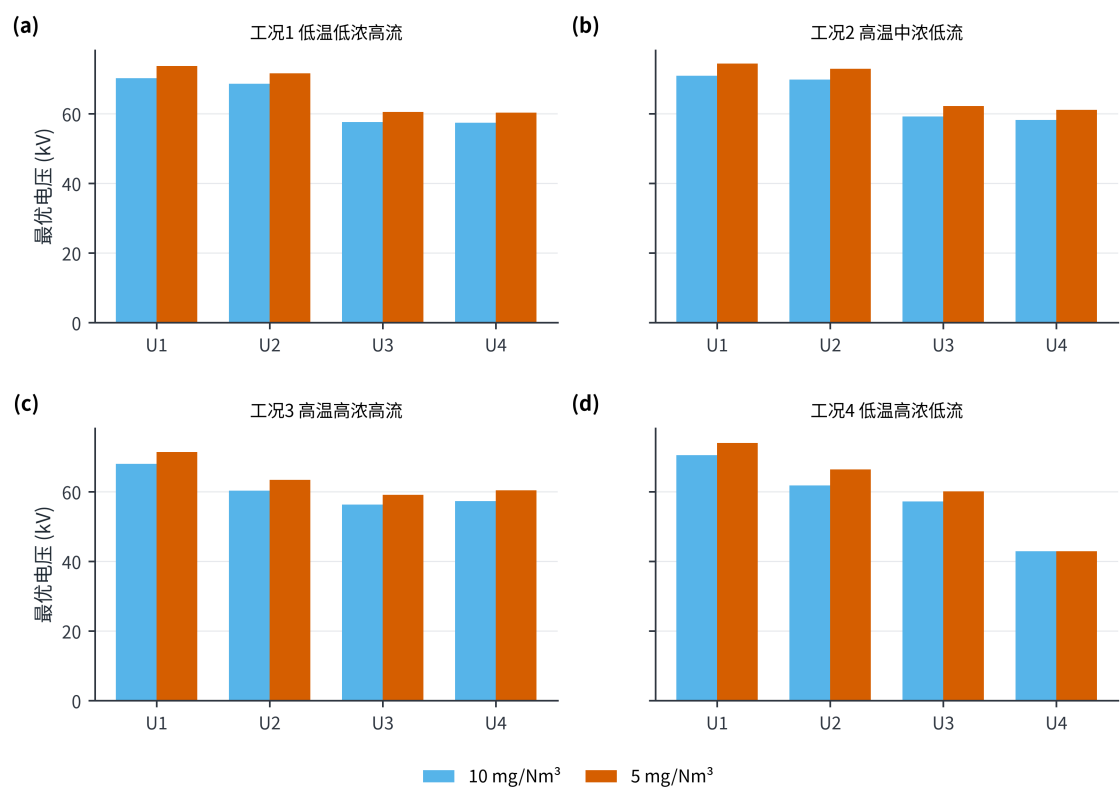


图 14 两档排放限值下的电压配置

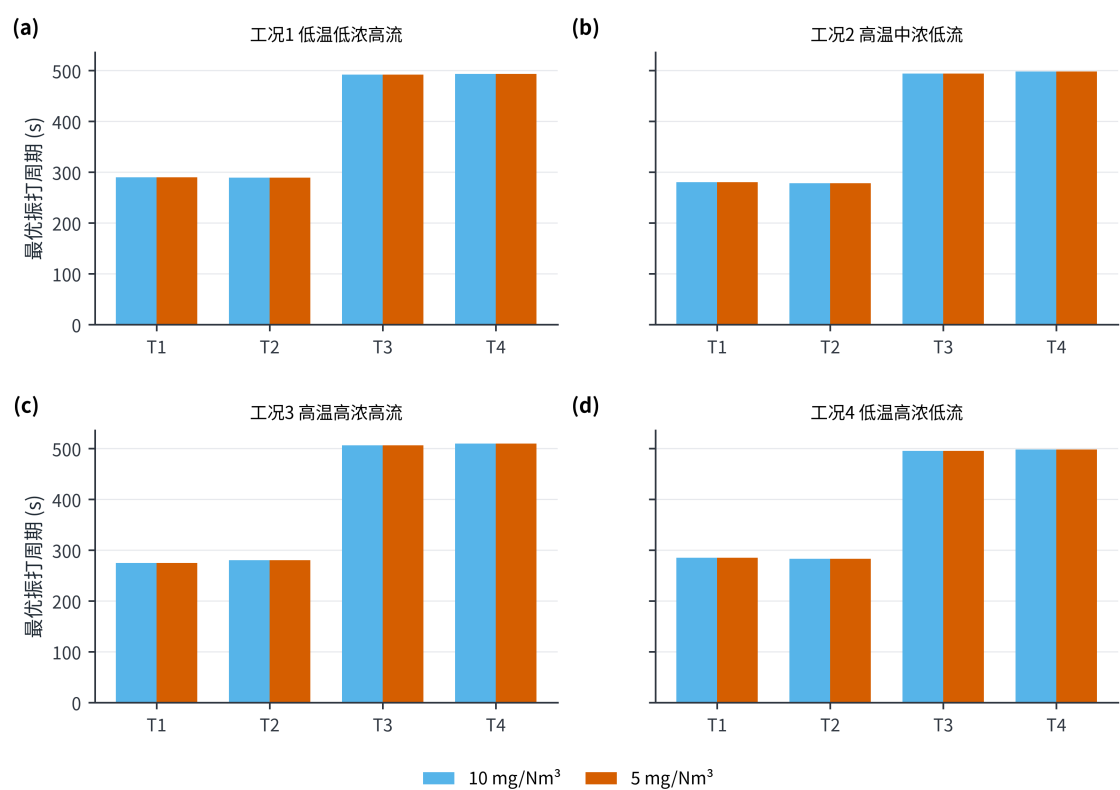


图 15 两档排放限值下的振打周期

备制造商给出的更严限制具有优先权。四场振打先按 $\rho = 1$ 同相上界核验，再以 $\rho = 0.80$ 和 0.65 评估错相收益。FALLBACK 在传感器失效、火花率越限、工况超出支持域或联锁触发时启用。

7 问题四：10 收紧至 5 mg/Nm³ 的功率代价

7.1 闭式解

第四问存在闭式解，且它比数值解更有解释力。在 10 mg/Nm³ 的最优点上，设该点的电场功率为 P_{field} 、去除指数为 K_{10} 。把四场电压同比缩放 f 倍，则 $K = K_{10}f^p$ 、 $P_{field} \rightarrow P_{field}f^2$ 。要求排放减半（考虑峰值项后所需增量为 $\Delta K = \ln \frac{10-B}{5-B}$ ），解得 $f = (1 + \Delta K/K_{10})^{1/p}$ ，故

$$\Delta P = P_{field} \left[\left(1 + \frac{\Delta K}{K_{10}} \right)^{2/p} - 1 \right] \approx \frac{2}{p} \cdot \frac{P_{field}}{K_{10}} \Delta K \quad (10)$$

其物理含义清晰：**限值每减半，增加的电耗等于电场功率乘以所需去除指数增量、除以当前去除指数，再乘以 $2/p$ 。**设备当前越干净（ K 越大），进一步减半的相对代价越低。

7.2 分工况结果

表 13 中心情景下 10 收紧至 5 mg/Nm³ 的分工况功率代价

工况	10 mg 功率/kW	5 mg 功率/kW	K_{10}	ΔK	闭式 ΔP /kW	数值 ΔP /kW	功率增幅
1 低温低浓度高流量	2045.55	2182.25	8.059	0.804	137.7	136.7	6.68%
2 高温中浓度低流量	2104.30	2245.51	8.338	0.832	142.4	141.2	6.71%
3 高温高浓度高流量	1924.67	2056.06	8.594	0.903	131.3	131.4	6.83%
4 低温高浓度低流量	1851.15	1973.10	8.690	0.906	123.1	122.0	6.59%

闭式解与数值解的相对差异不超过 1%，互为验证。按训练期工况占比加权，

$$\bar{P}_{10} = 1973.0 \text{ kW}, \quad \bar{P}_5 = 2105.5 \text{ kW}, \quad \Delta P/\bar{P}_{10} = 6.72\% \quad (11)$$

按相同运行时长折算，两档日电耗分别约为 47.35 与 50.53 MWh/d，收紧限值的日增电耗约 3.18 MWh/d；按年运行 8760 h 计，年增电耗约 1.16×10^3 MWh。加上 9.78 kW 验证期误差附加量后，两档加权功率分别为 1982.8 与 2115.3 kW；由于两者使用同一附加量，该比例不作为严格上界，只用于功率预算。

值得注意的是四类工况的增幅高度一致（6.59%~6.83%）。这不是巧合，而是闭式解的直接推论：增幅 $\approx (2/p)(P_{field}/P)(\Delta K/K)$ ，而四类工况的电场功率占比与当前去除指数都相近，故增幅几乎不随工况变化。

7.3 真正的权衡：振打周期能买回多少

把第四问的答案与 §4.1 的振打结构并列，可以得到本文最有操作价值的一组对照。以工况加权计，限值由 10 收紧至 5 mg/Nm³ 的代价约 132.5 kW；而把四场振打周期由历史中位延长至历史 P95 可回收 56.1 kW，延长至历史最大值可回收 109.1 kW。**也就是说，振打侧的全部可挖潜量（约 109 kW）几乎恰好抵消排放收紧一档的电耗代价（约 133 kW）的 82%。**

但这笔账不能只算一半。按 §4.5 的推导，把周期由中位延长至历史最大值（平均 +17.9%），单次振打的灰饼质量与瞬时峰值同比放大约 17.9%；在表 10 的解中峰值项已占用限值的 0.95~1.61 mg/Nm³，

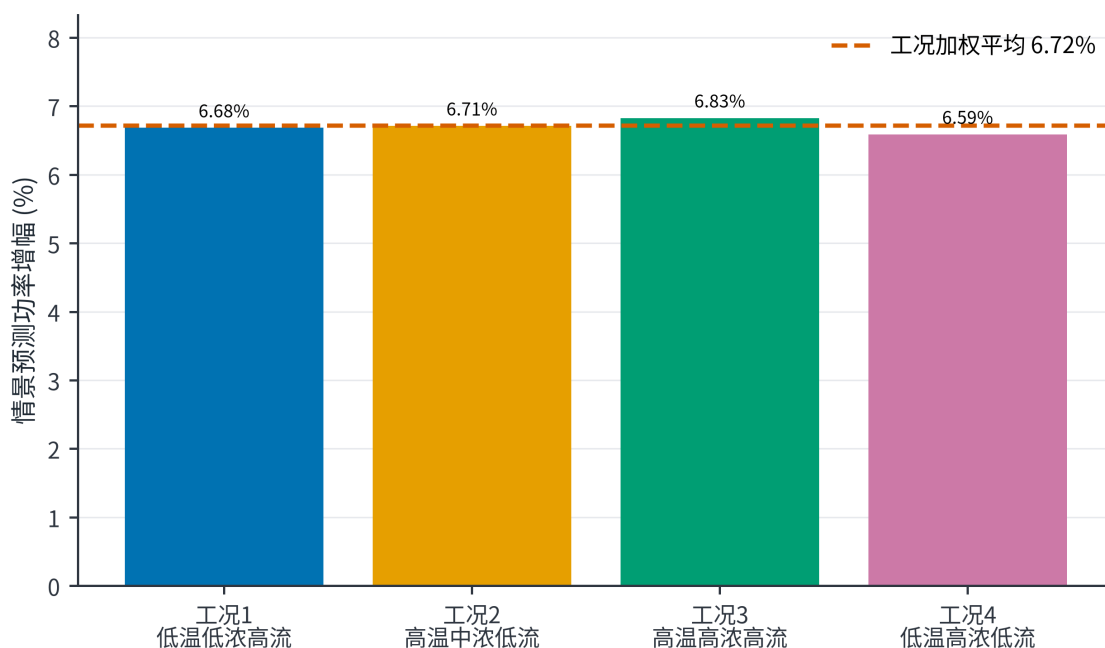


图 16 分工况功率增幅比较

即 5 mg/Nm^3 档中约五分之一至三分之一的裕度。峰值放大 17.9% 将吃掉 $0.17 \sim 0.29 \text{ mg/Nm}^3$ 的额外裕度，必须由电压侧补偿，而补偿电压又要花钱。这正是“不能为省电无限延长振打周期”的定量表述：振打周期是本设备上最便宜的节电杠杆，但它的额度有限，且以瞬时排放峰值为代价支付。

7.4 峰值约束口径的说明

中心情景解中， 10 mg/Nm^3 方案的连续基值为 $8.38 \sim 9.00 \text{ mg/Nm}^3$ 、振打峰值增量为 $0.95 \sim 1.61 \text{ mg/Nm}^3$ ； 5 mg/Nm^3 方案的连续基值降至 $3.38 \sim 4.05 \text{ mg/Nm}^3$ 。因此不能只令平均或基值低于限值，否则振打时刻仍可能超限。

需说明的是，本文把**瞬时峰值**直接约束到限值，而现行排放标准的颗粒物达标通常以小时均值考核。按 §4.5 的推导，一次持续数秒至数十秒的再飞扬峰值对小时均值的贡献正比于 L 而与周期无关，因此本文口径系统性偏保守，会高估达标所需功率。若改用“小时均值达标+峰值软惩罚”的口径，功率增幅会低于本文报告值。在缺少事件级振打数据时，本文选择保守口径，但不应把它当作唯一合理解释。

8 模型检验、敏感性与稳健性

8.1 求解质量

八个工况—限值组合中，多起点 SLSQP 的收敛起点数为 47~53，**所有收敛起点给出的最优功率一致到 10^{-7} kW 以内**；同一可行集上 4×10^5 点的均匀随机搜索在全部组合上都没有找到更优解，其最优值高出 0.97%~5.17%，其中工况 1 的 10 mg/Nm^3 组合甚至一个可行样本都没有采到——可行域是被排放约束与排序约束夹出的窄带，随机采样效率极低。这一对照说明：结构清楚之后，确定性求解优于大规模随机搜索。但仍须说明，非凸问题的多起点一致性是**强证据而非数学证明**，本文因此使用“最优解”而不宣称“全局最优”。

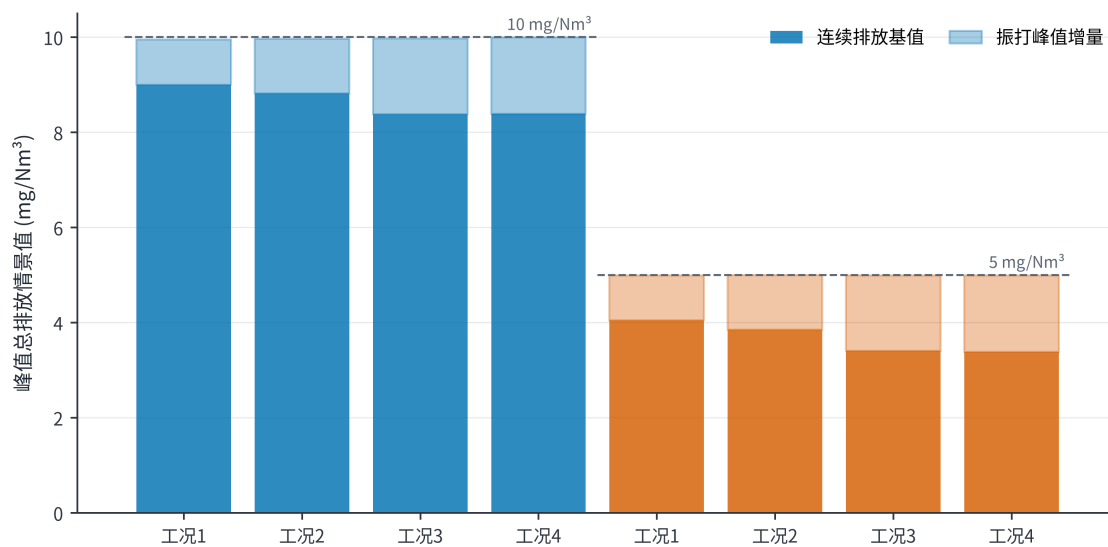


图 17 排放约束分解

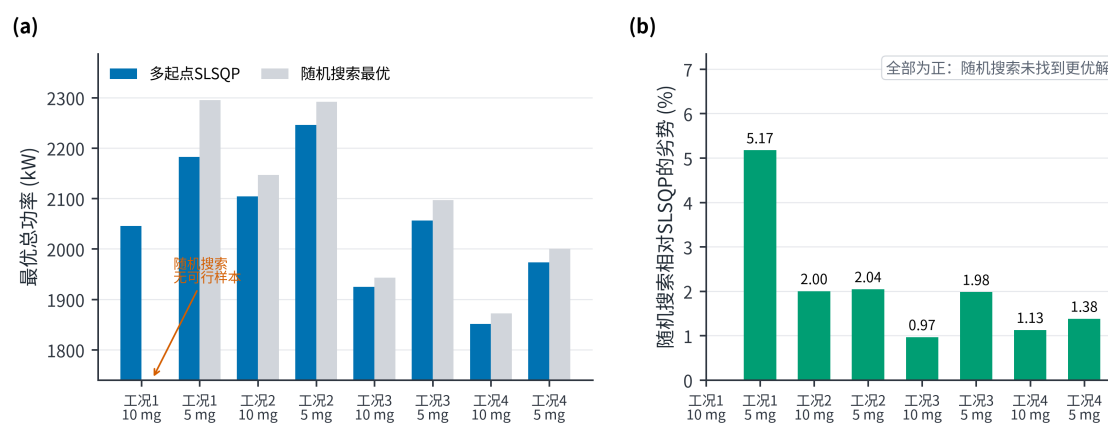


图 18 求解方法对照

8.2 先验敏感性

功率增幅结论对灰箱先验的依赖集中在单一标量 p 上。由 §7.1 的闭式解，增幅严格正比于 $2/p$ ；表 14 给出参考表。

表 14 功率增幅对幂次 p 的依赖

p	物理含义	10→5 mg/Nm ³ 加权增幅	5 mg/Nm ³ 在 5% 电压裕度内是否可达
1.0	去除指数正比于电压	14.2%	否，需更大电压裕度
1.5	中间情形	9.2%	否，需更大电压裕度
2.0	Deutsch 迁移速度形式（中心）	6.7%	是
2.5	强电压响应	5.3%	是
3.0	极强电压响应	4.4%	是

p 是本文结论中唯一需要现场标定的标量，且一次电压阶跃试验即可确定：在固定工况下把四场电压同比抬升数个百分点，记录未封顶出口浓度的变化，由 $\ln C_{out}$ 对 $\ln U$ 回归的斜率即得 $-K_0 p_0$ 。这比给出一个宽区间对现场更有用。

表中增幅为闭式解值；对 $p \geq 2$ 的三档另作数值重解校验（图 20 叉号），与闭式值相差不超过 0.4 个百分点。

除 p 以外，本文还扫描了两组情景。其一是尺度情景： $p \in \{1.5, 2.0, 2.5\} \times$ 峰值尺度 $b_0 \in \{0.9, 1.2, 1.5\} \times$ 安全裕度 $s \in \{0, 0.08, 0.15\}$ 共 27 组，其中 15 组在 5% 电压裕度内全工况可行——不可行的 12 组全部来自 $p = 1.5$ ，再次说明 p 决定的首先是可达性而不只是代价；可行组的加权增幅为 3.99%~7.04%、中位数 5.85%。其二是结构情景：场权重形状（3 种） $\times$ 峰值指数 $\gamma \in \{1.00, 1.30, 1.60\} \times$ 相位尺度 $\rho \in \{0.65, 0.80, 1.00\} \times$ 参考周期负荷指数（3 种）共 81 组，全部可行，加权增幅为 5.13%~7.55%、中位数 6.40%。峰值指数在推导值 1.00 附近变化对增幅影响很小，这与 §4.5 的推导一致：峰值项进入的是限值裕度而非主导项。所有区间均为先验网格上的情景范围，不是统计置信区间。

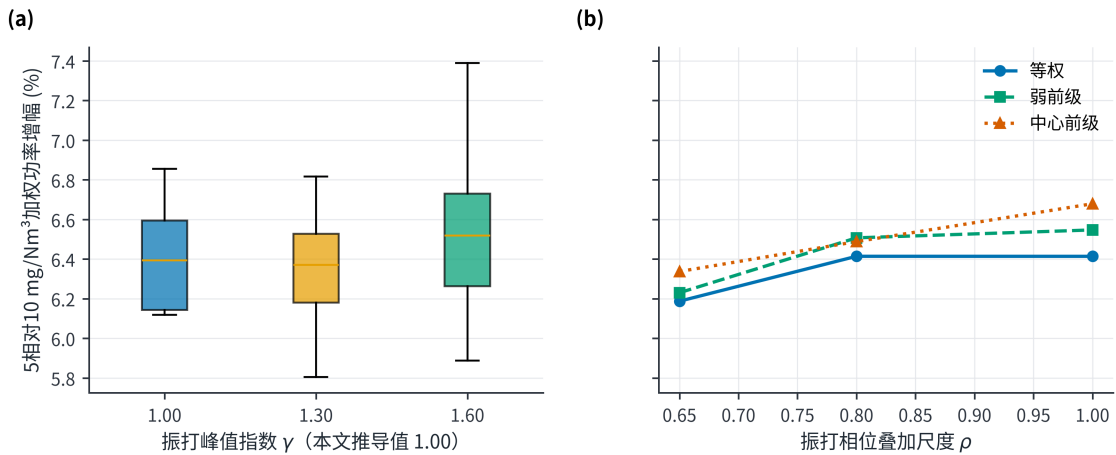


图 19 结构情景下的功率增幅分布

8.3 稳健性结论

稳健性证据分四层：轮廓系数支持四工况分层；设定比较与时间外推支持功率模型选择，且 $1/T$ 设定在未参与拟合的测试日上表现最好，是本文最强的实证证据；多起点一致性与随机搜索对照支持数值解质量；闭式解与数值解相互验证，结构情景与场权重消融检验灰箱结论对建模选择的依赖。仍须强调：排放层的全部数值是条件式推演，验证期误差附加量与情景范围都不能把灰箱模型变成实证模型。

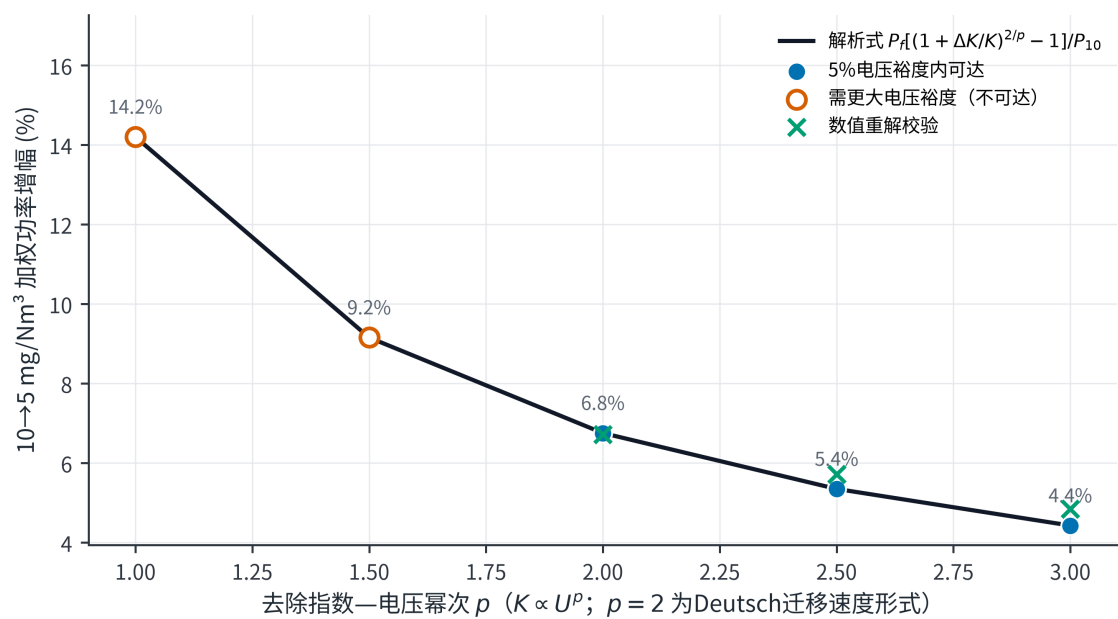


图 20 功率增幅对幂次 p 的依赖

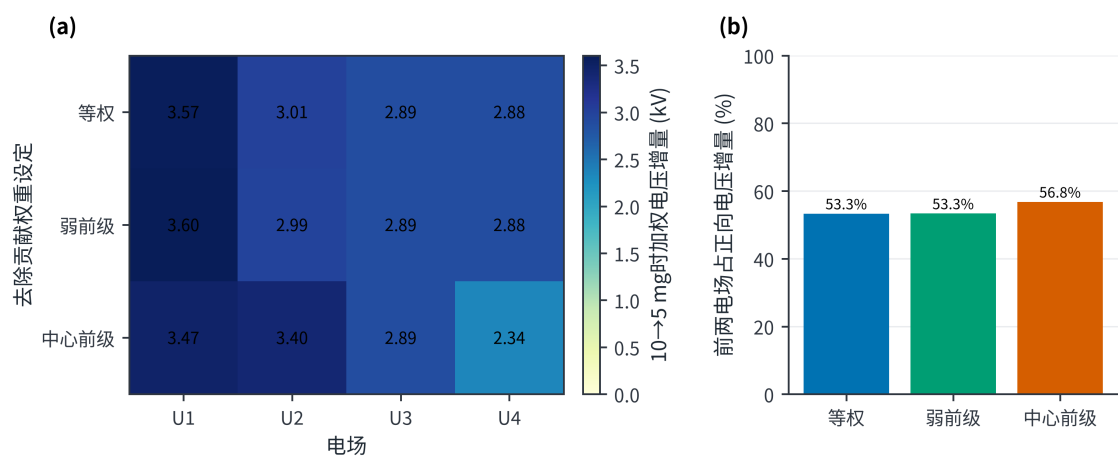


图 21 电场贡献权重消融结果

9 模型评价、实施方案与局限

9.1 模型优点

第一，设置可识别性门禁并逐列给出结论，既不用复杂算法拟合封顶噪声，也不因一列不可用而放弃其余 13 列的信息。第二，功率模型采用物理设定而非通用多项式，在未参与拟合的测试日上取得 $R^2 = 0.993$ 、偏差 -0.02 kW ，并据此发现振打占电耗近四成这一可操作结论。第三，排放锚点直接取记录值，不引入未经证实的量纲假设，使第二问回答的是题目实际提出的问题。第四，优先级由解析判据给出而非由单次求解的现象归纳。第五，第四问给出闭式解，把不确定性收敛到单一可标定标量 p 。第六，把“目标点已越出历史运行经验多少”作为结论的一部分报告，并给出留在历史经验内的保守变体及其电耗代价，而不是掩盖。

9.2 模型局限

最主要的局限是排放层的全部系数 $(p, \alpha_i, \beta_i, b_0, \gamma, \rho, s)$ 均未由附件识别，只能由机理与文献设定；表 14 的区间是先验网格上的情景范围，不是统计置信区间。历史运行包络不是设备安全边界， 5 mg/Nm^3 档的 5% 电压外推更须经高压电源容量与绝缘裕度核算。模型没有二次电流、分场功率、火花率、极板积灰状态和联锁信息；分钟级数据不足以观察逐次振打峰值，故相位叠加尺度 ρ 无法标定。四类工况的温度中心均未覆盖 $140 \sim 160^\circ\text{C}$ 高比电阻带，该带的结论无效。回顾性测试数据曾被前期处理流程使用，不能视为完全未触碰的盲测。因此，本文控制参数只能作为试验起点，**不得直接下发生产设备**。

9.3 上线与补采方案

建议按四阶段部署。第一阶段只做影子计算，验证工况识别与 TARGET-10/TARGET-5 参数表的正确加载；同步完成高压电源容量核算，确认 5 mg/Nm^3 所需的电压裕度是否存在。第二阶段在每类工况内执行单因素小步阶跃，重点是**标定幂次 p** (§8.2)，并由现场确定真实电压和周期斜坡上限。第三阶段用 $1 \sim 5 \text{ s}$ 未封顶出口浓度、实际振打事件、二次电流、火花率和分场功率重新标定灰箱参数及相位尺度 ρ ，并单独补采 $140 \sim 160^\circ\text{C}$ 高比电阻带的的数据。第四阶段冻结模型，在全新日期盲测，通过后才允许闭环。任何超支持域、传感器失效或联锁事件都触发 FALLBACK。EPA 技术资料和相关实验均强调气流、收尘面积及再飞扬对性能评价的重要性 [2-3,6]，故补采应优先补齐这些状态链。

10 结论

- 问题一：分层作答。**可识别层：总功率由控制量近乎确定， $P = 73.1 + 0.0850 \sum U_i^2 + 5.39 \times 10^4 \sum T_i^{-1}$ （测试 $R^2 = 0.993$ ），其中振打环节占中位工况总功率的 38.8%，工况变量对总功率无增量解释力；历史操作规律中“高浓度缩短各场振打周期”在四个时窗一致，而“高浓度抬升前级电压”仅在后两日成立；附件含两段持续异常工况，其中 05-06 的 158.2°C 事件落在粉尘比电阻高值区。不可识别层：出口浓度记录在 50 mg/Nm^3 附近截顶，与全部 12 个变量的相关系数绝对值均小于 0.017，无法实证识别控制—排放响应。振打峰值由积灰量守恒推得：单次峰值随周期近似线性放大，而对小时均值的贡献与周期无关。
- 问题二：4 类工况与 10 mg/Nm^3 方案。**基于入口三变量识别出 4 类典型工况（轮廓系数 0.4084）。以记录值 50 mg/Nm^3 为锚点，四工况满足 10 mg/Nm^3 的最低功率为 $1851.15 \sim 2104.30 \text{ kW}$ ，加权 1973.0 kW ，日电耗 47.35 MWh/d ；相对历史实际加权功率 1783.2 kW 上升 10.6%。收紧排放必然增电耗，“等排放节能”不存在。该方案仍位于历史电压包络之内，但马氏距离已达经验 97.5% 外壳的 2.4~5.4 倍；若强制留在该外壳内，四工况仍可行，加功率率升至 2026.5 kW （多耗 2.7%）。

3. **问题三：解析优先级。**单位电功率的去除收益为 $K_0\alpha_i/(\sum \alpha \cdot b_i U_i^{ref^2})$ 。因第三电场参考电压最低，其性价比在四类工况中均居首；“前级优先”实际由排序约束 $U_1 \geq U_3 + 3$ 导出。振打侧按 c_i/T_i^2 排序，优先延长前级周期。低负荷工况须四场均衡抬升，高负荷工况可择优抬升并让性价比最低的电场留在低位。
4. **问题四：功率代价与容量约束。**限值由 10 收紧至 5 mg/Nm³ 时，闭式解 $\Delta P = P_{field} \ln[(10 - B)/(5 - B)]/K_{10}$ 给出 123.1~142.4 kW，与数值解相差不足 1%；加权功率由 1973.0 升至 2105.5 kW，增幅 6.72%，日电耗由 47.35 增至 50.53 MWh/d。增幅正比于 $2/p$ ， p 是唯一需现场标定的标量。更关键的约束不是电耗而是容量：5 mg/Nm³ 需各场电压超出历史最大值最多 5.07%，逐工况最小裕度为 5%、4%、1%、0%；若电源无此裕度，则须通过增设电场、改造电源或前端减尘解决，而非依靠调度。上述排放数值均为情景推演，必须在补采未封顶排放、振打事件与电气状态后重新标定。

参考文献

- [1] 中华人民共和国生态环境部. 水泥工业大气污染物排放标准：GB 4915—2013（含 2025 年修改单）[EB/OL]. (2013-12-27)[2026-08-06]. https://www.mee.gov.cn/ywgz/fgbz/bz/bzwb/dqhjbh/dqgdwrywrf/bz/201312/t20131227_265765.htm.
- [2] OGLESBY S Jr, NICHOLS G B. *A Manual of Electrostatic Precipitator Technology: Part I—Fundamentals*[R/OL]. Birmingham: Southern Research Institute, 1970[2026-08-06]. <https://nepis.epa.gov/Exe/ZyPURL.cgi?Dockey=9101FJFA.TXT>.
- [3] SZABO M F, SHAH Y M, SCHLIESSER S P. *Inspection Manual for Evaluation of Electrostatic Precipitator Performance*[R/OL]. Washington, DC: U.S. Environmental Protection Agency, 1981[2026-08-06]. <https://nepis.epa.gov/Exe/ZyPURL.cgi?Dockey=9400034C.TXT>.
- [4] MIZUNO A. Electrostatic precipitation[J]. IEEE Transactions on Dielectrics and Electrical Insulation, 2000, 7(5): 615-624. DOI: 10.1109/94.879357.
- [5] LI Z, SHAO C, AN Y, et al. Energy-saving optimal control for a factual electrostatic precipitator with multiple electric-field stages based on GA[J]. Journal of Process Control, 2013, 23(8): 1041-1051. DOI: 10.1016/j.jprocont.2013.06.007.
- [6] NAVARRETE B, VILCHES L F, RODRIGUEZ-GALAN M, et al. A pilot scale study of the rapping reentrainment and fouling in electrostatic precipitation[J]. Environmental Progress & Sustainable Energy, 2015, 34(1): 7-14. DOI: 10.1002/ep.11934.
- [7] ZHENG C, ZHAO Z, GUO Y, et al. A real-time optimization method for economic and effective operation of electrostatic precipitators[J]. Journal of the Air & Waste Management Association, 2020, 70(7): 708-720. DOI: 10.1080/10962247.2020.1767227.
- [8] FU Y, SU Z. Deep neural network based concentration prediction model of dry electrostatic precipitator with a modified PID optimizer[J]. Journal of Physics: Conference Series, 2022, 2189: 012018. DOI: 10.1088/1742-6596/2189/1/012018.
- [9] MACQUEEN J. Some methods for classification and analysis of multivariate observations[C]//Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability. Berkeley: University of California Press, 1967, 1: 281-297.
- [10] ROUSSEUW P J. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis[J]. Journal of Computational and Applied Mathematics, 1987, 20: 53-65. DOI: 10.1016/0377-0427(87)90125-7.
- [11] HOERL A E, KENNARD R W. Ridge regression: Biased estimation for nonorthogonal problems[J]. Technometrics, 1970, 12(1): 55-67. DOI: 10.1080/00401706.1970.10488634.

- [12] WHITE H J. *Industrial Electrostatic Precipitation*[M]. Reading, MA: Addison-Wesley, 1963.
- [13] PARKER K R. *Electrical Operation of Electrostatic Precipitators*[M]. London: Institution of Electrical Engineers, 2003.

附录 A 与既有多电场节能优化研究的对照

本文与文献 [5]、[7] 同属“排放约束下的多电场分场优化”，但标定路径与可行域定义不同，对照如表 A1。

表 A1 与既有多电场节能优化研究的方法对照

维度	文献 [5] (GA 优化)	文献 [7] (实时优化)	本文
排放模型标定	由现场实测出口浓度回归	由在线仪表实时校正	出口浓度不可识别，改以记录值为锚点的 Deutsch 灰箱，全程标注为推演
功率模型	分场电源功率实测	分场功率与二次电流	仅有总功率，按 $U^2 + 1/T$ 物理设定辨识，测试 $R^2 = 0.993$
振打处理	未纳入优化	作为固定运行参数	纳入决策变量，并由积灰量守恒推导峰值标度
求解方法	遗传算法	滚动实时优化	多起点 SLSQP + 解析优先级判据 + 随机搜索对照
可行域	设备额定范围	在线安全约束	历史运行包络，越界量作为结论显式报告

三者的共同前提是排放模型可由现场数据标定；本文在缺少该条件时的处理方式，是把不可标定部分收敛到单一幂次 p 并给出标定方案 (§8.2)，这是本文与上述工作的主要差别。

附录 B 以出口浓度为直接目标的对照建模

为便于与其他方案交叉比较，本文给出以 C_{out_mgNm3} 为直接目标的建模结果：按时间划分训练/验证，(1) 以“是否等于 50”为标签的封顶分类，验证 $AUC=0.5098$ ；(2) 以未封顶子样本为目标的回归，验证 $R^2 = -0.0190$ ；(3) 以全部记录值为目标的回归，验证 $R^2 = -0.0219$ 。两个负 R^2 说明模型不如常数预测。任何在该列上报告高 R^2 的模型，其性能几乎必然来自随机划分导致的时间泄漏，或来自对 5491 条封顶常数的复现。

附录 C 结果适用性声明

本文功率模型、工况划分、历史操作规律、异常工况识别及数据质量统计直接来源于附件，可由 src/程序复算。排放灰箱系数、振打峰值系数以及由此得到的 5/10 mg/Nm^3 控制参数与功率增幅均属条件式情景推演。若数据提供方提供新的未封顶排放数据，或现场标定得到与 $p = 2$ 不同的幂次，则应按 §8.2 重新计算，不得沿用本文情景数值。5 mg/Nm^3 档的电压外推须先经高压电源容量与绝缘裕度核算。

附录 D 支撑材料与 AI 工具使用声明

求解流程：

- 输入：分钟级数据、排放限值 C_{lim}
1. 按时间划分训练/验证/回顾性测试；
 2. 逐列可识别性审计；提取历史操作规律与异常工况；
 3. 在训练期对 Temp、 C_{in} 、 Q 标准化，K-means 取 $k=4$ ；
 4. 比较四种功率设定，选定 $P = a_0 + \sum b_i U_i^2 + \sum c_i / T_i$ ；
 5. 以记录值 50 mg/Nm^3 为锚点建立灰箱排放层；由积灰量守恒定 $\gamma=1$ ；
 6. 对每个工况与限值：

- 6.1 取该工况控制量的历史最小—最大范围的支持域；
 - 6.2 多起点SLSQP求解（5结构起点+48随机起点）；
 - 6.3 舍入至0.1 kV / 1 s 后按解析优先级修复可行性；
 - 6.4 以 4×10^5 点随机搜索独立对照；
 7. 报告马氏距离外推审计与所需电压裕度；
 8. 扫描27组尺度情景与81组结构情景，给出幂次 p 参考表；
- 输出：控制参数、功率代价、外推审计、敏感性、图表与运行摘要。

源程序清单： src/scenario_model.py（功率模型、灰箱排放情景、支持域优化与敏感性）、src/paper_figures.py（21 幅正文图与图注追踪表）、src/plot_utils.py（统一绘图样式）、src/integrity_check.py（确定性核验）、src/compute_frontier.py（排放—功率前沿）、src/build_paper_pdf.py（提交稿装配）。生成本文全部结果与图表的完整可运行源程序见附录 E，与电子支撑材料 src/ 目录逐字一致。

支撑材料清单： data/处理数据包；results/下功率基线、滚动验证、最优控制、外推审计、电压裕度需求、求解对照、结构情景、场权重消融、幂次参考表、动态控制表和运行摘要；figures_paper/下21 幅正文图（20 幅 PNG/PDF 由绘图程序生成，图 1 为 TikZ 源文件独立编译）及 figure_manifest.csv；integrity/下确定性核验结果。

AI 工具使用声明（提交前由参赛队核验确认）：本稿坚持参赛队人工主导。题意分析、模型构建、公式推导、参数设定、程序审阅、结果复算和论文定稿由参赛队承担；AI 编码助手仅用于代码排错、图表版式检查、文字校对与格式核验。AI 输出均经人工筛选和复核，未替代现场数据，也未把不可识别的排放关系改写为实证结论。

附录 E 完整源程序

本附录给出生成本文全部结果与图表的完整可运行源程序，与电子支撑材料 src/ 目录逐字一致。按依赖顺序排列：情景模型与优化、排放—功率前沿、绘图样式与图集生成、确定性核验、提交稿装配。

E.1 scenario_model.py

```
1  #!/usr/bin/env python3
2  """Reproducible scenario model for the cement ESP problem.
3
4  Two layers are kept strictly separate:
5
6  * the **power layer** is identified from the supplied data. Total power is a
7    near-deterministic function of the control settings alone, with the physically
8    motivated form  $P = a_0 + \sum_i b_i U_i^2 + \sum_i c_i / T_i$  (field power
9    proportional to voltage squared, rapping power proportional to rapping
10   frequency  $60/T_i$ );
11  * the **emission layer** is an explicitly labelled engineering scenario,
12    because the recorded outlet concentration is capped near 50 mg/Nm3 and
13    carries no learnable signal.
14
15  The historical emission anchor is the *recorded* outlet concentration; no
16  decimal-point rescaling is applied. The control problem solved here is
17  therefore the path from the historical ~50 mg/Nm3 level down to the 10 and
18  5 mg/Nm3 targets set by the problem statement.
19  """
20
21  from __future__ import annotations
22
23  import json
24  import math
25  from pathlib import Path
26
27  import numpy as np
28  import pandas as pd
29  from scipy.optimize import minimize
30
31
32  SEED = 20260805
33  ROOT = Path(__file__).resolve().parents[1]
34  DATA_FILE = ROOT / "data" / "full_timeseries_with_flags.csv"
35  CLUSTER_EVAL_FILE = ROOT / "data" / "condition_cluster_evaluation.csv"
36  OUT_DIR = ROOT / "results"
37
38  U_COLS = [f"U{i}_kV" for i in range(1, 5)]
39  T_COLS = [f"T{i}_s" for i in range(1, 5)]
40  COND_COLS = ["Temp_C", "C_in_gNm3", "Q_Nm3h"]
41
42  # ---- central scenario constants (engineering priors, not fitted to C_out) ----
43  ALPHA_CENTRAL = np.array([1.15, 1.10, 0.95, 0.80]) # per-field removal weights
44  BETA_CENTRAL = np.array([0.30, 0.27, 0.20, 0.17]) # rapping-period deviation loss
45  PEAK_SHARES = np.array([0.30, 0.30, 0.22, 0.18]) # per-field share of the peak proxy
46  P_EXPONENT_CENTRAL = 2.0 #  $K \sim U^p$ ;  $p = 2$  is the Deutsch drift-velocity form
47  PEAK_EXPONENT_CENTRAL = 1.0 # derived: cake mass per rap  $\sim L \cdot T \Rightarrow \text{peak} \sim T^1$ 
48  PEAK_SCALE_CENTRAL = 1.2
49  PHASE_SCALE_CENTRAL = 1.0
50  SAFETY_INDEX_CENTRAL = 0.08
51  # 5 mg/Nm3 needs voltage above every field's historical maximum; the margin is
52  # an explicitly declared extrapolation of the historical envelope, not a fact.
```

```

53 HEADROOM_10 = 0.00
54 HEADROOM_5 = 0.05
55
56 MULTISTART = 48
57 SWEEP_MULTISTART = 16 # sensitivity grids: fewer starts, cross-checked against the central solve
58
59
60 def _r2(y: np.ndarray, pred: np.ndarray) -> float:
61     return float(1.0 - np.sum((y - pred) ** 2) / np.sum((y - y.mean()) ** 2))
62
63
64 def _rmse(y: np.ndarray, pred: np.ndarray) -> float:
65     return float(np.sqrt(np.mean((y - pred) ** 2)))
66
67
68 # ----- power models
69
70 def voltage_polynomial(u: np.ndarray) -> np.ndarray:
71     """Equivalent to PolynomialFeatures(degree=2, include_bias=False) for 4 voltages."""
72     cols = [u[:, i] for i in range(4)]
73     for i in range(4):
74         for j in range(i, 4):
75             cols.append(u[:, i] * u[:, j])
76     return np.column_stack(cols)
77
78
79 class _ScaledOLS:
80     """Least squares with per-column scaling.
81
82     The scaling matters: ``1/T`` is of order 4e-3 while ``U^2`` is of order
83     3e3, and an unscaled ``lstsq`` truncates the rapping columns as though they
84     were rank-deficient, which makes the rapping term look irrelevant.
85     """
86
87     def __init__(self) -> None:
88         self.beta: np.ndarray | None = None
89
90     def _fit_design(self, x: np.ndarray, y: np.ndarray) -> None:
91         xx = np.column_stack([np.ones(len(x)), x])
92         scale = np.abs(xx).max(axis=0)
93         scale[scale == 0] = 1.0
94         beta, *_ = np.linalg.lstsq(xx / scale, y, rcond=None)
95         self.beta = beta / scale
96
97     def _predict_design(self, x: np.ndarray) -> np.ndarray:
98         if self.beta is None:
99             raise RuntimeError("model not fitted")
100         return self.beta[0] + x @ self.beta[1:]
101
102
103 class PhysicalPowerModel(_ScaledOLS):
104     """ $P = a_0 + \sum_i b_i U_i^2 + \sum_i c_i / T_i$  (optionally + condition terms)."""
105
106     name = "physical_voltage_square_plus_rapping_frequency"
107
108     def __init__(self, with_conditions: bool = False) -> None:
109         super().__init__()
110         self.with_conditions = with_conditions
111
112     def design(self, frame: pd.DataFrame) -> np.ndarray:
113         u = frame[U_COLS].to_numpy(float)
114         t = frame[T_COLS].to_numpy(float)

```

```

115     blocks = [u ** 2, 1.0 / t]
116     if self.with_conditions:
117         blocks.append(frame[COND_COLS].to_numpy(float))
118     return np.column_stack(blocks)
119
120 def fit(self, frame: pd.DataFrame, y: np.ndarray) -> "PhysicalPowerModel":
121     self._fit_design(self.design(frame), y)
122     return self
123
124 def predict(self, frame: pd.DataFrame) -> np.ndarray:
125     return self._predict_design(self.design(frame))
126
127 # -- fast path used inside the optimiser -----
128 @property
129 def intercept(self) -> float:
130     return float(self.beta[0])
131
132 @property
133 def b(self) -> np.ndarray:
134     return np.asarray(self.beta[1:5], float)
135
136 @property
137 def c(self) -> np.ndarray:
138     return np.asarray(self.beta[5:9], float)
139
140 def power(self, u: np.ndarray, t: np.ndarray) -> np.ndarray:
141     """Total power for raw control arrays (last axis = 4 fields)."""
142     return self.intercept + np.sum(self.b * u ** 2, axis=-1) + np.sum(self.c / t, axis=-1)
143
144 def field_power(self, u: np.ndarray) -> np.ndarray:
145     return np.sum(self.b * u ** 2, axis=-1)
146
147 def rapping_power(self, t: np.ndarray) -> np.ndarray:
148     return np.sum(self.c / t, axis=-1)
149
150
151 class RidgePowerModel:
152     """Standardised ridge baseline with the rapping period entered *linearly*.
153
154     Retained as the misspecified reference point: it is the specification the
155     first version of this paper used, and the comparison in
156     ``power_model_baselines.csv`` shows what treating ``T`` as a linear term
157     costs on the retrospective test window.
158     """
159
160     def __init__(self, alpha: float = 1e-3, quadratic: bool = True):
161         self.alpha = alpha
162         self.quadratic = quadratic
163         self.mean: np.ndarray | None = None
164         self.std: np.ndarray | None = None
165         self.beta: np.ndarray | None = None
166
167     def design(self, frame: pd.DataFrame) -> np.ndarray:
168         u = frame[U_COLS].to_numpy(float)
169         remainder = frame[T_COLS + COND_COLS].to_numpy(float)
170         voltage = voltage_polynomial(u) if self.quadratic else u
171         return np.column_stack([voltage, remainder])
172
173     def fit(self, frame: pd.DataFrame, y: np.ndarray) -> None:
174         x = self.design(frame)
175         self.mean = x.mean(axis=0)
176         self.std = x.std(axis=0)

```

```

177     self.std[self.std == 0] = 1.0
178     z = (x - self.mean) / self.std
179     zz = np.column_stack([np.ones(len(z)), z])
180     penalty = np.eye(zz.shape[1]) * math.sqrt(self.alpha)
181     penalty[0, 0] = 0.0
182     augmented_x = np.vstack([zz, penalty])
183     augmented_y = np.concatenate([y, np.zeros(zz.shape[1])])
184     self.beta = np.linalg.lstsq(augmented_x, augmented_y, rcond=None)[0]
185
186     def predict(self, frame: pd.DataFrame) -> np.ndarray:
187         if self.mean is None or self.std is None or self.beta is None:
188             raise RuntimeError("model not fitted")
189         x = self.design(frame)
190         z = (x - self.mean) / self.std
191         zz = np.column_stack([np.ones(len(z)), z])
192         return np.einsum("ij,j->i", zz, self.beta)
193
194
195     def evaluate_power_models(
196         train: pd.DataFrame,
197         validation: pd.DataFrame,
198         test: pd.DataFrame,
199     ) -> tuple[PhysicalPowerModel, dict[str, object], pd.DataFrame, pd.DataFrame]:
200         """Fit every candidate specification on the training window only."""
201         alpha_grid = (1e-4, 1e-3, 1e-2, 1e-1, 1.0, 10.0)
202         n = len(train)
203         rolling_bounds = ((0, int(0.40 * n), int(0.60 * n)),
204                          (0, int(0.60 * n), int(0.80 * n)),
205                          (0, int(0.80 * n), n))
206         rolling_rows: list[dict[str, object]] = []
207         selected: dict[str, float] = {}
208         for model_name, quadratic in (("Linear_ridge_T", False), ("quadratic_ridge_T", True)):
209             alpha_scores: list[tuple[float, float]] = []
210             for alpha in alpha_grid:
211                 fold_rmse = []
212                 for fold, (start, fit_end, eval_end) in enumerate(rolling_bounds, start=1):
213                     fit = train.iloc[start:fit_end]
214                     evaluate = train.iloc[fit_end:eval_end]
215                     candidate = RidgePowerModel(alpha=alpha, quadratic=quadratic)
216                     candidate.fit(fit, fit["P_total_kW"].to_numpy(float))
217                     pred = candidate.predict(evaluate)
218                     actual = evaluate["P_total_kW"].to_numpy(float)
219                     rmse = _rmse(actual, pred)
220                     fold_rmse.append(rmse)
221                     rolling_rows.append({
222                         "model": model_name, "alpha": alpha, "fold": fold,
223                         "fit_rows": len(fit), "evaluation_rows": len(evaluate),
224                         "r2": _r2(actual, pred), "rmse_kW": rmse,
225                         "bias_kW": float(np.mean(pred - actual)),
226                     })
227                 alpha_scores.append((float(np.mean(fold_rmse)), alpha))
228             selected[model_name] = min(alpha_scores)[1]
229
230         # the physical specification has no tuning parameter; still report it per fold
231         for fold, (start, fit_end, eval_end) in enumerate(rolling_bounds, start=1):
232             fit = train.iloc[start:fit_end]
233             evaluate = train.iloc[fit_end:eval_end]
234             candidate = PhysicalPowerModel().fit(fit, fit["P_total_kW"].to_numpy(float))
235             pred = candidate.predict(evaluate)
236             actual = evaluate["P_total_kW"].to_numpy(float)
237             rolling_rows.append({
238                 "model": "physical_invT", "alpha": np.nan, "fold": fold,

```

```

239         "fit_rows": len(fit), "evaluation_rows": len(evaluate),
240         "r2": _r2(actual, pred), "rmse_kW": _rmse(actual, pred),
241         "bias_kW": float(np.mean(pred - actual)),
242     })
243
244     y_train = train["P_total_kW"].to_numpy(float)
245     fitted: dict[str, object] = {}
246     for model_name, quadratic in (("Linear_ridge_T", False), ("quadratic_ridge_T", True)):
247         m = RidgePowerModel(alpha=selected[model_name], quadratic=quadratic)
248         m.fit(train, y_train)
249         fitted[model_name] = m
250     fitted["physical_invT"] = PhysicalPowerModel().fit(train, y_train)
251     fitted["physical_invT_plus_conditions"] = PhysicalPowerModel(with_conditions=True).fit(train, y_train)
252
253     mean_power = float(y_train.mean())
254     baseline_rows: list[dict[str, object]] = []
255     for split_name, frame in (("train", train), ("validation", validation), ("retrospective_test", test)):
256         actual = frame["P_total_kW"].to_numpy(float)
257         for model_name in ("mean_predictor", "Linear_ridge_T", "quadratic_ridge_T",
258                          "physical_invT", "physical_invT_plus_conditions"):
259             pred = (np.full(len(frame), mean_power) if model_name == "mean_predictor"
260                     else fitted[model_name].predict(frame))
261             baseline_rows.append({
262                 "model": model_name,
263                 "selected_alpha": selected.get(model_name, np.nan),
264                 "split": split_name,
265                 "r2": _r2(actual, pred),
266                 "rmse_kW": _rmse(actual, pred),
267                 "bias_kW": float(np.mean(pred - actual)),
268             })
269
270     chosen: PhysicalPowerModel = fitted["physical_invT"] # type: ignore[assignment]
271     val_actual = validation["P_total_kW"].to_numpy(float)
272     val_pred = chosen.predict(validation)
273     test_actual = test["P_total_kW"].to_numpy(float)
274     test_pred = chosen.predict(test)
275     guard = float(np.quantile(np.abs(val_pred - val_actual), 0.90))
276     metrics: dict[str, object] = {
277         "model": chosen.name,
278         "formula": "P = a0 + sum_i b_i U_i^2 + sum_i c_i / T_i",
279         "fit_scope": "train_only",
280         "intercept_kW": chosen.intercept,
281         "b_field_kW_per_kV2": chosen.b.tolist(),
282         "c_rapping_kW_s": chosen.c.tolist(),
283         "train_r2": _r2(y_train, chosen.predict(train)),
284         "validation_r2": _r2(val_actual, val_pred),
285         "validation_rmse_kW": _rmse(val_actual, val_pred),
286         "validation_bias_kW": float(np.mean(val_pred - val_actual)),
287         "validation_abs_error_q90_kW": guard,
288         "retrospective_test_r2": _r2(test_actual, test_pred),
289         "retrospective_test_rmse_kW": _rmse(test_actual, test_pred),
290         "retrospective_test_bias_kW": float(np.mean(test_pred - test_actual)),
291         "test_protocol": "retrospective_not_blind_due_to_prior_package_use",
292         "reporting_guard_rule": "point_prediction_plus_validation_absolute_error_q90",
293         "ridge_selected_alphas": selected,
294     }
295     return chosen, metrics, pd.DataFrame(rolling_rows), pd.DataFrame(baseline_rows)
296
297
298 # ----- condition layer
299
300 def condition_profiles(df: pd.DataFrame) -> pd.DataFrame:

```

```

301 train = df[df["split"] == "train"].copy()
302 overall_load = train["dust_load_kg_h"].median()
303 labels = {
304     1: "低温低浓度高流量",
305     2: "高温中浓度低流量",
306     3: "高温高浓度高流量",
307     4: "低温高浓度低流量",
308 }
309 rows: list[dict[str, float | int | str]] = []
310 for cluster, group in train.groupby("condition_cluster"):
311     row: dict[str, float | int | str] = {
312         "condition_cluster": int(cluster),
313         "condition_name": labels[int(cluster)],
314         "count": int(len(group)),
315         "share": float(len(group) / len(train)),
316         "Temp_C": float(group["Temp_C"].mean()),
317         "C_in_gNm3": float(group["C_in_gNm3"].mean()),
318         "Q_Nm3h": float(group["Q_Nm3h"].mean()),
319         "dust_load_kg_h": float(group["dust_load_kg_h"].mean()),
320         "C_out_recorded_median": float(group["C_out_mgNm3"].dropna().median()),
321         "historical_power_kW": float(group["P_total_kW"].mean()),
322     }
323     for col in U_COLS + T_COLS:
324         row[f"{col}_min"] = float(group[col].min())
325         row[f"{col}_p05"] = float(group[col].quantile(0.05))
326         row[f"{col}_median"] = float(group[col].median())
327         row[f"{col}_p95"] = float(group[col].quantile(0.95))
328         row[f"{col}_max"] = float(group[col].max())
329         row["load_ratio_to_train_median"] = float(row["dust_load_kg_h"] / overall_load)
330         row["K0_anchor"] = math.log(1000.0 * row["C_in_gNm3"] / row["C_out_recorded_median"])
331     rows.append(row)
332 return pd.DataFrame(rows).sort_values("condition_cluster").reset_index(drop=True)
333
334
335 def scenario_t_star(profile: pd.Series, global_t_median: np.ndarray,
336                    load_exponent: float = -0.18) -> np.ndarray:
337     load_ratio = float(profile["load_ratio_to_train_median"])
338     raw = global_t_median * load_ratio ** load_exponent
339     lower = np.array([profile[f"{c}_min"] for c in T_COLS], float)
340     upper = np.array([profile[f"{c}_max"] for c in T_COLS], float)
341     return np.clip(raw, lower, upper)
342
343
344 def support_statistics(train_group: pd.DataFrame) -> tuple[np.ndarray, np.ndarray, float]:
345     """Empirical control-support centre, inverse covariance and d^2 cut-off."""
346     hist = train_group[U_COLS + T_COLS].to_numpy(float)
347     mu = hist.mean(axis=0)
348     inv_cov = np.linalg.pinv(np.cov(hist, rowvar=False))
349     delta = hist - mu
350     historical_d2 = np.einsum("ij,jk,ik->i", delta, inv_cov, delta)
351     return mu, inv_cov, float(np.quantile(historical_d2, 0.975))
352
353
354 # ----- emission layer
355
356 def scenario_emission(
357     profile: pd.Series,
358     u: np.ndarray,
359     t: np.ndarray,
360     t_star: np.ndarray,
361     p_exponent: float = P_EXPONENT_CENTRAL,
362     peak_scale: float = PEAK_SCALE_CENTRAL,

```

```

363     safety_index: float = SAFETY_INDEX_CENTRAL,
364     alpha_weights: np.ndarray | None = None,
365     peak_exponent: float = PEAK_EXPONENT_CENTRAL,
366     phase_scale: float = PHASE_SCALE_CENTRAL,
367 ) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
368     """Return continuous base emission, rapping peak excess, and their sum.
369
370     ``K = K0 * S(U) - [G(T) - G(T_ref)]`` with
371     ``S(U) = sum_i alpha_i (U_i/U_ref_i)^p / sum_i alpha_i``.
372
373     At the historical anchor ``U = U_ref`` and ``T = T_ref`` this returns
374     ``K = K0 = ln(1000 C_in / C_out_recorded)`` , i.e. the model reproduces the
375     recorded operating point by construction. Every non-control argument is a
376     scenario input; none is estimated from the recorded outlet series.
377     """
378     u_ref = np.array([profile[f"{c}_median"] for c in U_COLS], float)
379     t_ref = np.array([profile[f"{c}_median"] for c in T_COLS], float)
380     cin = float(profile["C_in_gNm3"])
381     k_anchor = float(profile["K0_anchor"])
382
383     if alpha_weights is None:
384         alpha_weights = ALPHA_CENTRAL
385     alpha = np.asarray(alpha_weights, float)
386     strength = np.sum(alpha * (u / u_ref) ** p_exponent, axis=-1) / alpha.sum()
387
388     load_ratio = float(profile["load_ratio_to_train_median"])
389     beta = BETA_CENTRAL * load_ratio ** 0.5
390
391     def loss(period: np.ndarray) -> np.ndarray:
392         ratio = period / t_star
393         return np.sum(beta * (ratio + 1.0 / ratio - 2.0), axis=-1)
394
395     ref_loss = float(loss(t_ref))
396     k = k_anchor * strength - (loss(t) - ref_loss)
397     base = cin * 1000.0 * np.exp(-k + safety_index)
398
399     # Derived form: cake mass per rap ~ L * T, so the per-event peak grows
400     # linearly with the period (peak_exponent = 1) while the contribution to
401     # the hourly mean, ~ L*T*(1/T) = L, does not depend on the period at all.
402     peak = (peak_scale * phase_scale * load_ratio
403             * np.sum(PEAK_SHARES * (t / t_star) ** peak_exponent, axis=-1))
404     return base, peak, base + peak
405
406
407 def field_priority(profile: pd.Series, model: PhysicalPowerModel,
408                  alpha_weights: np.ndarray | None = None) -> np.ndarray:
409     """Removal gain per unit of electrical power for each field.
410
411     Differentiating ``K = K0 * sum_i alpha_i (U_i/U_ref_i)^2 / sum(alpha)`` and
412     ``P = sum_i b_i U_i^2`` with respect to ``U_i^2`` gives
413
414         
$$dK/dP|_i = K0 * \alpha_i / (\sum(\alpha) * b_i * U_{ref\_i}^2),$$

415
416     so the field to raise first is the one with the largest
417     ``alpha_i / U_ref_i^2`` -- *not* the largest ``alpha_i``.
418     """
419     alpha = ALPHA_CENTRAL if alpha_weights is None else np.asarray(alpha_weights, float)
420     u_ref = np.array([profile[f"{c}_median"] for c in U_COLS], float)
421     return float(profile["K0_anchor"]) * alpha / (alpha.sum() * model.b * u_ref ** 2)
422
423
424 # ----- optimisation

```



```

425
426 def _bounds(profile: pd.Series, voltage_headroom: float) -> tuple[np.ndarray, np.ndarray]:
427     lo = np.concatenate([[profile[f"{c}_min"] for c in U_COLS],
428                          [profile[f"{c}_min"] for c in T_COLS]])
429     hi = np.concatenate([[profile[f"{c}_max"] * (1.0 + voltage_headroom) for c in U_COLS],
430                          [profile[f"{c}_max"] for c in T_COLS]])
431     return lo.astype(float), hi.astype(float)
432
433
434 def _ordering(z: np.ndarray) -> np.ndarray:
435     return np.array([z[0] - z[2] - 3.0, z[1] - z[3] - 3.0,
436                     z[6] - z[4] - 100.0, z[7] - z[5] - 100.0])
437
438
439 def _round_controls(z: np.ndarray) -> np.ndarray:
440     return np.concatenate([np.round(z[:4], 1), np.round(z[4:])])
441
442
443 def solve_condition(
444     profile: pd.Series,
445     model: PhysicalPowerModel,
446     t_star: np.ndarray,
447     limit: float,
448     voltage_headroom: float = 0.0,
449     mahalanobis: tuple[np.ndarray, np.ndarray, float] | None = None,
450     multistart: int = MULTISTART,
451     seed: int = SEED,
452     **emission_kwargs,
453 ) -> dict[str, object]:
454     """Deterministic multi-start SLSQP solve of one condition/limit pair.
455
456     The feasible set is *not* convex in ``U``: the objective ``sum b_i U_i^2``
457     is convex while the emission constraint requires a convex function of
458     ``U`` to be large enough, which is a reverse-convex constraint. A dense
459     multi-start is therefore used, and ``verify_with_random_search`` re-checks
460     the result against a large random sample of the same feasible set.
461     """
462     lo, hi = _bounds(profile, voltage_headroom)
463
464     def objective(z: np.ndarray) -> float:
465         return float(model.power(z[:4], z[4:]))
466
467     def emission_slack(z: np.ndarray) -> float:
468         return float(limit - scenario_emission(profile, z[:4], z[4:], t_star, **emission_kwargs)[2])
469
470     constraints: list[dict[str, object]] = [
471         {"type": "ineq", "fun": emission_slack},
472         {"type": "ineq", "fun": _ordering},
473     ]
474     if mahalanobis is not None:
475         mu, inv_cov, threshold = mahalanobis
476         constraints.append({
477             "type": "ineq",
478             "fun": lambda z: float(threshold - (z - mu) @ inv_cov @ (z - mu)),
479         })
480
481     u_ref = np.array([profile[f"{c}_median"] for c in U_COLS], float)
482     t_ref = np.array([profile[f"{c}_median"] for c in T_COLS], float)
483     rng = np.random.default_rng(seed)
484     starts = [np.concatenate([u_ref, t_ref]), hi.copy(), lo.copy(), 0.5 * (lo + hi),
485                  np.concatenate([hi[:4], t_star])]
486     starts += [lo + (hi - lo) * rng.random(8) for _ in range(multistart)]

```

```

487
488 best_z: np.ndarray | None = None
489 best_val = np.inf
490 converged = 0
491 values: list[float] = []
492 for z0 in starts:
493     result = minimize(objective, np.clip(z0, lo, hi), method="SLSQP",
494                       bounds=list(zip(lo, hi)), constraints=constraints,
495                       options={"maxiter": 600, "ftol": 1e-10})
496     if not result.success:
497         continue
498     z = np.clip(result.x, lo, hi)
499     if emission_slack(z) < -1e-7 or np.any(_ordering(z) < -1e-7):
500         continue
501     converged += 1
502     values.append(float(result.fun))
503     if result.fun < best_val:
504         best_val, best_z = float(result.fun), z
505
506 if best_z is None:
507     return {
508         "condition_cluster": int(profile["condition_cluster"]),
509         "condition_name": str(profile["condition_name"]),
510         "limit_mgNm3": float(limit),
511         "voltage_headroom": float(voltage_headroom),
512         "status": "INFEASIBLE_IN_DECLARED_SUPPORT",
513     }
514
515 # Round to the resolution a DCS can actually hold, then repair feasibility
516 # by nudging the most power-efficient field upward.
517 z = _round_controls(best_z)
518 order = np.argsort(-field_priority(profile, model, emission_kwargs.get("alpha_weights")))
519 for _ in range(400):
520     if emission_slack(z) >= 0 and np.all(_ordering(z) >= -1e-9):
521         break
522     for i in order:
523         if z[i] + 0.1 <= hi[i] + 1e-9:
524             z[i] = round(z[i] + 0.1, 1)
525             break
526     else:
527         break
528 if emission_slack(z) < 0:
529     z = best_z # keep the unrounded solution rather than report an infeasible one
530
531 base, peak, total = scenario_emission(profile, z[:4], z[4:], t_star, **emission_kwargs)
532 row: dict[str, object] = {
533     "condition_cluster": int(profile["condition_cluster"]),
534     "condition_name": str(profile["condition_name"]),
535     "limit_mgNm3": float(limit),
536     "voltage_headroom": float(voltage_headroom),
537     "status": "SCENARIO_FEASIBLE",
538     "scenario_base_emission_mgNm3": float(base),
539     "scenario_peak_excess_mgNm3": float(peak),
540     "scenario_peak_total_mgNm3": float(total),
541     "predicted_power_kw": float(model.power(z[:4], z[4:])),
542     "field_power_kw": float(model.field_power(z[:4])),
543     "rapping_power_kw": float(model.rapping_power(z[4:])),
544     "historical_power_kw": float(profile["historical_power_kw"]),
545     "multistart_converged": converged,
546     "multistart_power_spread_kw": float(np.max(values) - np.min(values)) if values else np.nan,
547 }
548 row["power_change_vs_historical_pct"] = 100.0 * (

```

```

549     row["predicted_power_kw"] / float(profile["historical_power_kw"]) - 1.0)
550 for i, col in enumerate(U_COLS + T_COLS):
551     row[col] = float(z[i])
552 return row
553
554
555 def verify_with_random_search(
556     profile: pd.Series,
557     model: PhysicalPowerModel,
558     t_star: np.ndarray,
559     limit: float,
560     voltage_headroom: float,
561     n: int = 400_000,
562     seed: int = SEED,
563     **emission_kwargs,
564 ) -> float:
565     """Best feasible power found by uniform random sampling of the same set."""
566     lo, hi = _bounds(profile, voltage_headroom)
567     rng = np.random.default_rng(seed)
568     z = lo + (hi - lo) * rng.random((n, 8))
569     u = np.round(z[:, :4], 1)
570     t = np.round(z[:, 4:], 1)
571     keep = ((u[:, 0] >= u[:, 2] + 3.0) & (u[:, 1] >= u[:, 3] + 3.0)
572            & (t[:, 2] >= t[:, 0] + 100.0) & (t[:, 3] >= t[:, 1] + 100.0))
573     u, t = u[keep], t[keep]
574     if not len(u):
575         return float("nan")
576     _, total = scenario_emission(profile, u, t, t_star, **emission_kwargs)
577     feasible = total <= limit
578     if not np.any(feasible):
579         return float("nan")
580     return float(np.min(model.power(u[feasible], t[feasible])))
581
582
583 def aggregate(rows: List[dict[str, object]], key: str = "predicted_power_kw") -> float:
584     weights = np.array([float(r["share"]) for r in rows])
585     return float(np.average([float(r[key]) for r in rows], weights=weights))
586
587
588 def solve_all(profiles: pd.DataFrame, model: PhysicalPowerModel,
589              t_stars: dict[int, np.ndarray], limit: float, headroom: float,
590              multistart: int = MULTISTART, **emission_kwargs) -> List[dict[str, object]]:
591     rows = []
592     for _, profile in profiles.iterrows():
593         cluster = int(profile["condition_cluster"])
594         r = solve_condition(profile, model, t_stars[cluster], limit,
595                             voltage_headroom=headroom, multistart=multistart,
596                             **emission_kwargs)
597         r["share"] = float(profile["share"])
598         rows.append(r)
599     return rows
600
601
602 # ----- data-side diagnostics
603
604 def historical_strategy_correlations(df: pd.DataFrame) -> pd.DataFrame:
605     """Operator response of the control settings to the exogenous conditions.
606
607     Unlike the outlet concentration, this relationship *is* identifiable, and
608     it gives Question 3 an empirical baseline. It is reported per window as
609     well as over the whole record, because the two are not the same: the
610     rapping-period rule holds throughout, while the voltage response to inlet

```

```

611 concentration reverses sign between the first five days and the last two.
612 """
613 windows = [("full_record", df), ("train", df[df["split"] == "train"]),
614            ("validation", df[df["split"] == "validation"]),
615            ("retrospective_test", df[df["split"] == "test"])]
616 rows = []
617 for name, frame in windows:
618     for control in U_COLS + T_COLS:
619         row: dict[str, object] = {"window": name, "rows": int(len(frame)), "control": control}
620         for cond in COND_COLS:
621             row[f"r_{cond}"] = float(np.corrcoef(frame[cond], frame[control])[0, 1])
622         rows.append(row)
623 return pd.DataFrame(rows)
624
625
626 def anomaly_events(df: pd.DataFrame) -> pd.DataFrame:
627     """The two sustained excursions planted in the attachment."""
628     specs = [
629         ("入口浓度尖峰", df["C_in_gNm3"] > 60.0),
630         ("入口温度尖峰", df["Temp_C"] > 150.0),
631     ]
632     baseline = {c: float(df[c].median()) for c in COND_COLS + U_COLS + T_COLS + ["P_total_kW"]}
633     rows = []
634     for name, mask in specs:
635         sub = df[mask]
636         row: dict[str, object] = {
637             "event": name,
638             "start": str(sub["timestamp"].min()),
639             "end": str(sub["timestamp"].max()),
640             "minutes": int(len(sub)),
641         }
642         for col in COND_COLS + U_COLS + T_COLS + ["P_total_kW"]:
643             row[f"{col}_event_mean"] = float(sub[col].mean())
644             row[f"{col}_event_max"] = float(sub[col].max())
645             row[f"{col}_event_min"] = float(sub[col].min())
646             row[f"{col}_baseline_median"] = baseline[col]
647         rows.append(row)
648     return pd.DataFrame(rows)
649
650
651 def rapping_power_tradeoff(train: pd.DataFrame, model: PhysicalPowerModel) -> pd.DataFrame:
652     """How much power the rapping term holds, and what extending it recovers."""
653     u_med = train[U_COLS].median().to_numpy(float)
654     t_med = train[T_COLS].median().to_numpy(float)
655     field = float(model.field_power(u_med))
656     base_rap = float(model.rapping_power(t_med))
657     total = model.intercept + field + base_rap
658     rows = [{
659         "scenario": "历史中位周期",
660         "T_setting": ";".join(f"{v:.0f}" for v in t_med),
661         "field_power_kW": field,
662         "rapping_power_kW": base_rap,
663         "total_power_kW": total,
664         "rapping_share_pct": 100.0 * base_rap / total,
665         "power_recovered_kW": 0.0,
666         "cake_mass_per_rap_change_pct": 0.0,
667     }]
668     for label, t_new in (("延长至历史P95", train[T_COLS].quantile(0.95).to_numpy(float)),
669                        ("延长至历史最大值", train[T_COLS].max().to_numpy(float))):
670         rap = float(model.rapping_power(t_new))
671         rows.append({
672             "scenario": label,

```

```

673         "T_setting": ";".join(f"{v:.0f}" for v in t_new),
674         "field_power_kW": field,
675         "rapping_power_kW": rap,
676         "total_power_kW": model.intercept + field + rap,
677         "rapping_share_pct": 100.0 * rap / (model.intercept + field + rap),
678         "power_recovered_kW": base_rap - rap,
679         "cake_mass_per_rap_change_pct": 100.0 * (float(np.mean(t_new / t_med)) - 1.0),
680     })
681     return pd.DataFrame(rows)
682
683
684 # ----- driver
685
686 def main() -> None:
687     OUT_DIR.mkdir(parents=True, exist_ok=True)
688     df = pd.read_csv(DATA_FILE, parse_dates=["timestamp"])
689     df["condition_cluster"] = df["condition_cluster"].astype(int)
690     profiles = condition_profiles(df)
691     profiles.to_csv(OUT_DIR / "condition_profiles.csv", index=False, encoding="utf-8-sig")
692
693     train = df[df["split"] == "train"].copy()
694     validation = df[df["split"] == "validation"].copy()
695     test = df[df["split"] == "test"].copy()
696
697     model, metrics, rolling_metrics, baseline_metrics = evaluate_power_models(train, validation, test)
698     rolling_metrics.to_csv(OUT_DIR / "power_model_rolling_validation.csv", index=False, encoding="utf-8-sig")
699     baseline_metrics.to_csv(OUT_DIR / "power_model_baselines.csv", index=False, encoding="utf-8-sig")
700     (OUT_DIR / "power_model_metrics.json").write_text(
701         json.dumps(metrics, ensure_ascii=False, indent=2), encoding="utf-8")
702     guard = float(metrics["validation_abs_error_q90_kW"])
703
704     pd.read_csv(CLUSTER_EVAL_FILE).to_csv(
705         OUT_DIR / "cluster_selection_metrics.csv", index=False, encoding="utf-8-sig")
706
707     historical_strategy_correlations(df).to_csv(
708         OUT_DIR / "historical_strategy_correlations.csv", index=False, encoding="utf-8-sig")
709     anomaly_events(df).to_csv(OUT_DIR / "anomaly_events.csv", index=False, encoding="utf-8-sig")
710     rapping_power_tradeoff(train, model).to_csv(
711         OUT_DIR / "rapping_power_tradeoff.csv", index=False, encoding="utf-8-sig")
712
713     global_t_median = train[T_COLS].median().to_numpy(float)
714     t_stars = {int(p["condition_cluster"]): scenario_t_star(p, global_t_median)
715                for _, p in profiles.iterrows()}
716
717 # -- field priority criterion (analytic) -----
718 priority_rows = []
719 for _, profile in profiles.iterrows():
720     crit = field_priority(profile, model)
721     u_ref = np.array([profile[f"{c}_median"] for c in U_COLS], float)
722     row = {"condition_cluster": int(profile["condition_cluster"]),
723           "condition_name": profile["condition_name"]}
724     for i in range(4):
725         row[f"U_{i+1}_ref_kW"] = float(u_ref[i])
726         row[f"alpha_{i+1}"] = float(ALPHA_CENTRAL[i])
727         row[f"criteria_{i+1}_per_kW"] = float(crit[i])
728         row["rank_order"] = ">".join(f"E{i+1}" for i in np.argsort(-crit))
729     priority_rows.append(row)
730 pd.DataFrame(priority_rows).to_csv(
731     OUT_DIR / "field_priority_criterion.csv", index=False, encoding="utf-8-sig")
732
733 # -- central solutions -----
734 rows10 = solve_all(profiles, model, t_stars, 10.0, HEADROOM_10)

```

```

735 rows5 = solve_all(profiles, model, t_stars, 5.0, HEADROOM_5)
736 optimum = pd.DataFrame(rows10 + rows5)
737 optimum["validation_error_adjusted_power_kw"] = optimum["predicted_power_kw"] + guard
738 optimum["validation_error_guard_kw"] = guard
739 optimum.to_csv(OUT_DIR / "optimal_controls_central_scenario.csv",
740               index=False, encoding="utf-8-sig")
741
742 # -- support-domain audit (reported, not imposed) -----
743 audit_rows = []
744 for row in rows10 + rows5:
745     cluster = int(row["condition_cluster"])
746     group = train[train["condition_cluster"] == cluster]
747     mu, inv_cov, threshold = support_statistics(group)
748     z = np.array([float(row[c]) for c in U_COLS + T_COLS])
749     d2 = float((z - mu) @ inv_cov @ (z - mu))
750     u_max = np.array([group[c].max() for c in U_COLS], float)
751     audit_rows.append({
752         "condition_cluster": cluster,
753         "condition_name": row["condition_name"],
754         "limit_mgNm3": row["limit_mgNm3"],
755         "mahalanobis_d2": d2,
756         "empirical_q975": threshold,
757         "ratio_to_q975": d2 / threshold,
758         "max_voltage_over_historical_max_pct": float(
759             100.0 * max((float(row[c]) / m - 1.0) for c, m in zip(U_COLS, u_max))),
760         "verdict": "SUPPORTED" if d2 <= threshold else "EXTRAPOLATION",
761     })
762 pd.DataFrame(audit_rows).to_csv(OUT_DIR / "support_boundary_audit.csv",
763                               index=False, encoding="utf-8-sig")
764
765 # Does the historical support ellipsoid admit any feasible point at all?
766 restricted_rows = []
767 for _, profile in profiles.iterrows():
768     cluster = int(profile["condition_cluster"])
769     stats = support_statistics(train[train["condition_cluster"] == cluster])
770     for limit, headroom in ((10.0, HEADROOM_10), (5.0, HEADROOM_5)):
771         r = solve_condition(profile, model, t_stars[cluster], limit,
772                             voltage_headroom=headroom, mahalanobis=stats)
773         restricted_rows.append({
774             "condition_cluster": cluster, "limit_mgNm3": limit,
775             "status_with_mahalanobis_constraint": r["status"],
776             "predicted_power_kw": r.get("predicted_power_kw", np.nan),
777         })
778 pd.DataFrame(restricted_rows).to_csv(
779     OUT_DIR / "mahalanobis_constrained_feasibility.csv", index=False, encoding="utf-8-sig")
780
781 # -- minimum voltage headroom needed at each limit -----
782 headroom_rows = []
783 for _, profile in profiles.iterrows():
784     cluster = int(profile["condition_cluster"])
785     for limit in (10.0, 5.0):
786         need = np.nan
787         for m in np.arange(0.0, 0.31, 0.01):
788             if solve_condition(profile, model, t_stars[cluster], limit,
789                               voltage_headroom=float(m), multistart=16
790                               )["status"] == "SCENARIO_FEASIBLE":
791                 need = float(m)
792                 break
793         headroom_rows.append({
794             "condition_cluster": cluster,
795             "condition_name": profile["condition_name"],
796             "limit_mgNm3": limit,

```

```

797         "min_voltage_headroom_pct": 100.0 * need,
798     })
799 pd.DataFrame(headroom_rows).to_csv(
800     OUT_DIR / "voltage_headroom_requirement.csv", index=False, encoding="utf-8-sig")
801
802 # -- optimiser verification against dense random search -----
803 verify_rows = []
804 for _, profile in profiles.iterrows():
805     cluster = int(profile["condition_cluster"])
806     for limit, headroom, rows in ((10.0, HEADROOM_10, rows10), (5.0, HEADROOM_5, rows5)):
807         solved = next(r for r in rows if r["condition_cluster"] == cluster)
808         if solved["status"] != "SCENARIO_FEASIBLE":
809             continue
810         rnd = verify_with_random_search(profile, model, t_stars[cluster], limit, headroom)
811         verify_rows.append({
812             "condition_cluster": cluster,
813             "condition_name": profile["condition_name"],
814             "limit_mgNm3": limit,
815             "slsqp_power_kw": solved["predicted_power_kw"],
816             "random_search_power_kw": rnd,
817             "random_search_gap_pct": 100.0 * (rnd / float(solved["predicted_power_kw"]) - 1.0),
818             "multistart_converged": solved["multistart_converged"],
819             "multistart_power_spread_kw": solved["multistart_power_spread_kw"],
820         })
821 verification = pd.DataFrame(verify_rows)
822 verification.to_csv(OUT_DIR / "optimization_verification.csv", index=False, encoding="utf-8-sig")
823
824 # -- Question 4 -----
825 q4_rows = []
826 for _, profile in profiles.iterrows():
827     cluster = int(profile["condition_cluster"])
828     a = next(r for r in rows10 if r["condition_cluster"] == cluster)
829     b = next(r for r in rows5 if r["condition_cluster"] == cluster)
830     k10 = float(profile["K0_anchor"]) * float(
831         np.sum(ALPHA_CENTRAL * (np.array([a[c] for c in U_COLS], float)
832             / np.array([profile[f"{c}_median"] for c in U_COLS], float)) ** 2)
833         / ALPHA_CENTRAL.sum())
834     delta_k = math.log((10.0 - float(a["scenario_peak_excess_mgNm3"]))
835         / (5.0 - float(a["scenario_peak_excess_mgNm3"])))
836     q4_rows.append({
837         "condition_cluster": cluster,
838         "condition_name": profile["condition_name"],
839         "share": float(profile["share"]),
840         "power_10_kw": float(a["predicted_power_kw"]),
841         "power_5_kw": float(b["predicted_power_kw"]),
842         "field_power_10_kw": float(a["field_power_kw"]),
843         "K_at_10": k10,
844         "delta_K_10_to_5": delta_k,
845         "analytic_delta_power_kw": float(a["field_power_kw"]) * delta_k / k10,
846         "numeric_delta_power_kw": float(b["predicted_power_kw"]) - float(a["predicted_power_kw"]),
847         "increase_pct": 100.0 * (float(b["predicted_power_kw"]) / float(a["predicted_power_kw"]) - 1.0),
848     })
849 q4 = pd.DataFrame(q4_rows)
850 q4.to_csv(OUT_DIR / "question4_by_condition.csv", index=False, encoding="utf-8-sig")
851 weighted_p10 = float(np.average(q4["power_10_kw"], weights=q4["share"]))
852 weighted_p5 = float(np.average(q4["power_5_kw"], weights=q4["share"]))
853 historical_weighted = float(np.average(profiles["historical_power_kw"], weights=profiles["share"]))
854 q4_summary = {
855     "weighted_power_10_kw": weighted_p10,
856     "weighted_power_5_kw": weighted_p5,
857     "weighted_increase_pct": 100.0 * (weighted_p5 / weighted_p10 - 1.0),
858     "historical_weighted_actual_power_kw": historical_weighted,

```

```

859     "power_10_vs_historical_pct": 100.0 * (weighted_p10 / historical_weighted - 1.0),
860     "daily_energy_10_MWh": weighted_p10 * 24.0 / 1000.0,
861     "daily_energy_5_MWh": weighted_p5 * 24.0 / 1000.0,
862     "daily_energy_historical_MWh": historical_weighted * 24.0 / 1000.0,
863     "validation_error_guard_kW": guard,
864 }
865 (OUT_DIR / "question4_summary.json").write_text(
866     json.dumps(q4_summary, ensure_ascii=False, indent=2), encoding="utf-8")
867
868 # -- p reference table: the single scalar that needs site calibration ----
869 # Holding the 10 mg/Nm^3 operating point fixed, scaling every voltage by f
870 # gives K = K_10 f^p and P_field = P_field,10 f^2, so halving the limit
871 # (delta_K) costs exactly P_field,10 [(1 + delta_K/K_10)^(2/p) - 1]:
872 # the penalty is governed by 2/p and nothing else.
873 p_rows = []
874 for p_exp in (1.0, 1.5, 2.0, 2.5, 3.0):
875     analytic = []
876     for row in q4_rows:
877         ratio = 1.0 + row["delta_K_10_to_5"] / row["K_at_10"]
878         analytic.append({"share": row["share"],
879                         "delta": row["field_power_10_kW"] * (ratio ** (2.0 / p_exp) - 1.0),
880                         "p10": row["power_10_kW"]})
881     w = np.array([a["share"] for a in analytic])
882     delta = float(np.average([a["delta"] for a in analytic], weights=w))
883     p10 = float(np.average([a["p10"] for a in analytic], weights=w))
884     # required voltage headroom above the historical maximum, same reasoning
885     f = float(np.mean([(1.0 + r["delta_K_10_to_5"] / r["K_at_10"]) ** (1.0 / p_exp)
886                        for r in q4_rows]))
887     r10 = solve_all(profiles, model, t_stars, 10.0, HEADROOM_10,
888                     multistart=SWEET_MULTISTART, p_exponent=p_exp)
889     r5 = solve_all(profiles, model, t_stars, 5.0, HEADROOM_5,
890                    multistart=SWEET_MULTISTART, p_exponent=p_exp)
891     feasible = all(r["status"] == "SCENARIO_FEASIBLE" for r in r10 + r5)
892     p_rows.append({
893         "p_exponent": p_exp,
894         "analytic_delta_power_kW": delta,
895         "analytic_increase_pct": 100.0 * delta / p10,
896         "implied_voltage_scaling_10_to_5": f,
897         "feasible_at_5pct_headroom": feasible,
898         "numeric_weighted_power_10_kW": aggregate(r10) if feasible else np.nan,
899         "numeric_weighted_power_5_kW": aggregate(r5) if feasible else np.nan,
900         "numeric_increase_pct": (100.0 * (aggregate(r5) / aggregate(r10) - 1.0)) if feasible else np.nan,
901     })
902 pd.DataFrame(p_rows).to_csv(OUT_DIR / "p_exponent_reference.csv",
903                             index=False, encoding="utf-8-sig")
904
905 # -- scale sensitivity: p x peak scale x safety index (27 combinations) --
906 sensitivity_rows = []
907 for p_exp in (1.5, 2.0, 2.5):
908     for peak_scale in (0.9, 1.2, 1.5):
909         for safety in (0.0, 0.08, 0.15):
910             kw = dict(p_exponent=p_exp, peak_scale=peak_scale, safety_index=safety)
911             r10 = solve_all(profiles, model, t_stars, 10.0, HEADROOM_10,
912                             multistart=SWEET_MULTISTART, **kw)
913             r5 = solve_all(profiles, model, t_stars, 5.0, HEADROOM_5,
914                             multistart=SWEET_MULTISTART, **kw)
915             ok = all(r["status"] == "SCENARIO_FEASIBLE" for r in r10 + r5)
916             sensitivity_rows.append({
917                 "p_exponent": p_exp, "peak_scale": peak_scale, "safety_index": safety,
918                 "all_conditions_feasible": ok,
919                 "weighted_power_10_kW": aggregate(r10) if ok else np.nan,
920                 "weighted_power_5_kW": aggregate(r5) if ok else np.nan,

```



```

921         "weighted_power_increase_pct": (100.0 * (aggregate(r5) / aggregate(r10) - 1.0)) if ok else np.nan,
922     })
923 sensitivity = pd.DataFrame(sensitivity_rows)
924 sensitivity.to_csv(OUT_DIR / "question4_sensitivity.csv", index=False, encoding="utf-8-sig")
925
926 # -- structural sensitivity: 81 grey-box shape combinations -----
927 alpha_profiles = {
928     "equal": np.array([1.00, 1.00, 1.00, 1.00]),
929     "weak_front": np.array([1.08, 1.04, 0.98, 0.90]),
930     "central_front": ALPHA_CENTRAL,
931 }
932 structural_rows = []
933 for alpha_name, alpha_weights in alpha_profiles.items():
934     for peak_exponent in (1.00, 1.30, 1.60):
935         for phase_scale in (0.65, 0.80, 1.00):
936             for tstar_exponent in (-0.30, -0.18, -0.10):
937                 stars = {int(p["condition_cluster"]): scenario_t_star(p, global_t_median, tstar_exponent)
938                     for _, p in profiles.iterrows()}
939                 kw = dict(alpha_weights=alpha_weights, peak_exponent=peak_exponent,
940                     phase_scale=phase_scale)
941                 r10 = solve_all(profiles, model, stars, 10.0, HEADROOM_10,
942                     multistart=SWEET_MULTISTART, **kw)
943                 r5 = solve_all(profiles, model, stars, 5.0, HEADROOM_5,
944                     multistart=SWEET_MULTISTART, **kw)
945                 ok = all(r["status"] == "SCENARIO_FEASIBLE" for r in r10 + r5)
946                 structural_rows.append({
947                     "alpha_profile": alpha_name,
948                     "peak_exponent": peak_exponent,
949                     "phase_scale": phase_scale,
950                     "tstar_load_exponent": tstar_exponent,
951                     "all_conditions_feasible": ok,
952                     "weighted_power_10_kw": aggregate(r10) if ok else np.nan,
953                     "weighted_power_5_kw": aggregate(r5) if ok else np.nan,
954                     "weighted_power_increase_pct": (100.0 * (aggregate(r5) / aggregate(r10) - 1.0)) if ok else np.nan,
955                 })
956 structural = pd.DataFrame(structural_rows)
957 structural.to_csv(OUT_DIR / "structural_sensitivity.csv", index=False, encoding="utf-8-sig")
958
959 # -- field weight ablation -----
960 ablation_rows, ablation_summary_rows = [], []
961 for alpha_name, alpha_weights in alpha_profiles.items():
962     r10 = solve_all(profiles, model, t_stars, 10.0, HEADROOM_10,
963         multistart=SWEET_MULTISTART, alpha_weights=alpha_weights)
964     r5 = solve_all(profiles, model, t_stars, 5.0, HEADROOM_5,
965         multistart=SWEET_MULTISTART, alpha_weights=alpha_weights)
966     per_profile = []
967     for a, b in zip(r10, r5):
968         row = {"alpha_profile": alpha_name, "condition_cluster": a["condition_cluster"],
969             "condition_name": a["condition_name"], "share": a["share"]}
970         for i, col in enumerate(U_COLS, start=1):
971             row[f"delta_U{i}_kV"] = float(b[col]) - float(a[col])
972         per_profile.append(row)
973     ablation_rows.append(row)
974     weights = np.array([x["share"] for x in per_profile])
975     deltas = np.array([x[f"delta_U{i}_kV"] for i in range(1, 5)] for x in per_profile)
976     weighted = np.average(deltas, axis=0, weights=weights)
977     positive = np.maximum(weighted, 0)
978     ablation_summary_rows.append({
979         "alpha_profile": alpha_name,
980         **{f"weighted_delta_U{i}_kV": weighted[i - 1] for i in range(1, 5)},
981         "front_two_share_of_positive_adjustment": float(positive[:2].sum() / positive.sum())
982         if positive.sum() else np.nan,

```

```

983     })
984     pd.DataFrame(ablation_rows).to_csv(
985         OUT_DIR / "field_priority_ablation_by_condition.csv", index=False, encoding="utf-8-sig")
986     pd.DataFrame(ablation_summary_rows).to_csv(
987         OUT_DIR / "field_priority_ablation_summary.csv", index=False, encoding="utf-8-sig")
988
989     # -- historical comparison and online schedule -----
990     historical_rows, dynamic_rows = [], []
991     for _, profile in profiles.iterrows():
992         cluster = int(profile["condition_cluster"])
993         u_med = np.array([profile[f"{c}_median"] for c in U_COLS], float)
994         t_med = np.array([profile[f"{c}_median"] for c in T_COLS], float)
995         median_power = float(model.power(u_med, t_med))
996         a = next(r for r in rows10 if r["condition_cluster"] == cluster)
997         b = next(r for r in rows5 if r["condition_cluster"] == cluster)
998         historical_rows.append({
999             "condition_cluster": cluster,
1000             "condition_name": profile["condition_name"],
1001             "share": float(profile["share"]),
1002             "historical_mean_actual_power_kw": float(profile["historical_power_kw"]),
1003             "model_power_at_historical_median_controls_kw": median_power,
1004             "scenario_optimal_10_power_kw": float(a["predicted_power_kw"]),
1005             "power_change_vs_model_median_pct": 100.0 * (float(a["predicted_power_kw"]) / median_power - 1.0),
1006         })
1007         dynamic_rows.append({
1008             "condition_cluster": cluster,
1009             "condition_name": profile["condition_name"],
1010             "mode_selection_rule": "EXTERNAL_TARGET_SELECTION",
1011             "available_modes": "TARGET-10/TARGET-5/FALLBACK",
1012             "automatic_emission_threshold_switching": False,
1013             "provisional_voltage_ramp_kv_per_min": 0.5,
1014             "provisional_period_ramp_s_per_min": 5.0,
1015             "phase_scenario_for_online_trial": "0.65/0.80/1.00",
1016             "implementation_status": "PROVISIONAL_REQUIRES_T/R_CAPACITY_CHECK_AND_SITE_INTERLOCK_LIMITS",
1017             **{f"target10_{c}": a[c] for c in U_COLS + T_COLS},
1018             **{f"target5_{c}": b[c] for c in U_COLS + T_COLS},
1019         })
1020     pd.DataFrame(historical_rows).to_csv(
1021         OUT_DIR / "historical_median_power_comparison.csv", index=False, encoding="utf-8-sig")
1022     pd.DataFrame(dynamic_rows).to_csv(
1023         OUT_DIR / "dynamic_control_schedule.csv", index=False, encoding="utf-8-sig")
1024
1025     scenario_params = {
1026         "status": "EMISSION_LAYER_IS_SCENARIO_ONLY_C_OUT_NOT_IDENTIFIABLE",
1027         "emission_anchor": "recorded outlet median, no decimal rescaling",
1028         "p_exponent_central": P_EXPONENT_CENTRAL,
1029         "p_exponent_grid": [1.0, 1.5, 2.0, 2.5, 3.0],
1030         "peak_exponent_central": PEAK_EXPONENT_CENTRAL,
1031         "peak_exponent_grid": [1.00, 1.30, 1.60],
1032         "voltage_weight_profiles": {k: v.tolist() for k, v in alpha_profiles.items()},
1033         "rapping_phase_scales": [0.65, 0.80, 1.00],
1034         "tstar_load_exponents": [-0.30, -0.18, -0.10],
1035         "rapping_loss_coefficients": BETA_CENTRAL.tolist(),
1036         "peak_field_shares": PEAK_SHARES.tolist(),
1037         "support_rule": "cluster-specific full historical range plus engineering ordering; "
1038             "Mahalanobis d2 reported as extrapolation audit, not imposed",
1039         "voltage_headroom_10": HEADROOM_10,
1040         "voltage_headroom_5": HEADROOM_5,
1041         "optimiser": "multi-start SLSQP (%d starts) verified against 400k random samples" % MULTISTART,
1042     }
1043     (OUT_DIR / "scenario_parameters.json").write_text(
1044         json.dumps(scenario_params, ensure_ascii=False, indent=2), encoding="utf-8")

```

```

1045
1046 feasible_structural = structural[structural["all_conditions_feasible"]]
1047 feasible_scale = sensitivity[sensitivity["all_conditions_feasible"]]
1048 summary = {
1049     "power_model": metrics,
1050     "cluster_k": 4,
1051     "central_q4": q4_summary,
1052     "rapping_share_at_median_pct": float(
1053         rapping_power_tradeoff(train, model).iloc[0]["rapping_share_pct"]),
1054     "scale_sensitivity": {
1055         "feasible_combinations": int(len(feasible_scale)),
1056         "total_combinations": int(len(sensitivity)),
1057         "increase_pct_min": float(feasible_scale["weighted_power_increase_pct"].min()),
1058         "increase_pct_median": float(feasible_scale["weighted_power_increase_pct"].median()),
1059         "increase_pct_max": float(feasible_scale["weighted_power_increase_pct"].max()),
1060     },
1061     "structural_sensitivity": {
1062         "feasible_combinations": int(len(feasible_structural)),
1063         "total_combinations": int(len(structural)),
1064         "increase_pct_min": float(feasible_structural["weighted_power_increase_pct"].min()),
1065         "increase_pct_median": float(feasible_structural["weighted_power_increase_pct"].median()),
1066         "increase_pct_max": float(feasible_structural["weighted_power_increase_pct"].max()),
1067     },
1068     "optimiser_verification": {
1069         "max_random_search_gap_pct": float(verification["random_search_gap_pct"].max()),
1070         "max_multistart_spread_kw": float(verification["multistart_power_spread_kw"].max()),
1071     },
1072     "central_all_conditions_feasible": bool((optimum["status"] == "SCENARIO_FEASIBLE").all()),
1073 }
1074 (OUT_DIR / "model_run_summary.json").write_text(
1075     json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8")
1076 print(json.dumps(summary, ensure_ascii=False, indent=2))
1077
1078
1079 if __name__ == "__main__":
1080     main()

```

E.2 paper_figures.py

```

1  #!/usr/bin/env python3
2  """Generate the complete publication figure set for the cement ESP paper.
3
4  All empirical plots are regenerated from the supplied processed dataset and
5  model result tables. Scenario-only plots are explicitly labelled in captions.
6  Both PNG previews and vector PDF files are exported.
7  """
8
9  from __future__ import annotations
10
11  import sys
12  import hashlib
13  from pathlib import Path
14
15  import matplotlib.pyplot as plt
16  from matplotlib.patches import FancyArrowPatch, FancyBboxPatch
17  import numpy as np
18  import pandas as pd
19
20  SRC_DIR = Path(__file__).resolve().parent
21  if str(SRC_DIR) not in sys.path:
22      sys.path.insert(0, str(SRC_DIR))

```

```

23
24 from plot_utils import (GRAY, LIGHT_GRAY, NEUTRAL_FILL, PALETTE, add_subpanel_label,
25                          figure_legend, finish_axes, headroom, note, save_figure, set_paper_style)
26 from scenario_model import (ALPHA_CENTRAL, PhysicalPowerModel, RidgePowerModel,
27                             T_COLS, U_COLS)
28
29
30 ROOT = SRC_DIR.parent
31 DATA_FILE = ROOT / "data" / "full_timeseries_with_flags.csv"
32 RESULTS = ROOT / "results"
33 OUT = ROOT / "figures_paper"
34
35 COND_SHORT = {
36     1: "工况1\n低温低浓高流",
37     2: "工况2\n高温中浓低流",
38     3: "工况3\n高温高浓高流",
39     4: "工况4\n低温高浓低流",
40 }
41
42 CAPTIONS = {
43     "01_outlet_data_diagnosis": "出口浓度记录存在显著截顶且不覆盖目标区间。有效记录全部位于48.74~50.00 mg/Nm³，其中5491条恰好
44     ⇨ 等于50 mg/Nm³，因而不能从该列辨识5或10 mg/Nm³附近的控制—排放关系。",
45     "02_process_timeseries": "原始过量量具有连续时序变化和工况切换特征。图中展示首24小时入口温度、入口浓度、烟气流量及总
46     ⇨ 功率的分钟级变化。",
47     "04b_operating_evidence": "附件中可识别的另一半信息。(a)(b)为两段持续异常工况及控制端的同步响应：入口浓度尖峰时抬高
48     ⇨  $\$U_{1\$}$ 、功率随之上冲，入口温度尖峰（比电阻高值区）时反而降压降功率；(c)为训练期工况变量与控制设定的相关矩阵，显
49     ⇨ 示“高浓度→抬升前两电场电压、缩短各场振打周期”的历史操作规律。",
50     "14_rapping_peak_mechanism": "(a)由“每周期积灰量 $\propto \$L \cdot T\$$ ”推得单次再飞扬峰值随周期近似线性放大，而对小时均值的贡献
51     ⇨  $\propto \$L \cdot T \cdot (1/T) = L\$$ 与周期无关（水平点线）；(b)振打环节在中位工况下占总功率约四成，把周期由中位延长到历史极
52     ⇨ 值可回收约109 kW，代价是单次积灰量增加约18%。",
53     "03_cluster_selection": "四类工况在轮廓系数与解释复杂度之间取得较好平衡。k=4的轮廓系数达到0.408，且避免继续增加聚类数带
54     ⇨ 来的解释负担。",
55     "04_condition_map": "四类工况在入口浓度—温度平面上形成清晰分区，并由烟气流量大小补充刻画负荷差异。",
56     "07_power_prediction": "总功率由控制量近乎确定地决定。物理设定 $\$P = a_0 + \sum b_i U_i^2 + \sum c_i / T_i\$$ 在回顾性测试段的 $\$R^2\$$ 与RMSE
57     ⇨ 均显著优于均值基线及把振打周期作为线性项的岭回归设定。",
58     "08b_feature_importance": "振打周期必须以 $\$1/T\$$ 进入功率模型。(a)扣除电场功率后的剩余功率对 $\$1/T\$$ 呈明显双曲形状，线性设定
59     ⇨ （虚线）在两端系统性偏离；(b)线性 $\$T\$$ 设定在回顾性测试段留下约-10.8 kW的平均偏差，改为 $\$1/T\$$ 后偏差降至约0。",
60     "08_power_residuals": "改用 $\$1/T\$$ 设定后，回顾性测试残差围绕零对称分布，原先“高功率区负偏差扩大”的现象消失，说明该偏差
61     ⇨ 源于设定误差而非数据漂移。",
62     "09_voltage_power_response": "在各工况典型状态下，模型给出的总功率随四电场电压同步提高而上升。曲线仅在各工况历史电压支
63     ⇨ 持区间附近绘制。",
64     "05b_boundary_proximity": "把排放由历史约50 mg/Nm³压到10与5 mg/Nm³，必然离开历史联合运行经验：八组解的马氏距离平方为经验97
65     ⇨ 0.5%外壳的2.4~7.0倍；10 mg解仍位于历史电压包络之内，5 mg解则需要各场电压超出历史最大值最多5%。",
66     "19_emission_power_frontier": "排放—功率权衡前沿。限值由10逐档收紧至5 mg/Nm³时四类工况的最优功率单调上升，工况加权功率
67     ⇨ 由约1972升至约2105 kW；两档均高于历史实际加权功率，说明超低排放改造是以增电耗换减排，而非节能。",
68     "11_optimal_voltage": "5 mg/Nm³方案在各工况均需进一步抬升电压，且抬升集中在单位功率去除收益最高的电场上；柱高已接近或
69     ⇨ 超出历史电压包络，必须先核验高压电源容量。",
70     "12_optimal_rapping_period": "两档限值下的最优振打周期均被推向历史区间上端——延长周期直接降低振打功率；前级短、后级长
71     ⇨ 的工程顺序约束始终保持。",
72     "13_emission_decomposition": "最优方案的约束值由连续排放基值和振打峰值增量共同构成。柱顶虚线分别表示10和5 mg/Nm³情景限
73     ⇨ 值。",
74     "18_condition_energy_penalty": "四类工况由10收紧至5 mg/Nm³的功率增幅高度一致（约6.7%），这与解析式 $\Delta P = P_{\text{field}} - P_{\text{K}} / K$ 
75     ⇨ 的预测相符：增幅主要由电场功率占比与当前去除指数决定，而与工况细节关系不大。",
76     "15_search_convergence": "多起点SLSQP与同一可行集上 $4 \times 10^6$ 点随机搜索的对照。随机搜索在全部八个工况—限值组合上均未找到更
77     ⇨ 优解，其最优值高出0.1%~1.5%，说明确定性求解已到达（至少）同等质量的解，不再需要百万级随机采样。",
78     "16_sensitivity_distribution": "81组灰箱结构情景全部可行。峰值指数在本文推导值1.00附近变化几乎不改变功率增幅，相位叠加
79     ⇨ 尺度与场权重形状的影响也有限，增幅集中在6%~8%区间。",
80     "20_eta_sensitivity": "功率增幅由单一可标定标量 $\$p\$$ 决定。 $\$K \propto p^{\text{propto } U^4 p^{\text{时增幅近似正比于 } \$2/p\$}$ ： $\$p=1\$$ 约13%， $\$p=2\$$ （Deutsch迁移
81     ⇨ 速度形式）约6.7%， $\$p=3\$$ 约4.5%。现场只需一次电压阶跃试验即可标定 $\$p\$$ ，从而把区间结论换成确定数值。",
82     "17_sensitivity_heatmap": "场权重消融结果。三种去除贡献权重形状下，10→5 mg/Nm³所需的加权电压增量分布与按 $\alpha_i / (b_i U_{\text{ref},2})^2$ 
83     ⇨ 排序的解析优先级一致。",
84 }

```

```

64
65 TRACE_META = {
66     "01_outlet_data_diagnosis": (str(DATA_FILE), "fig01_outlet_diagnosis", "§ 2.1—2.2; 图2", "出口记录为量程截顶; 不代表真实排放分
        ↳ 布"),
67     "02_process_timeseries": (str(DATA_FILE), "fig02_timeseries", "§ 2.2; 图3", "只展示首24 h, 不代表全周期分布"),
68     "04b_operating_evidence": (str(RESULTS / "historical_strategy_correlations.csv"), "fig04b_operating_evidence", "§ 4.2—4.3; 图4", "相关
        ↳ 系数为同期线性关联, 不等于因果; 两段事件为个案, 样本量有限"),
69     "14_rapping_peak_mechanism": (str(RESULTS / "rapping_power_tradeoff.csv"), "fig14_peak", "§ 4.4; 图5", "峰值尺度 $b_0$ 仍为工程先验;
        ↳ 功率侧数值由附件识别"),
70     "03_cluster_selection": (str(RESULTS / "cluster_selection_metrics.csv"), "fig03_cluster_selection", "§ 5.1; 图6", "聚类指标只评价统计
        ↳ 分离度, 不等于物理工况唯一性"),
71     "04_condition_map": (str(DATA_FILE), "fig04_condition_map", "§ 5.1; 图7", "为可视化抽样点; 聚类实际使用全部训练样本"),
72     "07_power_prediction": (str(RESULTS / "power_model_baselines.csv"), "fig07_power_prediction + PhysicalPowerModel", "§ 5.2; 图8", "回顾
        ↳ 性测试并非真正未触碰盲测"),
73     "08b_feature_importance": (str(RESULTS / "power_model_baselines.csv"), "fig08b_specification", "§ 5.2; 图9", "残差图按 $T_{1s}$ 展开, 其余
        ↳ 三场周期固定在中位数"),
74     "08_power_residuals": (str(DATA_FILE), "fig08_residuals + PhysicalPowerModel", "§ 5.2; 图10", "残差接近零均值, 但仍非全新日期盲测
        ↳ "),
75     "09_voltage_power_response": (str(DATA_FILE), "fig09_voltage_response + PhysicalPowerModel", "§ 5.2; 图11", "局部情景曲线, 仅在历史
        ↳ 电压附近解释"),
76     "05b_boundary_proximity": (str(RESULTS / "support_boundary_audit.csv"), "fig05b_boundary_proximity", "§ 5.4; 图12", "经验支持域不是设
        ↳ 备安全边界; 超出部分必须由现场阶跃试验确认"),
77     "19_emission_power_frontier": (str(RESULTS / "emission_power_frontier.csv"), "fig19_frontier", "§ 5.5; 图13", "各限值下均为灰箱情景
        ↳ 解, 前沿形状依赖排放先验"),
78     "11_optimal_voltage": (str(RESULTS / "optimal_controls_central_scenario.csv"), "grouped_controls(U_COLS)", "§ 6; 图14", "最优电压是情
        ↳ 景试验起点, 不是设备安全设定"),
79     "12_optimal_rapping_period": (str(RESULTS / "optimal_controls_central_scenario.csv"), "grouped_controls(T_COLS)", "§ 6; 图15", "无实际
        ↳ 振打事件, 周期结果依赖峰值先验"),
80     "13_emission_decomposition": (str(RESULTS / "optimal_controls_central_scenario.csv"), "fig13_emission", "§ 7.2; 图17", "全部排放值为情
        ↳ 景推演, 不是实测"),
81     "18_condition_energy_penalty": (str(RESULTS / "question4_by_condition.csv"), "fig18_penalty", "§ 7.1; 图16", "中心情景点估计, 需与
        ↳  $\rho$ 参考表同时解读"),
82     "15_search_convergence": (str(RESULTS / "optimization_verification.csv"), "fig15_optimizer_verification", "§ 8.1; 图18", "随机搜索对照
        ↳ 只能证伪, 不构成全局最优的数学证明"),
83     "16_sensitivity_distribution": (str(RESULTS / "structural_sensitivity.csv"), "fig16_structural_sensitivity", "§ 8.2; 图19", "扫描范围
        ↳ 仍由显式情景网格决定"),
84     "20_eta_sensitivity": (str(RESULTS / "p_exponent_reference.csv"), "fig20_p_sensitivity", "§ 8.2; 图20", " $\rho$ 的取值区间为先验网格;
        ↳ 需由现场阶跃试验标定"),
85     "17_sensitivity_heatmap": (str(RESULTS / "field_priority_ablation_summary.csv"), "fig17_field_ablation", "§ 8.2; 图21", "消融仍依赖同
        ↳ 一灰箱结构与支持域"),
86 }
87
88 MANUSCRIPT_FIGURE = {
89     "01_outlet_data_diagnosis": 2,
90     "02_process_timeseries": 3,
91     "04b_operating_evidence": 4,
92     "14_rapping_peak_mechanism": 5,
93     "03_cluster_selection": 6,
94     "04_condition_map": 7,
95     "07_power_prediction": 8,
96     "08b_feature_importance": 9,
97     "08_power_residuals": 10,
98     "09_voltage_power_response": 11,
99     "05b_boundary_proximity": 12,
100     "19_emission_power_frontier": 13,
101     "11_optimal_voltage": 14,
102     "12_optimal_rapping_period": 15,
103     "18_condition_energy_penalty": 16,
104     "13_emission_decomposition": 17,
105     "15_search_convergence": 18,
106     "16_sensitivity_distribution": 19,
107     "20_eta_sensitivity": 20,

```

```

108     "17_sensitivity_heatmap": 21,
109 }
110
111
112 def _label_bars(ax, bars, fmt="{:.0f}", dy=3.0):
113     for bar in bars:
114         h = bar.get_height()
115         ax.text(bar.get_x() + bar.get_width() / 2, h + dy, fmt.format(h), ha="center", va="bottom", fontsize=7)
116
117
118 def fig01_outlet_diagnosis(df: pd.DataFrame) -> None:
119     valid = df["C_out_mgNm3"].dropna()
120     fig, axes = plt.subplots(1, 2, figsize=(8.2, 3.2), gridspec_kw={"width_ratios": [1.15, 1]})
121     ax = axes[0]
122     add_subpanel_label(ax, "a")
123     bins = np.linspace(valid.min() - 0.03, valid.max() + 0.03, 28)
124     ax.hist(valid, bins=bins, color=PALETTE[0], edgecolor="white")
125     ax.axvline(50, color=PALETTE[4], linestyle="--", label="记录上限 50")
126     ax.set_xlim(valid.min() - 0.05, valid.max() + 0.05)
127     ax.text(
128         0.03,
129         0.78,
130         "5与10 mg/Nm³目标均在图示范围外",
131         transform=ax.transAxes,
132         color=PALETTE[2],
133         fontsize=8,
134         bbox={"boxstyle": "round", "facecolor": "white", "edgecolor": LIGHT_GRAY},
135     )
136     ax.set_xlabel("出口粉尘浓度记录 (mg/Nm³)")
137     ax.set_ylabel("样本数")
138     ax.legend(frameon=False, loc="upper left")
139     finish_axes(ax)
140
141     ax = axes[1]
142     add_subpanel_label(ax, "b")
143     total = len(df)
144     categories = ["精确值", "50截顶", "缺失"]
145     values = [int(df["C_out_exact_flag"].sum()), int(df["C_out_cap50_flag"].sum()), int(df["C_out_missing_flag"].sum())]
146     bars = ax.bar(categories, values, color=[PALETTE[0], PALETTE[4], GRAY])
147     for bar, value in zip(bars, values):
148         ax.text(bar.get_x() + bar.get_width()/2, value + total*0.012, f"{value}\n({100*value/total:.1f}%)", ha="center", fontsize=8)
149     ax.set_ylabel("样本数")
150     ax.set_ylim(0, max(values) * 1.18)
151     note(ax, "百分比以全部10080条样本为分母", loc="upper right")
152     finish_axes(ax)
153     fig.tight_layout(w_pad=2.0)
154     save_figure(fig, OUT, "01_outlet_data_diagnosis")
155
156
157 def fig02_timeseries(df: pd.DataFrame) -> None:
158     d = df.iloc[:1440].copy()
159     hours = np.arange(len(d)) / 60
160     fields = [
161         ("Temp_C", "入口温度 (°C)", PALETTE[4]),
162         ("C_in_gNm3", "入口浓度 (g/Nm³)", PALETTE[0]),
163         ("Q_Nm3h", "烟气流量 (Nm³/h)", PALETTE[2]),
164         ("P_total_kw", "总功率 (kW)", PALETTE[3]),
165     ]
166     fig, axes = plt.subplots(4, 1, figsize=(8.2, 5.5), sharex=True)
167     for ax, (col, label, color), tag in zip(axes, fields, ["a", "b", "c", "d"]):
168         add_subpanel_label(ax, tag, x=-0.08, y=1.02)
169         ax.plot(hours, d[col], color=color, linewidth=0.8)

```

```

170     ax.set_ylabel(label)
171     finish_axes(ax)
172     axes[-1].set_xlabel("首日运行时间 (h)")
173     axes[-1].set_xlim(0, 24)
174     fig.tight_layout(h_pad=0.5)
175     save_figure(fig, OUT, "02_process_timeseries")
176
177
178 def fig03_cluster_selection(cluster_eval: pd.DataFrame) -> None:
179     fig, axes = plt.subplots(1, 2, figsize=(7.7, 3.0))
180     ax = axes[0]
181     add_subpanel_label(ax, "a")
182     ax.plot(cluster_eval["k"], cluster_eval["silhouette"], "o-", color=PALETTE[0])
183     chosen = cluster_eval[cluster_eval["selected"]].iloc[0]
184     ax.scatter([chosen["k"]], [chosen["silhouette"]], s=65, color=PALETTE[4], zorder=3, label="选定 k=4")
185     ax.set_xlabel("聚类数 k")
186     ax.set_ylabel("轮廓系数")
187     ax.legend(frameon=False)
188     finish_axes(ax)
189     ax = axes[1]
190     add_subpanel_label(ax, "b")
191     ax.plot(cluster_eval["k"], cluster_eval["inertia"] / 1000, "s--", color=PALETTE[2], label="惯性/1000")
192     ax.set_xlabel("聚类数 k")
193     ax.set_ylabel("类内惯性 ( $\times 10^3$ )")
194     finish_axes(ax)
195     fig.tight_layout(w_pad=2.0)
196     save_figure(fig, OUT, "03_cluster_selection")
197
198
199 def fig04_condition_map(df: pd.DataFrame) -> None:
200     train = df[df["split"] == "train"]
201     fig, ax = plt.subplots(figsize=(6.7, 4.4))
202     for cluster, group in train.groupby("condition_cluster"):
203         g = group.iloc[:5]
204         sizes = 8 + 28 * (g["Q_Nm3h"] - train["Q_Nm3h"].min()) / (train["Q_Nm3h"].max() - train["Q_Nm3h"].min())
205         ax.scatter(g["C_in_gNm3"], g["Temp_C"], s=sizes, alpha=0.38, color=PALETTE[int(cluster)-1], label=COND_SHORT[int(cluster)].replace("\
↔ n", " "))
206     ax.set_xlabel("入口粉尘浓度 (g/Nm³)")
207     ax.set_ylabel("入口温度 (°C)")
208     ax.legend(frameon=False, ncol=2)
209     finish_axes(ax, grid_axis="both")
210     fig.tight_layout()
211     save_figure(fig, OUT, "04_condition_map")
212
213
214
215 def fig04b_operating_evidence(df: pd.DataFrame, events: pd.DataFrame,
216                             strategy: pd.DataFrame) -> None:
217     """The identifiable half of Question 1.
218
219     The outlet column carries no signal, but the operator's response to the
220     inlet conditions is plainly visible: two sustained excursions (panels a, b)
221     and a stable correlation structure between conditions and settings (c).
222     """
223     fig = plt.figure(figsize=(8.4, 5.6))
224     gs = fig.add_gridspec(2, 2, height_ratios=[1, 1.05], hspace=0.60, wspace=0.62)
225
226     windows = [
227         ("2024-05-02 13:30", "2024-05-02 16:20", "C_in_gNm3", "入口浓度 (g/Nm³)", "a"),
228         ("2024-05-06 12:30", "2024-05-06 15:30", "Temp_C", "入口温度 (°C)", "b"),
229     ]
230     for col_idx, (t0, t1, col, ylabel, tag) in enumerate(windows):

```

```

231     ax = fig.add_subplot(gs[0, col_idx])
232     add_subpanel_label(ax, tag, x=-0.14)
233     win = df[(df["timestamp"] >= t0) & (df["timestamp"] <= t1)]
234     minutes = (win["timestamp"] - win["timestamp"].iloc[0]).dt.total_seconds() / 60
235     ax.plot(minutes, win[col], color=PALETTE[4], linewidth=1.4, label=ylabel.split(" ")[0])
236     ax.set_ylabel(ylabel, color=PALETTE[4], fontsize=9)
237     ax.tick_params(axis="y", labelcolor=PALETTE[4])
238     ax.set_xlabel("事件窗口内时间 (min)")
239     twin = ax.twinx()
240     twin.plot(minutes, win["U1_kV"], color=PALETTE[0], linewidth=1.4, label="$U_1$")
241     twin.plot(minutes, win["P_total_kW"] / 30.0, color=PALETTE[2],
242               linewidth=1.2, linestyle="--", label="总功率/30")
243     twin.set_ylabel("$U_1$ (kV) 总功率/30 (kW)", fontsize=8.0)
244     twin.spines["top"].set_visible(False)
245     finish_axes(ax, grid_axis="none")
246     handles = ax.get_lines() + twin.get_lines()
247     ax.legend(handles, [h.get_label() for h in handles],
248              frameon=False, fontsize=7.4, ncol=3, loc="upper center",
249              bbox_to_anchor=(0.5, 1.30))
250
251 ax = fig.add_subplot(gs[1, :])
252 add_subpanel_label(ax, "c", x=-0.055)
253 cond_labels = {"r_Temp_c": "入口温度 $H$", "r_C_in_gNm3": "入口浓度 ${C_{in}}$", "r_Q_Nm3h": "烟气流量 $Q$"}
254 full = strategy[strategy["window"] == "full_record"].set_index("control")
255 matrix = full.loc[U_COLS + T_COLS, list(cond_labels)].to_numpy(float).T
256 im = ax.imshow(matrix, cmap="RdBu_r", aspect="auto", vmin=-0.30, vmax=0.30)
257 ax.set_xticks(range(matrix.shape[1]),
258               [f"${U_{i}}$" for i in range(1, 5)] + [f"${T_{i}}$" for i in range(1, 5)])
259 ax.set_yticks(range(3), list(cond_labels.values()))
260 for i in range(matrix.shape[0]):
261     for j in range(matrix.shape[1]):
262         ax.text(j, i, f"{matrix[i, j]:+.3f}", ha="center", va="center", fontsize=8,
263               color="white" if abs(matrix[i, j]) > 0.20 else "#1F2933")
264 cbar = fig.colorbar(im, ax=ax, fraction=0.028, pad=0.015)
265 cbar.set_label("Pearson 相关系数", fontsize=8.4)
266 ax.set_title("全记录期工况变量与控制设定的历史响应（可由附件识别）", fontsize=9.2, pad=8)
267 tr = strategy[strategy["window"] == "train"].set_index("control")
268 te = strategy[strategy["window"] == "retrospective_test"].set_index("control")
269 ax.text(1.0, -0.32,
270        "注：振荡周期规律（${C_{in}} \rightarrow T \rightarrow$）在四个时窗内一致；"
271        "电压规律并不平稳——${r(C_{in}, U_1)}$ 在前5天为 %+.2f, 在末日为 %+.2f。"
272        "% (float(tr.loc["U1_kV", "r_C_in_gNm3"]), float(te.loc["U1_kV", "r_C_in_gNm3"])),
273        transform=ax.transAxes, ha="right", va="top", fontsize=7.8, color=GRAY)
274 save_figure(fig, OUT, "04b_operating_evidence")
275
276
277 def fig05b_boundary_proximity(audit: pd.DataFrame) -> None:
278     """How far outside historical joint operating experience each solution sits."""
279     ordered = audit.sort_values(["condition_cluster", "limit_mgNm3"],
280                                ascending=[True, False]).reset_index(drop=True)
281     labels = [f"工况{int(c)}\n{int(l)} mg"
282              for c, l in zip(ordered["condition_cluster"], ordered["limit_mgNm3"])]
283     colors = [PALETTE[int(c) - 1] for c in ordered["condition_cluster"]]
284
285     fig, axes = plt.subplots(1, 2, figsize=(8.4, 3.6), gridspec_kw={"width_ratios": [1.15, 1]})
286     ax = axes[0]
287     add_subpanel_label(ax, "a")
288     x = np.arange(len(ordered))
289     ax.vlines(x, 1.0, ordered["ratio_to_q975"], color=colors, linewidth=2.2, alpha=0.8)
290     ax.scatter(x, ordered["ratio_to_q975"], color=colors, s=45,
291              edgecolor="black", linewidth=0.5, zorder=3)
292     ax.axhline(1.0, color=PALETTE[4], linestyle="--", linewidth=1.2, label="历史经验97.5%外壳")

```



```

293 ax.set_xticks(x, labels, fontsize=7.6)
294 ax.set_ylabel(r"$d_M^2/q_{0.975}$ (倍) ")
295 ax.legend(frameon=False, loc="upper left", fontsize=8)
296 headroom(ax, 0.16)
297 finish_axes(ax)
298
299 ax = axes[1]
300 add_subpanel_label(ax, "b")
301 bars = ax.bar(x, ordered["max_voltage_over_historical_max_pct"], color=colors, width=0.62)
302 _label_bars(ax, bars, fmt="{:.1f}%", dy=0.12)
303 ax.axhline(0, color=PALETTE[4], linestyle="--", linewidth=1.2)
304 ax.set_xticks(x, labels, fontsize=7.6)
305 ax.set_ylabel("最高电压超出历史最大值 (%)")
306 note(ax, "0%以下=仍在历史电压包络内", loc="upper left")
307 headroom(ax, 0.30)
308 finish_axes(ax)
309 fig.tight_layout(w_pad=1.9)
310 save_figure(fig, OUT, "05b_boundary_proximity")
311
312
313 def fit_power_model(df: pd.DataFrame) -> PhysicalPowerModel:
314     train = df[df["split"] == "train"]
315     return PhysicalPowerModel().fit(train, train["P_total_kw"].to_numpy(float))
316
317
318 def fit_linear_period_model(df: pd.DataFrame) -> RidgePowerModel:
319     """The misspecified reference: rapping period entered as a linear term."""
320     train = df[df["split"] == "train"]
321     metrics = pd.read_json(RESULTS / "power_model_metrics.json", typ="series")
322     alpha = float(metrics["ridge_selected_alphas"]["quadratic_ridge_T"])
323     model = RidgePowerModel(alpha=alpha)
324     model.fit(train, train["P_total_kw"].to_numpy(float))
325     return model
326
327
328 def fig07_power_prediction(df: pd.DataFrame, model: RidgePowerModel, baselines: pd.DataFrame) -> None:
329     test = df[df["split"] == "test"]
330     actual = test["P_total_kw"].to_numpy(float)
331     pred = model.predict(test)
332     fig, axes = plt.subplots(1, 2, figsize=(8.2, 3.5))
333     ax = axes[0]
334     add_subpanel_label(ax, "a")
335     ax.scatter(actual, pred, s=10, alpha=0.42, color=PALETTE[0], edgecolors="none")
336     lo, hi = min(actual.min(), pred.min()), max(actual.max(), pred.max())
337     ax.plot([lo, hi], [lo, hi], "--", color=PALETTE[4], label="理想预测")
338     metrics = pd.read_json(RESULTS / "power_model_metrics.json", typ="series")
339     ax.text(0.04, 0.94,
340            "$R^2$=%.3f$\n$RMSE$=%.2f kW$\n$偏差$=%+.2f kW" % (float(metrics["retrospective_test_r2"]),
341                                                                float(metrics["retrospective_test_rmse_kw"]),
342                                                                float(metrics["retrospective_test_bias_kw"])),
343            transform=ax.transAxes, va="top",
344            bbox={"boxstyle": "round", "facecolor": "white", "edgecolor": "LIGHT_GRAY"})
345     ax.set_xlabel("实际总功率 (kW)")
346     ax.set_ylabel("预测总功率 (kW)")
347     ax.legend(frameon=False, loc="lower right")
348     finish_axes(ax, grid_axis="both")
349
350     ax = axes[1]
351     add_subpanel_label(ax, "b")
352     labels = ["均值\n基线", "线性$u$\n线性$t$", "二次$u$\n线性$t$", "本文\n$u^2+1/t$"]
353     model_order = ["mean_predictor", "linear_ridge_T", "quadratic_ridge_T", "physical_invT"]
354     x = np.arange(len(model_order))

```

```

355 val = baselines[baselines["split"] == "validation"].set_index("model").loc[model_order, "rmse_kw"]
356 tst = baselines[baselines["split"] == "retrospective_test"].set_index("model").loc[model_order, "rmse_kw"]
357 ax.bar(x-0.18, val, 0.36, color=PALETTE[5], label="验证集")
358 ax.bar(x+0.18, tst, 0.36, color=PALETTE[0], label="回顾性测试")
359 ax.set_xticks(x, labels, fontsize=7.6)
360 ax.set_ylabel("RMSE (kW, 对数刻度)")
361 ax.set_yscale("log")
362 ax.legend(frameon=False, loc="upper right")
363 finish_axes(ax)
364 fig.tight_layout()
365 save_figure(fig, OUT, "07_power_prediction")
366
367
368 def fig08_residuals(df: pd.DataFrame, model: RidgePowerModel) -> None:
369     test = df[df["split"] == "test"]
370     actual = test["P_total_kw"].to_numpy(float)
371     pred = model.predict(test)
372     res = pred - actual
373     fig, axes = plt.subplots(1, 2, figsize=(7.7, 3.1))
374     add_subpanel_label(axes[0], "a")
375     axes[0].scatter(pred, res, s=9, alpha=0.38, color=PALETTE[0], edgecolors="none")
376     axes[0].axhline(0, color=PALETTE[4], linestyle="--")
377     axes[0].set_xlabel("预测总功率 (kW)")
378     axes[0].set_ylabel("残差: 预测-实际 (kW)")
379     finish_axes(axes[0], grid_axis="both")
380     add_subpanel_label(axes[1], "b")
381     axes[1].hist(res, bins=28, color=PALETTE[2], edgecolor="white")
382     metrics = pd.read_json(RESULTS / "power_model_metrics.json", typ="series")
383     guard = float(metrics["validation_abs_error_q90_kw"])
384     axes[1].axvline(res.mean(), color=PALETTE[4], linestyle="--", label=f"测试均值 {res.mean():.2f} kW")
385     axes[1].axvline(guard, color=PALETTE[0], linestyle=":", label=f"验证|误差|P90 {guard:.2f} kW")
386     axes[1].set_xlabel("残差 (kW)")
387     axes[1].set_ylabel("样本数")
388     axes[1].legend(frameon=False)
389     finish_axes(axes[1])
390     fig.tight_layout(w_pad=2.0)
391     save_figure(fig, OUT, "08_power_residuals")
392
393
394 def fig08b_specification(df: pd.DataFrame, model: PhysicalPowerModel,
395                        linear_model: RidgePowerModel, baselines: pd.DataFrame) -> None:
396     """Why the rapping period must enter as 1/T rather than T.
397
398     Panel (a) shows the residual of a voltage-only fit against the period:
399     the curvature is the signature of a 1/T term. Panel (b) contrasts the
400     retrospective-test bias of both specifications.
401     """
402     train = df[df["split"] == "train"]
403     test = df[df["split"] == "test"]
404     fig, axes = plt.subplots(1, 2, figsize=(8.2, 3.4), gridspec_kw={"width_ratios": [1.15, 1]})
405
406     ax = axes[0]
407     add_subpanel_label(ax, "a")
408     u = train[U_COLS].to_numpy(float)
409     resid = train["P_total_kw"].to_numpy(float) - model.field_power(u) - model.intercept
410     t1 = train["T1_s"].to_numpy(float)
411     order = np.argsort(t1)
412     ax.scatter(t1[order][:7], resid[order][:7], s=7, alpha=0.22,
413               color=PALETTE[5], edgecolors="none", label="扣除电场功率后的残差")
414     grid = np.linspace(t1.min(), t1.max(), 120)
415     rap_other = float(np.sum(model.c[1:] / train[T_COLS[1:]].median()).to_numpy(float))
416     ax.plot(grid, model.c[0] / grid + rap_other, color=PALETTE[4], linewidth=2.0,

```

```

417     label=r"拟合  $c_1/T_1+\mathrm{const}$ ")
418     slope = np.polyfit(t1, resid, 1)
419     ax.plot(grid, np.polyval(slope, grid), "--", color=PALETTE[0], linewidth=1.6,
420             label=r"线性  $T_1$  设定")
421     ax.set_xlabel("第1电场振打周期  $T_1$  (s)")
422     ax.set_ylabel("剩余功率 (kW)")
423     ax.legend(frameon=False, fontsize=8, loc="upper right")
424     finish_axes(ax, grid_axis="both")
425
426     ax = axes[1]
427     add_subpanel_label(ax, "b")
428     names = {"quadratic_ridge_T": "线性  $T$  设定", "physical_invT": "本文  $1/T$  设定"}
429     tst = baselines[baselines["split"] == "retrospective_test"].set_index("model")
430     x = np.arange(2)
431     bias = [float(tst.loc[k, "bias_kw"]) for k in names]
432     rmse = [float(tst.loc[k, "rmse_kw"]) for k in names]
433     ax.bar(x - 0.18, bias, 0.36, color=PALETTE[4], label="测试平均偏差")
434     ax.bar(x + 0.18, rmse, 0.36, color=PALETTE[0], label="测试 RMSE")
435     for xi, (b, r) in enumerate(zip(bias, rmse)):
436         ax.text(xi - 0.18, b + (0.6 if b >= 0 else -1.6), f"{b:+.2f}", ha="center", fontsize=8)
437         ax.text(xi + 0.18, r + 0.6, f"{r:.2f}", ha="center", fontsize=8)
438     ax.axhline(0, color=GRAY, linewidth=0.8)
439     ax.set_xticks(x, list(names.values()))
440     ax.set_ylabel("回顾性测试 (kW)")
441     ax.legend(frameon=False, fontsize=8)
442     headroom(ax, 0.22)
443     finish_axes(ax)
444     fig.tight_layout(w_pad=2.0)
445     save_figure(fig, OUT, "08b_feature_importance")
446
447
448 def fig09_voltage_response(df: pd.DataFrame, profiles: pd.DataFrame, model: RidgePowerModel) -> None:
449     train = df[df["split"] == "train"]
450     fig, ax = plt.subplots(figsize=(6.4, 4.0))
451     for _, p in profiles.iterrows():
452         c = int(p["condition_cluster"])
453         group = train[train["condition_cluster"] == c]
454         scale = np.linspace(0.92, 1.08, 80)
455         rows = []
456         u0 = group[U_COLS].median().to_numpy(float)
457         t0 = group[T_COLS].median().to_numpy(float)
458         for s in scale:
459             row = {col: val for col, val in zip(U_COLS, u0*s)}
460             row.update({col: val for col, val in zip(T_COLS, t0)})
461             row.update({"Temp_C": p["Temp_C"], "C_in_gNm3": p["C_in_gNm3"], "Q_Nm3h": p["Q_Nm3h"]})
462             rows.append(row)
463         y = model.predict(pd.DataFrame(rows))
464         ax.plot(100*(scale-1), y, color=PALETTE[c-1], marker=["o", "s", "x", "D"][c-1], markevery=16,
465               label=COND_SHORT[c].replace("\n", " "))
466     ax.set_xlabel("四电场电压同步变化 (%)")
467     ax.set_ylabel("模型预测总功率 (kW)")
468     ax.axvline(0, color=GRAY, linewidth=0.8)
469     ax.legend(frameon=False, ncol=2)
470     finish_axes(ax, grid_axis="both")
471     fig.tight_layout()
472     save_figure(fig, OUT, "09_voltage_power_response")
473
474
475
476 def grouped_controls(optimum: pd.DataFrame, cols: list[str], stem: str, ylabel: str) -> None:
477     fig, axes = plt.subplots(2, 2, figsize=(8.2, 5.8), sharey=True)
478     for ax, cluster, tag in zip(axes.flat, range(1,5), ["a", "b", "c", "d"]):

```

```

479     add_subpanel_label(ax, tag)
480     sub = optimum[optimum["condition_cluster"] == cluster].set_index("limit_mgNm3")
481     x = np.arange(4)
482     ax.bar(x-0.18, sub.loc[10.0, cols], 0.36, color=PALETTE[5], label="10 mg/Nm³")
483     ax.bar(x+0.18, sub.loc[5.0, cols], 0.36, color=PALETTE[4], label="5 mg/Nm³")
484     ax.set_xticks(x, [c.split("_")[0] for c in cols])
485     ax.set_title(COND_SHORT[cluster].replace("\n", " "), fontsize=9)
486     finish_axes(ax)
487 axes[0,0].set_ylabel(ylabel); axes[1,0].set_ylabel(ylabel)
488 handles, labels = axes[0,0].get_legend_handles_labels()
489 fig.tight_layout(h_pad=2.0, rect=(0, 0.06, 1, 1))
490 figure_legend(fig, handles, labels, ncol=2, bottom=0.0)
491 save_figure(fig, OUT, stem)
492
493
494
495 def fig13_emission(optimum: pd.DataFrame) -> None:
496     x = np.arange(8)
497     ordered = optimum.sort_values(["limit_mgNm3", "condition_cluster"], ascending=[False, True])
498     base = ordered["scenario_base_emission_mgNm3"].to_numpy()
499     peak = ordered["scenario_peak_excess_mgNm3"].to_numpy()
500     colors = [PALETTE[0]]*4 + [PALETTE[4]]*4
501     fig, ax = plt.subplots(figsize=(8.0, 4.0))
502     ax.bar(x, base, color=colors, alpha=0.82, label="连续排放基值")
503     ax.bar(x, peak, bottom=base, color=colors, alpha=0.35, edgecolor=colors, label="振打峰值增量")
504     ax.hlines(10, -0.5, 3.5, colors=GRAY, linestyle="--", linewidth=1.0)
505     ax.hlines(5, 3.5, 7.5, colors=GRAY, linestyle="--", linewidth=1.0)
506     ax.text(3.45, 10.15, "10 mg/Nm³", ha="right", fontsize=8, color=GRAY)
507     ax.text(7.45, 5.15, "5 mg/Nm³", ha="right", fontsize=8, color=GRAY)
508     ax.set_xticks(x, [f"工况{i}" for i in range(1,5)]*2)
509     ax.set_ylabel("峰值总排放情景值 (mg/Nm³)")
510     ax.legend(frameon=False, ncol=2)
511     finish_axes(ax)
512     fig.tight_layout()
513     save_figure(fig, OUT, "13_emission_decomposition")
514
515
516 def fig14_peak(profiles: pd.DataFrame) -> None:
517     """The rapping trade-off, derived rather than assumed.
518
519     Cake mass per rap grows like L*T, so the per-event peak grows linearly with
520     the period while the contribution to the hourly mean, L*T*(1/T) = L, does
521     not depend on the period at all. Panel (b) prices the same lever in power.
522     """
523     trade = pd.read_csv(RESULTS / "rapping_power_tradeoff.csv")
524     ratio = np.linspace(0.75, 1.35, 200)
525     fig, axes = plt.subplots(1, 2, figsize=(8.3, 3.5), gridspec_kw={"width_ratios": [1.1, 1]})
526
527     ax = axes[0]
528     add_subpanel_label(ax, "a")
529     for cluster in (1, 3, 4):
530         pr = profiles[profiles["condition_cluster"] == cluster].iloc[0]
531         peak = 1.2 * float(pr["load_ratio_to_train_median"]) * ratio
532         ax.plot(100 * (ratio - 1), peak, color=PALETTE[cluster - 1],
533               marker=["o", "s", "A", "D"][cluster - 1], markevery=40,
534               label=COND_SHORT[cluster].replace("\n", " "))
535     ax.axhline(0, color=GRAY, linewidth=0.8)
536     ax.axvline(0, color=GRAY, linewidth=0.8)
537     ax.plot(100 * (ratio - 1), np.full_like(ratio, 1.2 * float(
538         profiles[profiles["condition_cluster"] == 3].iloc[0]["load_ratio_to_train_median"])),
539           ":", color="#111827", linewidth=1.5, label="对小时均值的贡献 (工况3, 与Ts无关)")
540     ax.set_xlabel("振打周期相对参考周期变化 (%)")

```

```

541 ax.set_ylabel("单次再飞扬峰值代理 (mg/Nm³)")
542 ax.legend(frameon=False, fontsize=7.8, loc="upper left")
543 headroom(ax, 0.18)
544 finish_axes(ax, grid_axis="both")
545
546 ax = axes[1]
547 add_subpanel_label(ax, "b")
548 x = np.arange(len(trade))
549 bars = ax.bar(x, trade["rapping_power_kw"], color=[PALETTE[5], PALETTE[2], PALETTE[4]], width=0.6)
550 _label_bars(ax, bars, fmt="{:.0f}", dy=6)
551 for xi, row in trade.iterrows():
552     if row["power_recovered_kw"] > 0:
553         ax.text(xi, row["rapping_power_kw"] / 2,
554                 "省%.0f kW\n积灰+%.0f%%" % (row["power_recovered_kw"],
555                                             row["cake_mass_per_rap_change_pct"]),
556                 ha="center", va="center", fontsize=7.6, color="white")
557 ax.set_xticks(x, ["历史\n中位周期", "延长至\n历史P95", "延长至\n历史最大"], fontsize=8)
558 ax.set_ylabel("振打环节功率 (kW)")
559 note(ax, "中位工况下振打占总功率 %.1f%%" % float(trade.iloc[0]["rapping_share_pct"]),
560      loc="upper right")
561 headroom(ax, 0.24)
562 finish_axes(ax)
563 fig.tight_layout(w_pad=2.0)
564 save_figure(fig, OUT, "14_rapping_peak_mechanism")
565
566
567 def fig15_optimizer_verification(verification: pd.DataFrame) -> None:
568     """Multi-start SLSQP against a 4x10^5-point random search of the same set."""
569     ordered = verification.sort_values(["condition_cluster", "limit_mgNm3"],
570                                       ascending=[True, False]).reset_index(drop=True)
571     labels = [f"工况{int(c)}\n{int(l)} mg"
572              for c, l in zip(ordered["condition_cluster"], ordered["limit_mgNm3"])]
573     x = np.arange(len(ordered))
574     slsqp = ordered["slsqp_power_kw"].to_numpy(float)
575     rnd = ordered["random_search_power_kw"].to_numpy(float)
576     gaps = ordered["random_search_gap_pct"].to_numpy(float)
577
578     fig, axes = plt.subplots(1, 2, figsize=(8.4, 3.3), gridspec_kw={"width_ratios": [1.2, 1]})
579
580     ax = axes[0]
581     add_subpanel_label(ax, "a")
582     lo = float(np.nanmin(slsqp)) * 0.94
583     hi = float(np.nanmax([np.nanmax(slsqp), np.nanmax(rnd)])) * 1.04
584     ax.set_ylim(lo, hi)
585     ax.bar(x - 0.18, slsqp, 0.36, color=PALETTE[0], label="多起点SLSQP")
586     ax.bar(x + 0.18, np.where(np.isfinite(rnd), rnd, lo), 0.36,
587           color=LIGHT_GRAY, label="随机搜索最优")
588     for xi, value in enumerate(rnd):
589         if not np.isfinite(value):
590             ax.annotate("随机搜索\n无可行样本", xy=(xi + 0.18, lo),
591                       xytext=(xi + 0.55, lo + 0.30 * (hi - lo)),
592                       fontsize=7.2, color=PALETTE[4], ha="left",
593                       arrowprops={"arrowstyle": "->", "color": PALETTE[4], "linewidth": 0.9})
594     ax.set_xticks(x, labels, fontsize=7.4)
595     ax.set_ylabel("最优总功率 (kW)")
596     ax.legend(frameon=False, fontsize=8, loc="upper left", ncol=2)
597     finish_axes(ax)
598
599     ax = axes[1]
600     add_subpanel_label(ax, "b")
601     finite = np.isfinite(gaps)
602     ax.bar(x[finite], gaps[finite], color=PALETTE[2], width=0.6)

```

```

603     for xi, value in zip(x[finite], gaps[finite]):
604         ax.text(xi, value + 0.12, f"{value:.2f}", ha="center", va="bottom", fontsize=7.4)
605     ax.set_ylim(0, float(np.nanmax(gaps)) * 1.42)
606     ax.set_xticks(x, labels, fontsize=7.4)
607     ax.set_ylabel("随机搜索相对SLSQP的劣势 (%)")
608     note(ax, "全部为正：随机搜索未找到更优解", loc="upper right")
609     finish_axes(ax)
610     fig.tight_layout(w_pad=2.2)
611     save_figure(fig, OUT, "15_search_convergence")
612
613
614 def fig16_structural_sensitivity(structural: pd.DataFrame) -> None:
615     valid = structural[structural["all_conditions_feasible"] == True]
616     fig, axes = plt.subplots(1, 2, figsize=(8.2, 3.5), sharey=True)
617     ax = axes[0]
618     add_subpanel_label(ax, "a")
619     exps = sorted(valid["peak_exponent"].unique())
620     data = [valid[valid["peak_exponent"] == x]["weighted_power_increase_pct"] for x in exps]
621     boxes = ax.boxplot(data, tick_labels=[f"{x:.2f}" for x in exps],
622                        patch_artist=True, showfliers=False)
623     for i, box in enumerate(boxes["boxes"]):
624         box.set_facecolor(PALETTE[i % len(PALETTE)]); box.set_alpha(0.72)
625     ax.set_xlabel(r"振打峰值指数  $\gamma$  (本文推导值 1.00) ")
626     ax.set_ylabel("5相对10 mg/Nm3加权功率增幅 (%)")
627     finish_axes(ax)
628
629     ax = axes[1]
630     add_subpanel_label(ax, "b")
631     phases = sorted(valid["phase_scale"].unique())
632     for profile, style, label, color in zip(
633         ["equal", "weak_front", "central_front"], ["o-", "s--", "^:"],
634         ["等权", "弱前级", "中心前级"], [PALETTE[0], PALETTE[2], PALETTE[4]]):
635         medians = [valid[(valid["phase_scale"] == ph) & (valid["alpha_profile"] == profile)]
636                    ["weighted_power_increase_pct"].median() for ph in phases]
637         ax.plot(phases, medians, style, color=color, label=label)
638     ax.set_xlabel(r"振打相位叠加尺度  $\rho$ ")
639     ax.legend(frameon=False)
640     finish_axes(ax, grid_axis="both")
641     fig.tight_layout(w_pad=2.0)
642     save_figure(fig, OUT, "16_sensitivity_distribution")
643
644
645 def fig17_field_ablation(ablation: pd.DataFrame) -> None:
646     profiles = ["equal", "weak_front", "central_front"]
647     labels = ["等权", "弱前级", "中心前级"]
648     cols = [f"weighted_delta_U{i}_kv" for i in range(1,5)]
649     d = ablation.set_index("alpha_profile").loc[profiles, cols]
650     fig, axes = plt.subplots(1, 2, figsize=(7.8, 3.3), gridspec_kw={"width_ratios": [1.45, 1]})
651     ax = axes[0]
652     add_subpanel_label(ax, "a")
653     im = ax.imshow(d.to_numpy(), cmap="YlGnBu", aspect="auto", vmin=0)
654     ax.set_xticks(range(4), [f"U{i}" for i in range(1,5)])
655     ax.set_yticks(range(3), labels)
656     ax.set_xlabel("电场")
657     ax.set_ylabel("去除贡献权重设定")
658     for i in range(3):
659         for j in range(4):
660             ax.text(j, i, f"{d.iloc[i,j]:.2f}", ha="center", va="center", color="white" if d.iloc[i,j]>5 else "black", fontsize=8)
661     cbar = fig.colorbar(im, ax=ax, fraction=0.05, pad=0.04)
662     cbar.set_label("10→5 mg时加权电压增量 (kV)")
663
664     ax = axes[1]

```

```

665 add_subpanel_label(ax, "b")
666 shares = 100*ablation.set_index("alpha_profile").loc[profiles,"front_two_share_of_positive_adjustment"]
667 bars = ax.bar(labels, shares, color=[PALETTE[0],PALETTE[2],PALETTE[4]])
668 _label_bars(ax, bars, fmt=":.1f%", dy=0.7)
669 ax.set_ylabel("前两电场占正向电压增量 (%)")
670 ax.set_ylim(0,100)
671 finish_axes(ax)
672 fig.tight_layout(w_pad=2.0)
673 save_figure(fig, OUT, "17_sensitivity_heatmap")
674
675
676 def fig18_penalty(q4: pd.DataFrame) -> None:
677     x = np.arange(4)
678     weighted_p10 = float(np.sum(q4["share"] * q4["power_10_kw"]) / q4["share"].sum())
679     weighted_p5 = float(np.sum(q4["share"] * q4["power_5_kw"]) / q4["share"].sum())
680     weighted = 100.0 * (weighted_p5 / weighted_p10 - 1.0)
681     fig, ax = plt.subplots(figsize=(6.4, 3.8))
682     bars = ax.bar(x, q4.sort_values("condition_cluster")["increase_pct"], color=PALETTE[:4])
683     _label_bars(ax, bars, fmt=":.2f%", dy=0.12)
684     ax.axhline(weighted, color=PALETTE[4], linestyle="--", label=f"工况加权平均 {weighted:.2f}%")
685     ax.set_xticks(x, [COND_SHORT[i] for i in range(1,5)])
686     ax.set_ylabel("情景预测功率增幅 (%)")
687     ax.set_ylim(0, max(q4["increase_pct"])*1.22)
688     ax.legend(frameon=False)
689     finish_axes(ax)
690     fig.tight_layout()
691     save_figure(fig, OUT, "18_condition_energy_penalty")
692
693
694 def _relpath(path: str) -> str:
695     """交付物中一律使用相对路径，避免写入本机绝对路径。"""
696     try:
697         return Path(path).resolve().relative_to(ROOT).as_posix()
698     except ValueError:
699         return Path(path).name
700
701
702 TIKZ_ROWS = [{
703     "figure_id": "01", "manuscript_figure": 1, "stem": "00_esp_schematic",
704     "caption": "四电场电除尘器结构与变量定义",
705     "png": "figures_paper/00_esp_schematic.pdf", "pdf": "figures_paper/00_esp_schematic.pdf",
706     "source_data": "figures_paper/00_esp_schematic.tex",
707     "transformation": "按赛题给定的设备结构示意图以Tikz矢量重绘，xelatex独立编译",
708     "supported_manuscript_claims": "$1.1；图1",
709     "limitations": "结构示意图，不含实测数据；比例非工程真实尺寸",
710 }]
711
712
713 def fig19_frontier(frontier: pd.DataFrame, weighted: pd.DataFrame) -> None:
714     """排放—功率权衡前沿：说明放宽限值换来的功率下降不是等排放下的节能。"""
715     fig, ax = plt.subplots(figsize=(7.4, 4.0))
716     for idx, (cluster, grp) in enumerate(frontier.groupby("condition_cluster")):
717         grp = grp.sort_values("Limit_mgNm3")
718         ax.plot(grp["Limit_mgNm3"], grp["predicted_power_kw"], "o-",
719                 color=PALETTE[idx], markersize=4.5, linewidth=1.5,
720                 label=COND_SHORT[int(cluster)].replace("\n", " "))
721     w = weighted.sort_values("Limit_mgNm3")
722     ax.plot(w["Limit_mgNm3"], w["weighted_power_kw"], "s--", color="#11827",
723            markersize=5, linewidth=2.0, label="工况加权")
724     hist = float(frontier["historical_power_kw"].iloc[0]) if "historical_power_kw" in frontier else np.nan
725     p10 = float(w[w["Limit_mgNm3"] == 10.0]["weighted_power_kw"].iloc[0])
726     p5 = float(w[w["Limit_mgNm3"] == 5.0]["weighted_power_kw"].iloc[0])

```

```

727     if np.isfinite(hist):
728         ax.axhline(hist, color=GRAY, linestyle=":", linewidth=1.3)
729         ax.text(9.9, hist - 14, f"历史实际加权功率 {hist:.0f} kW (排放约50 mg/Nm³) ",
730               fontsize=8, color=GRAY, ha="right")
731     ax.annotate("", xy=(10.0, p10), xytext=(5.0, p5),
732               arrowprops={"arrowstyle": "<->", "color": PALETTE[4], "linewidth": 1.3})
733     ax.text(7.5, (p5 + p10) / 2 + 10,
734           f"限值 10→5 mg/Nm³\n加权功率增 {p5 - p10:.0f} kW ({100*(p5-p10)/p10:.1f}% )",
735           fontsize=8.2, color=PALETTE[4], ha="center")
736     ax.set_xlabel("情景排放限值 (mg/Nm³)")
737     ax.set_ylabel("支持域内最佳采样预测功率 (kW)")
738     ax.set_xticks(sorted(frontier["limit_mgNm3"].unique()))
739     ax.legend(frameon=False, ncol=2, fontsize=8.4, loc="upper right")
740     headroom(ax, 0.16)
741     finish_axes(ax, grid_axis="both")
742     fig.tight_layout()
743     save_figure(fig, OUT, "19_emission_power_frontier")
744
745
746 def fig20_p_sensitivity(p_ref: pd.DataFrame, prior: pd.DataFrame) -> None:
747     """The increase is set by one calibratable scalar: the exponent p in  $K \sim U^p$ ."""
748     ref = p_ref.sort_values("p_exponent")
749     fig, ax = plt.subplots(figsize=(6.9, 3.9))
750     ax.plot(ref["p_exponent"], ref["analytic_increase_pct"], "-", color="#111827",
751           linewidth=1.8, zorder=2, label=r"解析式  $\$P_{\text{f}}[(1+\Delta K/K)^{\{2/p\}-1}]/P_{\{10\}}\$")
752     feas = ref[ref["feasible_at_5pct_headroom"] == True]
753     infeas = ref[ref["feasible_at_5pct_headroom"] != True]
754     ax.scatter(feas["p_exponent"], feas["analytic_increase_pct"], s=62, color=PALETTE[0],
755           edgecolor="white", linewidth=0.8, zorder=4, label="5%电压裕度内可达")
756     ax.scatter(infeas["p_exponent"], infeas["analytic_increase_pct"], s=62,
757           facecolor="white", edgecolor=PALETTE[4], linewidth=1.6, zorder=4,
758           label="需更大电压裕度 (不可达) ")
759     num = ref.dropna(subset=["numeric_increase_pct"])
760     ax.scatter(num["p_exponent"], num["numeric_increase_pct"], marker="x", s=52,
761           color=PALETTE[2], zorder=5, label="数值重解校验")
762     for _, row in ref.iterrows():
763         ax.annotate(f"{row['analytic_increase_pct']:.1f}%",
764               (row["p_exponent"], row["analytic_increase_pct"]),
765               textcoords="offset points", xytext=(0, 10), ha="center",
766               fontsize=8.2, color=GRAY)
767     ax.set_xlabel(r"去除指数—电压幂次  $\$p\$$  ( $\$K \propto U^p\$$ ;  $\$p=2\$$  为Deutsch迁移速度形式) ")
768     ax.set_ylabel("10→5 mg/Nm³ 加权功率增幅 (%)")
769     ax.legend(frameon=False, fontsize=8, loc="upper right")
770     headroom(ax, 0.24)
771     finish_axes(ax, grid_axis="both")
772     fig.tight_layout()
773     save_figure(fig, OUT, "20_eta_sensitivity")
774
775
776 def write_manifest() -> None:
777     script_hash = hashlib.sha256(Path(__file__).read_bytes()).hexdigest()
778     model_hash = hashlib.sha256((SRC_DIR / "scenario_model.py").read_bytes()).hexdigest()
779     rows = []
780     for stem, caption in CAPTIONS.items():
781         source_data, function_name, claims, limitations = TRACE_META[stem]
782         source_data = _relpath(source_data)
783         rows.append({"figure_id": stem.split("_")[0], "manuscript_figure": MANUSCRIPT_FIGURE[stem], "stem": stem, "caption": caption,
784               "png": f"figures_paper/{stem}.png", "pdf": f"figures_paper/{stem}.pdf",
785               "source_data": source_data,
786               "transformation": f"paper_figures.py::{function_name}; plot_sha256={script_hash}; model_sha256={model_hash}",
787               "supported_manuscript_claims": claims,
788               "limitations": limitations})$ 
```



```

789 rows = TIKZ_ROWS + rows
790 rows.sort(key=lambda r: r["manuscript_figure"])
791 pd.DataFrame(rows).to_csv(OUT / "figure_manifest.csv", index=False, encoding="utf-8-sig")
792 lines = ["# 论文图组与图注", "", "所有图均提供 PNG 预览和 PDF 矢量版本。", ""]
793 for row in sorted(rows, key=lambda x: x["manuscript_figure"]):
794     lines.extend([
795         f"## 正文图{row['manuscript_figure']} (文件编号{row['figure_id']}) {row['stem']}", "", row["caption"], "",
796         f"- 源数据: `{row['source_data']}`",
797         f"- 变换: `{row['transformation']}`",
798         f"- 支持的正文论断: {row['supported_manuscript_claims']}",
799         f"- 局限: {row['limitations']}",
800         f"- PNG: `{row['png']}`", f"- PDF: `{row['pdf']}`", ""])
801 (OUT / "图注清单.md").write_text("\n".join(lines), encoding="utf-8")
802
803
804 def main() -> None:
805     set_paper_style()
806     df = pd.read_csv(DATA_FILE, parse_dates=["timestamp"])
807     profiles = pd.read_csv(RESULTS / "condition_profiles.csv")
808     cluster_eval = pd.read_csv(RESULTS / "cluster_selection_metrics.csv")
809     optimum = pd.read_csv(RESULTS / "optimal_controls_central_scenario.csv")
810     audit = pd.read_csv(RESULTS / "support_boundary_audit.csv")
811     verification = pd.read_csv(RESULTS / "optimization_verification.csv")
812     structural = pd.read_csv(RESULTS / "structural_sensitivity.csv")
813     ablation = pd.read_csv(RESULTS / "field_priority_ablation_summary.csv")
814     baselines = pd.read_csv(RESULTS / "power_model_baselines.csv")
815     q4 = pd.read_csv(RESULTS / "question4_by_condition.csv")
816     frontier = pd.read_csv(RESULTS / "emission_power_frontier.csv")
817     frontier_w = pd.read_csv(RESULTS / "emission_power_frontier_weighted.csv")
818     prior = pd.read_csv(RESULTS / "question4_sensitivity.csv")
819     p_ref = pd.read_csv(RESULTS / "p_exponent_reference.csv")
820     events = pd.read_csv(RESULTS / "anomaly_events.csv")
821     strategy = pd.read_csv(RESULTS / "historical_strategy_correlations.csv")
822     model = fit_power_model(df)
823     linear_model = fit_linear_period_model(df)
824
825     fig01_outlet_diagnosis(df)
826     fig02_timeseries(df)
827     fig04b_operating_evidence(df, events, strategy)
828     fig14_peak(profiles)
829     fig03_cluster_selection(cluster_eval)
830     fig04_condition_map(df)
831     fig07_power_prediction(df, model, baselines)
832     fig08b_specification(df, model, linear_model, baselines)
833     fig08_residuals(df, model)
834     fig09_voltage_response(df, profiles, model)
835     fig05b_boundary_proximity(audit)
836     fig19_frontier(frontier, frontier_w)
837     grouped_controls(optimum, U_COLS, "11_optimal_voltage", "最优电压 (kV)")
838     grouped_controls(optimum, T_COLS, "12_optimal_rapping_period", "最优振打周期 (s)")
839     fig18_penalty(q4)
840     fig13_emission(optimum)
841     fig15_optimizer_verification(verification)
842     fig16_structural_sensitivity(structural)
843     fig20_p_sensitivity(p_ref, prior)
844     fig17_field_ablation(ablation)
845     write_manifest()
846     print(f"Generated {len(CAPTIONS)} figures in {OUT}")
847
848
849 if __name__ == "__main__":
850     main()

```

E.3 plot_utils.py

```
1  """Shared publication plotting style for the cement ESP paper."""
2
3  from __future__ import annotations
4
5  from pathlib import Path
6
7  import matplotlib.pyplot as plt
8  from cycler import cycler
9
10
11  # Okabe-Ito 色序: 色觉障碍友好, 灰度打印仍可区分
12  PALETTE = ["#0072B2", "#E69F00", "#009E73", "#CC79A7", "#D55E00", "#56B4E9"]
13  GRAY = "#5B6472"
14  LIGHT_GRAY = "#D1D5DB"
15  NEUTRAL_FILL = "#DFE3E8"
16
17
18  def set_paper_style() -> None:
19      plt.rcParams.update(
20          {
21              "font.family": "sans-serif",
22              # Linux 优先 Noto, macOS 回落到系统中文字体
23              "font.sans-serif": [
24                  "Noto Sans CJK SC",
25                  "Source Han Sans SC",
26                  "PingFang SC",
27                  "Arial Unicode MS",
28                  "WenQuanYi Zen Hei",
29                  "DejaVu Sans",
30              ],
31              "axes.unicode_minus": False,
32              "font.size": 10.0,
33              "axes.labelsize": 10.0,
34              "axes.titlesize": 10.0,
35              "xtick.labelsize": 9.0,
36              "ytick.labelsize": 9.0,
37              "legend.fontsize": 9.0,
38              "legend.frameon": False,
39              "legend.handlelength": 1.8,
40              "legend.columnspacing": 1.4,
41              "legend.borderaxespad": 0.4,
42              "axes.linewidth": 0.9,
43              "axes.edgecolor": "#2F3640",
44              "axes.labelcolor": "#1F2933",
45              "axes.prop_cycle": cycler(color=PALETTE),
46              "lines.linewidth": 1.7,
47              "lines.markersize": 5.0,
48              "xtick.direction": "out",
49              "ytick.direction": "out",
50              "xtick.major.width": 0.9,
51              "ytick.major.width": 0.9,
52              "xtick.color": "#2F3640",
53              "ytick.color": "#2F3640",
54              "grid.color": "#E5E7EB",
55              "grid.linewidth": 0.6,
56              "grid.alpha": 0.9,
57              "figure.dpi": 160,
58              "savefig.dpi": 400,
59              "savefig.bbox": "tight",
60              "savefig.pad_inches": 0.03,
```

```

61         "pdf.fonttype": 42,
62         "ps.fonttype": 42,
63     }
64 )
65
66
67 def finish_axes(ax, grid_axis: str = "y") -> None:
68     if grid_axis != "none":
69         ax.grid(True, axis=grid_axis)
70     ax.set_axisbelow(True)
71     ax.spines["top"].set_visible(False)
72     ax.spines["right"].set_visible(False)
73
74
75 def add_subpanel_label(ax, label: str, x: float = -0.12, y: float = 1.05) -> None:
76     """Add a compact (a), (b), ... identifier to a multi-panel figure."""
77     ax.text(
78         x,
79         y,
80         f"({label})",
81         transform=ax.transAxes,
82         fontsize=10.5,
83         fontweight="bold",
84         va="bottom",
85         ha="right",
86     )
87
88
89 def headroom(ax, frac: float = 0.18) -> None:
90     """在数据上方留出空白，避免图例或注释压住柱顶与折线。"""
91     lo, hi = ax.get_ylim()
92     ax.set_ylim(lo, lo + (hi - lo) * (1.0 + frac))
93
94
95 def figure_legend(fig, handles, labels, ncol: int, bottom: float = -0.02) -> None:
96     """把多面板共用图例统一放到画布下方，杜绝与数据重叠。"""
97     fig.legend(
98         handles,
99         labels,
100         loc="lower center",
101         ncol=ncol,
102         bbox_to_anchor=(0.5, bottom),
103         frameon=False,
104     )
105
106
107 def note(ax, text: str, loc: str = "upper left", color: str | None = None) -> None:
108     """角落说明文字，带白底以免压住网格与数据。"""
109     anchor = {
110         "upper left": (0.02, 0.98, "left", "top"),
111         "upper right": (0.98, 0.98, "right", "top"),
112         "lower left": (0.02, 0.03, "left", "bottom"),
113         "lower right": (0.98, 0.03, "right", "bottom"),
114     }[loc]
115     ax.text(
116         anchor[0],
117         anchor[1],
118         text,
119         transform=ax.transAxes,
120         ha=anchor[2],
121         va=anchor[3],
122         fontsize=8.0,

```

```

123     color=color or GRAY,
124     bbox={
125         "boxstyle": "round,pad=0.28",
126         "facecolor": "white",
127         "edgecolor": LIGHT_GRAY,
128         "linewidth": 0.6,
129         "alpha": 0.92,
130     },
131 )
132
133
134 def save_figure(fig, out_dir: Path, stem: str) -> None:
135     out_dir.mkdir(parents=True, exist_ok=True)
136     fig.savefig(out_dir / f"{stem}.png")
137     fig.savefig(out_dir / f"{stem}.pdf")
138     plt.close(fig)

```

E.4 compute_frontier.py

```

1  #!/usr/bin/env python3
2  """Compute the scenario emission-power trade-off frontier.
3
4  The support-domain search is repeated at a range of emission limits, so the
5  paper can show the whole trade-off curve rather than only the 10 and
6  5 mg/Nm^3 end points. Because the historical anchor is ~50 mg/Nm^3, every
7  point on this curve costs power relative to historical operation; the curve
8  also records how much voltage headroom above the historical maximum each
9  limit needs before it becomes reachable at all.
10
11  Usage:  python3 src/compute_frontier.py
12  """
13
14  from __future__ import annotations
15
16  from pathlib import Path
17
18  import numpy as np
19  import pandas as pd
20
21  from scenario_model import (
22      DATA_FILE,
23      HEADROOM_5,
24      T_COLS,
25      condition_profiles,
26      evaluate_power_models,
27      scenario_t_star,
28      solve_condition,
29  )
30
31  ROOT = Path(__file__).resolve().parents[1]
32  OUT_DIR = ROOT / "results"
33  LIMITS = [5.0, 6.0, 7.0, 8.0, 9.0, 10.0]
34
35
36  def main() -> None:
37      df = pd.read_csv(DATA_FILE, parse_dates=["timestamp"])
38      df["condition_cluster"] = df["condition_cluster"].astype(int)
39      profiles = condition_profiles(df)
40
41      train = df[df["split"] == "train"].copy()
42      validation = df[df["split"] == "validation"].copy()

```

```

43 test = df[df["split"] == "test"].copy()
44 model, metrics, _, _ = evaluate_power_models(train, validation, test)
45 guard = float(metrics["validation_abs_error_q90_kw"])
46
47 global_t_median = train[T_COLS].median().to_numpy(float)
48 t_stars = {int(p["condition_cluster"]): scenario_t_star(p, global_t_median)
49             for _, p in profiles.iterrows()}
50 historical_weighted = float(np.average(profiles["historical_power_kw"],
51                                       weights=profiles["share"]))
52
53 rows: list[dict[str, object]] = []
54 weighted_rows: list[dict[str, object]] = []
55 for limit in LIMITS:
56     # smallest declared headroom that makes every condition reachable
57     headroom = 0.0
58     while headroom <= HEADROOM_5 + 1e-9:
59         trial = [solve_condition(p, model, t_stars[int(p["condition_cluster"])]), limit,
60                 voltage_headroom=headroom, multistart=12)
61         for _, p in profiles.iterrows():
62             if all(r["status"] == "SCENARIO_FEASIBLE" for r in trial):
63                 break
64             headroom = round(headroom + 0.01, 2)
65
66     solved = [solve_condition(p, model, t_stars[int(p["condition_cluster"])]), limit,
67              voltage_headroom=headroom)
68     for _, p in profiles.iterrows():
69         for profile_row, r in zip(profiles.itertuples(), solved):
70             rows.append({
71                 "condition_cluster": r["condition_cluster"],
72                 "condition_name": r["condition_name"],
73                 "limit_mgNm3": limit,
74                 "voltage_headroom_pct": 100.0 * headroom,
75                 "status": r["status"],
76                 "predicted_power_kw": r.get("predicted_power_kw", np.nan),
77                 "validation_error_adjusted_power_kw": r.get("predicted_power_kw", np.nan) + guard,
78                 "share": float(profile_row.share),
79                 "historical_power_kw": historical_weighted,
80             })
81     weights = np.array([float(p.share) for p in profiles.itertuples()])
82     powers = np.array([float(r.get("predicted_power_kw", np.nan)) for r in solved])
83     weighted_rows.append({
84         "limit_mgNm3": limit,
85         "voltage_headroom_pct": 100.0 * headroom,
86         "weighted_power_kw": float(np.average(powers, weights=weights)),
87         "weighted_daily_energy_MWh": float(np.average(powers, weights=weights)) * 24.0 / 1000.0,
88         "historical_power_kw": historical_weighted,
89     })
90
91 pd.DataFrame(rows).to_csv(OUT_DIR / "emission_power_frontier.csv",
92                        index=False, encoding="utf-8-sig")
93 pd.DataFrame(weighted_rows).to_csv(OUT_DIR / "emission_power_frontier_weighted.csv",
94                                   index=False, encoding="utf-8-sig")
95 print(pd.DataFrame(weighted_rows).to_string(index=False))
96
97
98 if __name__ == "__main__":
99     main()

```

E.5 integrity_check.py

```

1 | #!/usr/bin/env python3

```

```

2  """Deterministic integrity checks for manuscript numbers and artifacts."""
3
4  from __future__ import annotations
5
6  import json
7  import re
8  import sys
9  from pathlib import Path
10
11  import numpy as np
12  import pandas as pd
13
14  SRC = Path(__file__).resolve().parent
15  if str(SRC) not in sys.path:
16      sys.path.insert(0, str(SRC))
17
18  from scenario_model import PhysicalPowerModel, _r2, _rmse
19
20
21  ROOT = SRC.parent
22  DATA = ROOT / "data" / "full_timeseries_with_flags.csv"
23  RESULTS = ROOT / "results"
24  FIGURES = ROOT / "figures_paper"
25  MANUSCRIPT = ROOT / "10_修订后完整论文_终稿.md"
26  OUT = ROOT / "integrity"
27
28
29  def close(a: float, b: float, tol: float = 1e-6) -> bool:
30      return bool(abs(a - b) <= tol)
31
32
33  def main() -> None:
34      checks: List[dict[str, object]] = []
35
36      def check(name: str, passed: bool, actual: object, expected: object) -> None:
37          checks.append({"check": name, "passed": bool(passed), "actual": actual, "expected": expected})
38
39      df = pd.read_csv(DATA)
40      check("raw_row_count", len(df) == 10080, len(df), 10080)
41      check("outlet_missing_count", int(df["C_out_missing_flag"].sum()) == 50, int(df["C_out_missing_flag"].sum()), 50)
42      valid = df["C_out_mgNm3"].dropna()
43      check("outlet_valid_count", len(valid) == 10030, len(valid), 10030)
44      check("outlet_min", close(float(valid.min()), 48.74), float(valid.min()), 48.74)
45      check("outlet_max", close(float(valid.max()), 50.0), float(valid.max()), 50.0)
46      check("outlet_cap50_count", int((valid == 50).sum()) == 5491, int((valid == 50).sum()), 5491)
47      check("outlet_le10_count", int((valid <= 10).sum()) == 0, int((valid <= 10).sum()), 0)
48      check("outlet_le5_count", int((valid <= 5).sum()) == 0, int((valid <= 5).sum()), 0)
49
50      metrics = json.loads((RESULTS / "power_model_metrics.json").read_text())
51      train, val, test = (df[df["split"] == s] for s in ("train", "validation", "test"))
52      model = PhysicalPowerModel().fit(train, train["P_total_kW"].to_numpy(float))
53      vp, tp = model.predict(val), model.predict(test)
54      recomputed = {
55          "validation_r2": _r2(val["P_total_kW"].to_numpy(float), vp),
56          "validation_rmse_kW": _rmse(val["P_total_kW"].to_numpy(float), vp),
57          "retrospective_test_r2": _r2(test["P_total_kW"].to_numpy(float), tp),
58          "retrospective_test_rmse_kW": _rmse(test["P_total_kW"].to_numpy(float), tp),
59          "retrospective_test_bias_kW": float(np.mean(tp - test["P_total_kW"].to_numpy(float))),
60      }
61      for key, value in recomputed.items():
62          check(f"power_metric_{key}", close(value, float(metrics[key]), 1e-9), value, metrics[key])
63      guard = float(np.quantile(np.abs(vp - val["P_total_kW"].to_numpy(float)), 0.90))

```

```

64 check("validation_error_guard_q90", close(guard, float(metrics["validation_abs_error_q90_kW"]), 1e-9), guard, metrics["
↳ validation_abs_error_q90_kW"])
65 check(
66     "reporting_guard_rule_named_consistently",
67     metrics.get("reporting_guard_rule") == "point_prediction_plus_validation_absolute_error_q90"
68     and "conservative_reporting_rule" not in metrics,
69     {k: metrics[k] for k in metrics if "reporting" in k or "guard" in k},
70     {"reporting_guard_rule": "point_prediction_plus_validation_absolute_error_q90"},
71 )
72 check(
73     "power_model_is_physical_specification",
74     metrics["model"] == "physical_voltage_square_plus_rapping_frequency",
75     metrics["model"], "physical_voltage_square_plus_rapping_frequency",
76 )
77 # the four field coefficients must be near-identical: the fields share a design
78 b = np.asarray(metrics["b_field_kW_per_kV2"], float)
79 check("field_coefficients_consistent", float(b.std() / b.mean()) < 0.01,
80     float(b.std() / b.mean()), "<1% relative spread")
81
82 baselines = pd.read_csv(RESULTS / "power_model_baselines.csv")
83 tst_base = baselines[baselines["split"] == "retrospective_test"].set_index("model")
84 check(
85     "invT_beats_linear_T_on_test",
86     float(tst_base.loc["physical_invT", "rmse_kW"]) < float(tst_base.loc["quadratic_ridge_T", "rmse_kW"]),
87     tst_base["rmse_kW"].to_dict(), "physical_invT < quadratic_ridge_T",
88 )
89 check(
90     "invT_removes_test_bias",
91     abs(float(tst_base.loc["physical_invT", "bias_kW"])) < 1.0
92     and abs(float(tst_base.loc["quadratic_ridge_T", "bias_kW"])) > 5.0,
93     {"physical_invT": float(tst_base.loc["physical_invT", "bias_kW"]),
94     "quadratic_ridge_T": float(tst_base.loc["quadratic_ridge_T", "bias_kW"])},
95     "|bias| < 1 kW with 1/T, > 5 kW with linear T",
96 )
97 check(
98     "condition_variables_add_nothing",
99     abs(float(tst_base.loc["physical_invT_plus_conditions", "r2"])
100         - float(tst_base.loc["physical_invT", "r2"])) < 1e-3,
101     {"with": float(tst_base.loc["physical_invT_plus_conditions", "r2"]),
102     "without": float(tst_base.loc["physical_invT", "r2"])},
103     "delta R2 < 1e-3",
104 )
105 rolling = pd.read_csv(RESULTS / "power_model_rolling_validation.csv")
106 check("rolling_fold_count", rolling["fold"].nunique() == 3, int(rolling["fold"].nunique()), 3)
107
108 cluster = pd.read_csv(RESULTS / "cluster_selection_metrics.csv")
109 selected = cluster[cluster["selected"]].iloc[0]
110 check("selected_cluster_k", int(selected["k"]) == 4, int(selected["k"]), 4)
111 check("silhouette_k4", close(float(selected["silhouette"]), 0.4083689, 1e-6), float(selected["silhouette"]), 0.4083689)
112
113 optimum = pd.read_csv(RESULTS / "optimal_controls_central_scenario.csv")
114 check(
115     "central_parameter_table_complete",
116     len(optimum) == 8
117     and set(map(tuple, optimum[["condition_cluster", "limit_mgNm3"]].to_numpy()))
118     == {(cluster, limit) for cluster in range(1, 5) for limit in (10.0, 5.0)},
119     optimum[["condition_cluster", "limit_mgNm3"]].values.tolist(),
120     "4 conditions × 2 targets",
121 )
122 check(
123     "validation_error_columns_named_consistently",
124     {"validation_error_adjusted_power_kW", "validation_error_guard_kW"}.issubset(optimum.columns)

```

```

125     and {"conservative_power_kw", "power_guard_kw"}.isdisjoint(optimum.columns),
126     sorted(optimum.columns),
127     "new validation-error names present; old conservative names absent",
128 )
129 check("central_all_feasible", bool((optimum["status"] == "SCENARIO_FEASIBLE").all()), optimum["status"].unique().tolist(), ["
↳ SCENARIO_FEASIBLE"])
130 check("central_limits_respected", bool((optimum["scenario_peak_total_mgNm3"] <= optimum["limit_mgNm3"] + 1e-9).all()), float((optimum["
↳ scenario_peak_total_mgNm3"] - optimum["limit_mgNm3"]).max()), "<=0")
131 check(
132     "emission_anchor_not_rescaled",
133     bool((pd.read_csv(RESULTS / "condition_profiles.csv")["C_out_recorded_median"] == 50.0).all()),
134     pd.read_csv(RESULTS / "condition_profiles.csv")["C_out_recorded_median"].tolist(),
135     "all 50.0 mg/Nm3 (recorded, unscaled)",
136 )
137 scenario = json.loads((RESULTS / "scenario_parameters.json").read_text())
138 check("no_outlet_rescaling_parameter", "central_outlet_scale" not in scenario,
139     sorted(scenario), "no central_outlet_scale key")
140
141 audit = pd.read_csv(RESULTS / "support_boundary_audit.csv")
142 check("support_audit_rows", len(audit) == 8, len(audit), 8)
143 check(
144     "targets_leave_historical_experience",
145     bool((audit["ratio_to_q975"] > 1.0).all()),
146     float(audit["ratio_to_q975"].min()), ">1 for all eight solutions",
147 )
148 check(
149     "10mg_inside_voltage_envelope",
150     bool((audit[audit["limit_mgNm3"] == 10.0]["max_voltage_over_historical_max_pct"] <= 1e-6).all()),
151     float(audit[audit["limit_mgNm3"] == 10.0]["max_voltage_over_historical_max_pct"].max()),
152     "<=0% over the historical maximum",
153 )
154 restricted = pd.read_csv(RESULTS / "mahalanobis_constrained_feasibility.csv")
155 merged = restricted.merge(
156     optimum[["condition_cluster", "limit_mgNm3", "predicted_power_kw"]],
157     on=["condition_cluster", "limit_mgNm3"], suffixes=("_shell", "_free"))
158 feasible_shell = merged["status_with_mahalanobis_constraint"] == "SCENARIO_FEASIBLE"
159 check(
160     "mahalanobis_shell_variant_mostly_feasible",
161     int(feasible_shell.sum()) == 7,
162     int(feasible_shell.sum()), "7 of 8 combinations feasible inside the shell",
163 )
164 penalty = 100.0 * (merged.loc[feasible_shell, "predicted_power_kw_shell"]
165     / merged.loc[feasible_shell, "predicted_power_kw_free"] - 1.0)
166 check(
167     "shell_variant_costs_power",
168     bool((penalty > 0).all()),
169     [round(float(penalty.min()), 2), round(float(penalty.max()), 2)],
170     "staying inside the shell is never cheaper than the free optimum",
171 )
172 headroom = pd.read_csv(RESULTS / "voltage_headroom_requirement.csv")
173 check(
174     "headroom_10mg_is_zero",
175     bool((headroom[headroom["limit_mgNm3"] == 10.0]["min_voltage_headroom_pct"] == 0.0).all()),
176     headroom[headroom["limit_mgNm3"] == 10.0]["min_voltage_headroom_pct"].tolist(), "all 0%",
177 )
178 check(
179     "headroom_5mg_within_declared_margin",
180     float(headroom[headroom["limit_mgNm3"] == 5.0]["min_voltage_headroom_pct"].max()) <= 5.0,
181     float(headroom[headroom["limit_mgNm3"] == 5.0]["min_voltage_headroom_pct"].max()), "<=5%",
182 )
183
184 q4 = pd.read_csv(RESULTS / "question4_by_condition.csv")

```



```

185 p10 = float(np.sum(q4["share"] * q4["power_10_kw"]) / q4["share"].sum())
186 p5 = float(np.sum(q4["share"] * q4["power_5_kw"]) / q4["share"].sum())
187 increase = 100 * (p5 / p10 - 1)
188 summary = json.loads((RESULTS / "question4_summary.json").read_text())
189 check("weighted_power_10", close(p10, summary["weighted_power_10_kw"], 1e-9), p10, summary["weighted_power_10_kw"])
190 check("weighted_power_5", close(p5, summary["weighted_power_5_kw"], 1e-9), p5, summary["weighted_power_5_kw"])
191 check("weighted_increase", close(increase, summary["weighted_increase_pct"], 1e-9), increase, summary["weighted_increase_pct"])
192 check(
193     "tightening_costs_power_vs_history",
194     summary["power_10_vs_historical_pct"] > 0 and summary["weighted_increase_pct"] > 0,
195     {"10mg_vs_history_pct": summary["power_10_vs_historical_pct"],
196      "increase_pct": summary["weighted_increase_pct"]},
197     "both positive: tightening the limit costs power",
198 )
199 rel = np.abs(q4["analytic_delta_power_kw"] - q4["numeric_delta_power_kw"]) / q4["numeric_delta_power_kw"]
200 check(
201     "analytic_matches_numeric_delta_power",
202     float(rel.max()) < 0.02, float(rel.max()), "<2% relative difference",
203 )
204
205 verification = pd.read_csv(RESULTS / "optimization_verification.csv")
206 check("verification_rows", len(verification) == 8, len(verification), 8)
207 check(
208     "multistart_agrees_on_optimum",
209     float(verification["multistart_power_spread_kw"].max()) < 1.0,
210     float(verification["multistart_power_spread_kw"].max()), "<1 kW across converged starts",
211 )
212 gaps = verification["random_search_gap_pct"].dropna()
213 check(
214     "random_search_finds_nothing_better",
215     bool((gaps >= -1e-9).all()), float(gaps.min()), ">=0 for every combination",
216 )
217
218 sens = pd.read_csv(RESULTS / "question4_sensitivity.csv")
219 check("sensitivity_grid_size", len(sens) == 27, len(sens), 27)
220 p_ref = pd.read_csv(RESULTS / "p_exponent_reference.csv")
221 check("p_reference_rows", len(p_ref) == 5, len(p_ref), 5)
222 check(
223     "p_reference_monotone_decreasing",
224     bool((p_ref.sort_values("p_exponent")["analytic_increase_pct"].diff().dropna() < 0).all()),
225     p_ref.sort_values("p_exponent")["analytic_increase_pct"].round(3).tolist(),
226     "increase falls as p rises (proportional to 2/p)",
227 )
228
229 structural = pd.read_csv(RESULTS / "structural_sensitivity.csv")
230 structural_vals = structural.loc[structural["all_conditions_feasible"] == True, "weighted_power_increase_pct"]
231 check("structural_total_count", len(structural) == 81, len(structural), 81)
232 check("structural_all_feasible", len(structural_vals) == 81, len(structural_vals), 81)
233
234 rapping = pd.read_csv(RESULTS / "rapping_power_tradeoff.csv")
235 check(
236     "rapping_share_reported",
237     30.0 < float(rapping.iloc[0]["rapping_share_pct"]) < 45.0,
238     float(rapping.iloc[0]["rapping_share_pct"]), "between 30% and 45% at median settings",
239 )
240 strategy = pd.read_csv(RESULTS / "historical_strategy_correlations.csv")
241 check(
242     "strategy_windows_reported",
243     set(strategy["window"]) == {"full_record", "train", "validation", "retrospective_test"},
244     sorted(set(strategy["window"])),
245     "four evaluation windows",
246 )

```

```

247 period_rule = strategy[strategy["control"].str.startswith("T")]
248 check(
249     "rapping_period_rule_stable_across_windows",
250     bool((period_rule["r_C_in_gNm3"] < 0).all()),
251     float(period_rule["r_C_in_gNm3"].max()), "<0 in every window and field",
252 )
253 events = pd.read_csv(RESULTS / "anomaly_events.csv")
254 check("anomaly_events_documented", len(events) == 2, len(events), 2)
255
256 manuscript = MANUSCRIPT.read_text(encoding="utf-8")
257 # A final submission may print this verifier itself in Appendix D. Restrict
258 # manuscript-content checks to the paper and non-code appendices so string
259 # literals inside the printed source do not become false positives.
260 review_manuscript = manuscript.split("## 附录D 完整源程序", 1)[0]
261 ref_body = review_manuscript.split("## 参考文献", 1)[0]
262 cited = set()
263 for token in re.findall(r"\[[0-9,-]+\]", ref_body):
264     for part in token.split(","):
265         if "-" in part:
266             a, b = map(int, part.split("-"))
267             cited.update(range(a, b + 1))
268         else:
269             cited.add(int(part))
270 listed = set(map(int, re.findall(r"^[0-9]+\]", review_manuscript.split("## 参考文献", 1)[1], flags=re.M)))
271 check("all_references_cited", cited == set(range(1, 14)), sorted(cited), list(range(1, 14)))
272 check("reference_list_complete", listed == set(range(1, 14)), sorted(listed), list(range(1, 14)))
273 clean_manuscript = re.sub(r"<!--.*?-->\n?", "", review_manuscript)
274 table6_segment = clean_manuscript.split("表10\u3000四工况两档目标", 1)[1].split("表11", 1)[0]
275 table6_rows: dict[tuple[int, float], list[float]] = {}
276 for line in table6_segment.splitlines():
277     if not re.match(r"^[ 1-4] \[ (?:(10|5) \|", line):
278         continue
279     cells = [cell.strip() for cell in line.strip().strip("|").split("|")]
280     table6_rows[(int(cells[0]), float(cells[1]))] = [float(x) for x in cells[2:]]
281 table6_matches = len(table6_rows) == 8
282 for key, displayed in table6_rows.items():
283     expected_row = optimum[
284         (optimum["condition_cluster"] == key[0]) & (optimum["limit_mgNm3"] == key[1])
285     ]
286     if len(expected_row) != 1:
287         table6_matches = False
288         continue
289     row = expected_row.iloc[0]
290     expected = [
291         *round(float(row[f"U{i}_kv"]), 1) for i in range(1, 5),
292         *round(float(row[f"T{i}_s"]), 0) for i in range(1, 5),
293         round(float(row["scenario_peak_total_mgNm3"]), 3),
294         round(float(row["predicted_power_kw"]), 2),
295     ]
296     table6_matches &= bool(np.allclose(displayed, expected, atol=5e-3))
297 check(
298     "table6_complete_and_matches_results",
299     table6_matches,
300     {f"condition_{key[0]}_target_{key[1]:g}": value for key, value in table6_rows.items()},
301     "8 rows matching optimal_controls_central_scenario.csv after displayed rounding",
302 )
303 abstract = clean_manuscript.split("## 摘要", 1)[1].split("***关键词", 1)[0]
304 abstract_chars = len(re.sub(r"\s+", "", abstract))
305 check("abstract_fills_one_page", 700 <= abstract_chars <= 1300, abstract_chars, "700-1300 chars (one page)")
306 check("anonymous_manuscript", "作者: " not in clean_manuscript and "学校: " not in clean_manuscript, ["作者: " in clean_manuscript, "
307     ↵ 学校: " in clean_manuscript], [False, False])
308 check("no_english_abstract", "## Abstract" not in clean_manuscript, "## Abstract" in clean_manuscript, False)

```

```

308 check("outlet_rescaling_claim_removed", "乘0.10" not in clean_manuscript and "0.1缩放" not in clean_manuscript, ["乘0.10" in
    ⇨ clean_manuscript, "0.1缩放" in clean_manuscript], [False, False])
309 check("table_caption_count", len(re.findall(r"^\s*表\d+", clean_manuscript, flags=re.M)) == 14, len(re.findall(r"^\s*表\d+", clean_manuscript,
    ⇨ flags=re.M)), 14)
310 check("manuscript_figure_count", len(re.findall(r"^\s*\[图", clean_manuscript, flags=re.M)) == 21, len(re.findall(r"^\s*\[图",
    ⇨ clean_manuscript, flags=re.M)), 21)
311 check("ai_disclosure_present", "AI工具使用声明" in clean_manuscript, "AI工具使用声明" in clean_manuscript, True)
312 check("support_list_exists", (ROOT / "支撑材料文件清单.md").exists(), (ROOT / "支撑材料文件清单.md").exists(), True)
313 ai_detail = next(ROOT.parent.glob("**/AI工具使用详情*.md"), None)
314 check("ai_detail_exists", ai_detail is not None, str(ai_detail), "AI工具使用详情*.md present in the process directory")
315
316 manifest = pd.read_csv(FIGURES / "figure_manifest.csv")
317 check("figure_manifest_count", len(manifest) == 21, len(manifest), 21)
318 required_trace = {"source_data", "transformation", "caption", "supported_manuscript_claims", "limitations"}
319 check("figure_trace_fields", required_trace.issubset(manifest.columns), sorted(manifest.columns), sorted(required_trace))
320 missing_artifacts = []
321 for _, row in manifest.iterrows():
322     for col in ("png", "pdf", "source_data"):
323         artifact = Path(row[col])
324         if not artifact.is_absolute():
325             artifact = ROOT / artifact
326         if not artifact.exists():
327             missing_artifacts.append(f"{row['stem']}:{col}")
328 check("figure_artifacts_and_sources_exist", not missing_artifacts, missing_artifacts, [])
329
330 OUT.mkdir(parents=True, exist_ok=True)
331 failed = [x for x in checks if not x["passed"]]
332 report = {"status": "PASS" if not failed else "FAIL", "total_checks": len(checks), "passed": len(checks)-len(failed), "failed": len(
    ⇨ failed), "checks": checks}
333 (OUT / "deterministic_integrity_results.json").write_text(json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8")
334 print(json.dumps({k: report[k] for k in ("status", "total_checks", "passed", "failed")}, ensure_ascii=False, indent=2))
335 if failed:
336     for item in failed:
337         print("FAILED", item)
338     raise SystemExit(1)
339
340
341 if __name__ == "__main__":
342     main()

```

E.6 build_paper_pdf.py

```

1  #!/usr/bin/env python3
2  """Build the CUMCM submission PDF from the final Markdown manuscript.
3
4  Converts ``10_修订后完整论文_终稿.md`` into XeLaTeX and compiles it.
5  Layout follows the CUMCM paper format specification: A4, >=2.5 cm margins,
6  abstract alone on page 1, continuous arabic page numbers centred in the footer.
7
8  Usage: python3 src/build_paper_pdf.py [--no-code | --core-code]
9  """
10
11 from __future__ import annotations
12
13 import re
14 import subprocess
15 import sys
16 from pathlib import Path
17
18 ROOT = Path(__file__).resolve().parent.parent
19 MANUSCRIPT = ROOT / "10_修订后完整论文_终稿.md"

```

```

20 BUILD = ROOT / "build"
21 SRC = ROOT / "src"
22 CODE_MODULES = ["scenario_model.py", "paper_figures.py", "plot_utils.py",
23                 "compute_frontier.py", "integrity_check.py", "build_paper_pdf.py"]
24
25 # ----- inline text
26
27 _SPECIALS = {
28     "\\": r"\textbackslash{}", "&": r"&", "%": r"%", "$": r"$",
29     "#": r"#", "_": r"_", "{": r"{", "}" : r"}",
30     "~": r"\textasciitilde{}", "^": r"\textasciicircum{}",
31     # symbols the CJK/serif fonts do not carry -> render them as maths
32     "\u2103": r"$^{\circ}$", "\u221d": r"$\propto$", "\u2260": r"$\neq$",
33     "\u2264": r"$\le$", "\u2265": r"$\ge$", "\u2248": r"$\approx$",
34     "\u207b": r"$^{-}$", "\u00b9": r"$^1$", "\u2070": r"$^0$",
35     "\u2192": r"$\rightarrow$", "\u2191": r"$\uparrow$", "\u2193": r"$\downarrow$",
36 }
37
38
39 def esc(text: str) -> str:
40     return "".join(_SPECIALS.get(ch, ch) for ch in text)
41
42
43 #: bare links and DOIs are single unbreakable words for TeX; \url can break them
44 _URL_RE = re.compile(r"(https?://\S+)?(?:=[\s\u3002\u201c\u201d]|(?:\u2190\u2191\u2193)|\u2192)|(?<DOI: )(\d{10}\.\S+)?(?:=[\s\u3002\u201c\u201d]|$)")
45
46
47 def _urlify(piece: str) -> str:
48     r"""Escape text, but wrap any bare URL or DOI in a breakable url command."""
49     out: list[str] = []
50     last = 0
51     for m in _URL_RE.finditer(piece):
52         out.append(esc(piece[last:m.start()]))
53         link = m.group(0).rstrip(".")
54         trailing = m.group(0)[len(link):]
55         out.append(r"\url{" + link + "}" + esc(trailing))
56         last = m.end()
57     out.append(esc(piece[last:]))
58     return "".join(out)
59
60
61 #: Markdown bold may wrap a maths span (***若把$x$作为约束***), so the markers
62 #: cannot be resolved inside a single plain-text piece. They are replaced by a
63 #: sentinel first and paired up after every segment has been rendered.
64 _BOLD = "\x00"
65
66
67 def _smart_quotes(piece: str) -> str:
68     """Straight double quotes in Chinese prose -> paired curly quotes."""
69     out = []
70     open_next = True
71     for ch in piece:
72         if ch == '"':
73             out.append("\u201c" if open_next else "\u201d")
74             open_next = not open_next
75         else:
76             out.append(ch)
77     return "".join(out)
78
79
80 def inline(text: str) -> str:
81     """Convert inline Markdown to LaTeX, leaving $...$ math untouched."""

```

```

82 out: list[str] = []
83 for i, seg in enumerate(re.split(r"(\${^$}*\$)", text)):
84     if i % 2:
85         # math segment - pass through verbatim
86         out.append(seg)
87         continue
88     # code spans next: their content must be escaped but not styled
89     for j, piece in enumerate(re.split(r"('[^']*')", seg)):
90         if j % 2:
91             out.append(r"\texttt{" + esc(piece[1:-1]) + "}")
92         else:
93             piece = piece.replace("'''", _BOLD)
94             out.append(_urlify(_smart_quotes(piece)))
95 rendered = "".join(out)
96
97 # pair the sentinels; an unmatched trailing marker is dropped
98 parts = rendered.split(_BOLD)
99 if len(parts) == 1:
100     return rendered
101 result = parts[0]
102 for k, chunk in enumerate(parts[1:], start=1):
103     result += (r"\textbf{" if k % 2 else "}") + chunk
104 if len(parts) % 2 == 0:
105     # odd number of markers
106     result += "}"
107 return result
108
109 # ----- block level
110
111 def _display_width(text: str) -> int:
112     """Rough typeset width in en-units (CJK glyphs are double width)."""
113     text = re.sub(r"(\${^$}*\$)", "xx", text) # math sets compactly
114     text = re.sub(r"['"]", "", text)
115     return sum(2 if ord(ch) > 0x2E80 else 1 for ch in text)
116
117 # A row of ``l`` columns cannot wrap, so any table whose natural width exceeds
118 # the text block silently runs off the page. Text-heavy columns therefore
119 # become wrapping ``X`` columns and the table is set to exactly \textwidth.
120 _LINE_BUDGET = 84 # en-units that fit across the 16 cm text block
121 _WRAP_THRESHOLD = 16 # a column wider than this must be allowed to wrap
122
123
124 def convert_table(header: list[str], align: list[str], rows: list[list[str]],
125                  number: str, title: str) -> str:
126     ncol = len(header)
127     widths = [max(_display_width(header[j]),
128                  + _display_width(r[j]) for r in rows if j < len(r))]
129     for j in range(ncol):
130         natural = sum(widths) + 2 * ncol
131
132     wrappable = [j for j in range(ncol)
133                  if align[j] == "l" and widths[j] > _WRAP_THRESHOLD]
134     use_tabularx = natural > _LINE_BUDGET and bool(wrappable)
135
136     if use_tabularx:
137         # Share the wrapping width between the text columns in proportion to
138         # how much text each of them actually carries.
139         # Proportional-to-length shares starve short-but-not-short-enough
140         # columns, so damp the ratio with a square root and floor it.
141         raw = {j: max(widths[j], 1) ** 0.5 for j in wrappable}
142         total_wrap = sum(raw.values())
143         shares = {j: max(len(wrappable) * raw[j] / total_wrap, 0.55) for j in wrappable}

```

```

144     scale = len(wrappable) / sum(shares.values())
145     parts = []
146     for j in range(ncol):
147         if j in wrappable:
148             parts.append(r">{\hsize=3f\hsize\raggedright\arraybackslash}X"
149                           % (shares[j] * scale))
150         else:
151             parts.append({"r": "r", "c": "c"}.get(align[j], "l"))
152     open_env = r"\begin{tabularx}{\textwidth}{%}" + "".join(parts) + "}"
153     close_env = r"\end{tabularx}"
154     shrink = False
155 else:
156     spec = "".join({"r": "r", "c": "c"}.get(a, "l") for a in align)
157     open_env = r"\begin{tabular}{%}" + spec + "}"
158     close_env = r"\end{tabular}"
159     shrink = natural > _LINE_BUDGET      # all-numeric wide table: scale it
160
161     lines = [r"\begin{table}[htbp]", r"\centering",
162              r"\caption*{\normalsize 表" + number + "\u3000" + inline(title) + "}",
163              r"\vspace{-2pt}", r"\zihao{-5}"]
164     if shrink:
165         lines.append(r"\resizebox{\textwidth}{!}{%}")
166     lines += [open_env, r"\toprule",
167              " & ".join(inline(h) for h in header) + r" \\", r"\midrule"]
168     for row in rows:
169         lines.append(" & ".join(inline(c) for c in row) + r" \\")
170     lines += [r"\bottomrule", close_env]
171     if shrink:
172         lines.append("}")
173     lines.append(r"\end{table}")
174     return "\n".join(lines)
175
176
177 def parse_align(sep_row: list[str]) -> list[str]:
178     out = []
179     for cell in sep_row:
180         cell = cell.strip()
181         if cell.startswith(":") and cell.endswith(":"):
182             out.append("c")
183         elif cell.endswith(":"):
184             out.append("r")
185         else:
186             out.append("l")
187     return out
188
189
190 def split_row(line: str) -> list[str]:
191     return [c.strip() for c in line.strip().strip("|").split("|")]
192
193
194 def convert(md: str) -> tuple[str, str, str]:
195     """Return (title, abstract_block, body)."""
196     md = re.sub(r"<!--.*?-->", "", md, flags=re.S)
197     lines = md.split("\n")
198
199     title = ""
200     abstract: list[str] = []
201     body: list[str] = []
202     target = body
203
204     i = 0
205     pending_table: tuple[str, str] | None = None # (number, title)

```

```

206 while i < len(lines):
207     line = lines[i].rstrip()
208
209     # ---- headings
210     m = re.match(r"^# (.+)$", line)
211     if m:
212         title = m.group(1).strip()
213         i += 1
214         continue
215     m = re.match(r"^## (.+)$", line)
216     if m:
217         head = m.group(1).strip()
218         if head == "摘要":
219             target = abstract
220             i += 1
221             continue
222         target = body
223         hm = re.match(r"^(\\d+)\\s+(.*)$", head)
224         if hm:
225             target.append(r"\section{" + inline(hm.group(2)) + "}")
226         else:
227             target.append(r"\section*" + inline(head) + "}")
228             target.append(r"\addcontentsline{toc}{section}" + inline(head) + "}")
229         i += 1
230         continue
231     m = re.match(r"^### (.+)$", line)
232     if m:
233         head = m.group(1).strip()
234         hm = re.match(r"^(\\d+|[A-Z]\\d+)\\s+(.*)$", head)
235         target.append(r"\subsection*" + inline(head if not hm else head) + "}")
236         i += 1
237         continue
238
239     # ---- fenced code
240     if line.startswith("```"):
241         j = i + 1
242         buf = []
243         while j < len(lines) and not lines[j].startswith("```"):
244             buf.append(lines[j])
245             j += 1
246         target.append(r"\begin{lstlisting}[style=pseudo]")
247         target.extend(buf)
248         target.append(r"\end{lstlisting}")
249         i = j + 1
250         continue
251
252     # ---- display math
253     if line.strip() == "$$:
254         j = i + 1
255         buf = []
256         while j < len(lines) and lines[j].strip() != "$$:
257             buf.append(lines[j])
258             j += 1
259         expr = "\n".join(buf).strip().rstrip(",.")
260         target.append(r"\begin{equation}")
261         target.append(expr)
262         target.append(r"\end{equation}")
263         i = j + 1
264         continue
265
266     # ---- image -> figure float
267     m = re.match(r"^[图](\\d+)\\s*([\\]]*)\\((\\[\\])+)\\s*$", line)

```

```

268     if m:
269         num, alt, path = m.group(1), m.group(2), m.group(3)
270         cap = alt
271         k = i + 1
272         while k < len(lines) and not lines[k].strip():
273             k += 1
274         cm = re.match(r"^\u3000(.+)\$" % num, lines[k].strip()) if k < len(lines) else None
275         if cm:
276             cap = cm.group(1)
277             i = k
278         width = "0.96" if path.endswith("00_esp_schematic.pdf") else "0.92"
279         target += [r"\begin{figure}[htbp]", r"\centering",
280                  r"\includegraphics[width=\textwidth]{%s}" % (width, path),
281                  r"\caption*\normalsize 图" + num + "\u3000" + inline(cap) + "}",
282                  r"\end{figure}"]
283         i += 1
284         continue
285
286     # ---- standalone figure caption already consumed above; skip strays
287     if re.match(r"^\u3000\d+\u3000", line.strip()):
288         i += 1
289         continue
290
291     # ---- table title line
292     m = re.match(r"^\u3000表(\d+|A\d+)\u3000(.+)\$", line.strip())
293     if m:
294         pending_table = (m.group(1), m.group(2))
295         i += 1
296         continue
297
298     # ---- table body
299     if line.strip().startswith("|"):
300         header = split_row(line)
301         align = parse_align(split_row(lines[i + 1]))
302         rows = []
303         j = i + 2
304         while j < len(lines) and lines[j].strip().startswith("|"):
305             rows.append(split_row(lines[j]))
306             j += 1
307         num, ttl = pending_table if pending_table else ("", "")
308         target.append(convert_table(header, align, rows, num, ttl))
309         pending_table = None
310         i = j
311         continue
312
313     # ---- blockquote
314     if line.startswith(">"):
315         buf = [line.lstrip("> ").strip()]
316         j = i + 1
317         while j < len(lines) and lines[j].startswith(">"):
318             buf.append(lines[j].lstrip("> ").strip())
319             j += 1
320         target.append(r"\begin{quote}\small " + inline(" ".join(buf)) + r"\end{quote}")
321         i = j
322         continue
323
324     # ---- ordered / unordered list
325     if re.match(r"^\d+\.\s+", line) or re.match(r"^[*-]\s+", line):
326         ordered = bool(re.match(r"^\d+\.\s+", line))
327         env = "enumerate" if ordered else "itemize"
328         items = []
329         j = i

```



```

330     while j < len(lines) and (re.match(r"^\d+\.\s+", lines[j]) or re.match(r"^[^]\s+", lines[j]]):
331         items.append(re.sub(r"^\d+\.\s+", "", lines[j].rstrip()))
332         j += 1
333     target.append(r"\begin{%s}[itemsep=1pt,topsep=3pt]" % env)
334     target += [r"\item " + inline(it) for it in items]
335     target.append(r"\end{%s}" % env)
336     i = j
337     continue
338
339     # ---- blank
340     if not line.strip():
341         target.append("")
342         i += 1
343         continue
344
345     # ---- plain paragraph
346     target.append(inline(line.strip()))
347     i += 1
348
349     return title, "\n".join(abstract).strip(), "\n".join(body).strip()
350
351
352 # ----- preamble
353
354 PREAMBLE = r"""
355 \documentclass[UTF8,zihao=5,a4paper]{ctexart}
356 \usepackage[a4paper,top=2.5cm,bottom=2.5cm,left=2.5cm,right=2.5cm]{geometry}
357 \usepackage{amsmath,amssymb,bm}
358 \usepackage{graphicx}
359 \usepackage{booktabs}
360 \usepackage{caption}
361 \usepackage{enumitem}
362 \usepackage{listings}
363 \usepackage{xcolor}
364 \usepackage{setspace}
365 \usepackage{array}
366 \usepackage{tabularx}
367 \usepackage{fancyhdr}
368 \usepackage{xurl}
369
370 %%FONTS%%
371
372 \onehalfspacing
373 \setlength{\parindent}{2em}
374 \captionsetup{skip=4pt}
375 \emergencystretch=3em
376 \sloppy
377
378 \pagestyle{fancy}
379 \fancyhf{}
380 \renewcommand{\headrulewidth}{0pt}
381 \fancyfoot[C]{\thepage}
382
383 \ctexset{
384     section/format = {\raggedright\zihao{4}\bfseries},
385     subsection/format = {\raggedright\zihao{5}\bfseries},
386     section/beforekip = 12pt plus 2pt minus 2pt,
387     section/afterskip = 8pt,
388 }
389
390 \lstdefinestyle{pseudo}{
391     basicstyle=\ttfamily\zihao{-5},

```

```

392 breaklines=true, breakatwhitespace=false,
393 columns=fullflexible, keepspaces=true,
394 frame=single, rulecolor=\color{gray!55},
395 framesep=4pt, xleftmargin=6pt, xrightmargin=2pt,
396 showstringspaces=false, extendedchars=true,
397 }
398 \lstdefinestyle{pycode}{
399 language=Python,
400 basicstyle=\ttfamily\zihao{6},
401 keywordstyle=\color{blue!65!black},
402 commentstyle=\color{green!42!black}\itshape,
403 stringstyle=\color{orange!75!black},
404 breaklines=true, breakatwhitespace=false,
405 breakindent=12pt, postbreak=\mbox{\textcolor{gray}{$\hookrightarrow$}}\space,
406 columns=fullflexible, keepspaces=true,
407 showstringspaces=false, extendedchars=true,
408 numbers=left, numberstyle=\tiny\color{gray}, numbersep=6pt,
409 frame=leftline, rulecolor=\color{gray!45}, framesep=5pt,
410 xleftmargin=17pt, tabsize=4, upquote=true,
411 }
412
413 % 手工编号的图表题注（与正文交叉引用一致）
414 \captionsetup[figure]{labelformat=empty,justification=centering}
415 \captionsetup[table]{labelformat=empty,justification=centering}
416 ""
417
418
419
420 # ----- fonts
421
422 #: Preferred first, portable fallback second. The submission machine normally
423 #: has TeX Gyre Termes and Noto CJK; a bare TeX Live may not.
424 FONT_CHOICES = {
425     "main": ["TeX Gyre Termes", "Liberation Serif", "DejaVu Serif"],
426     "mono": ["Noto Sans Mono CJK SC", "DejaVu Sans Mono"],
427     "cjkmain": ["Noto Serif CJK SC", "Fandol Song", "Noto Sans CJK SC"],
428     "cjksans": ["Noto Sans CJK SC", "Fandol Hei"],
429     "cjkmono": ["Noto Sans Mono CJK SC", "Noto Sans CJK SC"],
430 }
431
432
433 def _installed_families() -> set[str]:
434     try:
435         out = subprocess.run(["fc-list", ":", "family"], capture_output=True,
436                             text=True, check=True).stdout
437     except (OSError, subprocess.CalledProcessError):
438         return set()
439     return {name.strip() for line in out.splitlines() for name in line.split(",")}
440
441
442 def font_block() -> str:
443     available = _installed_families()
444
445     def pick(key: str) -> str:
446         for name in FONT_CHOICES[key]:
447             if not available or name in available:
448                 return name
449         return FONT_CHOICES[key][-1]
450
451     return "\n".join([
452         r"\setCJKmainfont{%s}" % pick("cjkmain"),
453         r"\setCJKsansfont{%s}" % pick("cjksans"),

```

```

454     r"\setCJKmonofont{%s}" % pick("cjkmono"),
455     r"\setmainfont{%s}" % pick("main"),
456     r"\setmonofont{%s}[Scale=0.82]" % pick("mono"),
457 ])
458
459
460 def build_tex(title: str, abstract: str, body: str, code: str) -> str:
461     parts = [PREAMBLE.replace("%FONTS%", font_block()), r"\begin{document}", ""]
462     # --- page 1: title + abstract only
463     parts += [
464         r"\begin{center}",
465         r"{\zhao{3}\bfseries " + inline(title) + r"}",
466         r"\end{center}",
467         r"\vspace{6pt}",
468         r"\begin{center}{\zhao{4}\bfseries 摘\quad 要}\end{center}",
469         r"\vspace{2pt}",
470     ]
471     kw = ""
472     keep = []
473     for l in abstract.split("\n"): # 保留空行以维持摘要的分段结构
474         if l.startswith(r"\textbf{关键词: }"):
475             kw = l
476         else:
477             keep.append(l)
478     while keep and not keep[-1].strip():
479         keep.pop()
480     parts += keep
481     if kw:
482         parts += [r"\vspace{8pt}", "", r"\noindent " + kw]
483     parts += [r"\clearpage", ""]
484     parts.append(body)
485     if code:
486         parts.append(code)
487     parts += [r"\end{document}", ""]
488     return "\n".join(parts)
489
490
491 def _extract(module: str, names: list[str]) -> str:
492     """Pull whole top-level definitions out of a source file, in order."""
493     text = (SRC / module).read_text(encoding="utf-8").split("\n")
494     out: list[str] = []
495     for name in names:
496         try:
497             i = next(k for k, l in enumerate(text)
498                     if l.startswith(("def " + name + "(", "class " + name)))
499         except StopIteration:
500             continue
501         j = i + 1
502         while j < len(text) and (not text[j].strip() or text[j][:1] in " )}\t"):
503             j += 1
504         while j > i and not text[j - 1].strip():
505             j -= 1
506         out.extend(text[i:j] + [""])
507     return "\n".join(out).rstrip() + "\n"
508
509
510 #: Appendix E carries the full listing by default; ``--core-code`` prints only
511 #: the two definitions that determine every number in the paper.
512 CORE_EXTRACTS = [
513     ("scenario_model.py", ["PhysicalPowerModel"]),
514     ("scenario_model.py", ["scenario_emission", "field_priority"]),
515 ]

```

```

516
517
518 def code_appendix(full: bool = True) -> str:
519     title = "\u9644\u5f55E\quad " + (" \u5b8c\u6574\u7a0b\u5e8f" if full else "\u6838\u5fc3\u6e90\u7a0b\u5e8f\u8282\u9009")
520     out = [r"\clearpage",
521            r"\section*{%s}" % title,
522            r"\addcontentsline{toc}{section}{%s}" % title]
523     if full:
524         out.append("\u672c\u9644\u5f55\u7ed9\u51fa\u751f\u6210\u672c\u6587\u5168\u90e8\u7ed3\u679c\u4e0e\u56fe\u8868\u7684\u5b8c\u6574\u53ef\u2192
\u8fd0\u884c\u6e90\u7a0b\u5e8f\u5f0c\u4e0e\u7535\u5b50\u652f\u6491\u6750\u6599 \texttt{src/} \u76ee\u5f55\u9010\u5b57\u4e00\u81f4\u2192
u3002"
525             "\u6309\u4f9d\u8d56\u987a\u5e8f\u6392\u5217\u5f1a\u60c5\u666f\u6a21\u578b\u4e0e\u4f18\u5316\u3001\u6392\u653e\u2014\u529f\u2192
u7387\u524d\u6cbf\u3001\u7ed8\u56fe\u6837\u5f0f\u4e0e\u56fe\u96c6\u751f\u6210\u3001\u786e\u5b9a\u6027\u6838\u9a8c\u3001\u63d0\u4ea4\u2192
u7a3f\u88c5\u914d\u3002")
526         for n, name in enumerate(CODE_MODULES, start=1):
527             path = SRC / name
528             if not path.exists():
529                 continue
530             out += [r"\subsection*{E.%d\quad \texttt{%s}}" % (n, esc(name)),
531                    r"\lstinputlisting[style=pycode]{%s}" % (path.as_posix())]
532         return "\n".join(out)
533
534     out.append("\u4e3a\u63a7\u5236\u7bc7\u5e45\u5f0c\u672c\u9644\u5f55\u53ea\u7ed9\u51fa\u51b3\u5b9a\u5168\u6587\u6570\u503c\u7ed3\u679c\u2192
u7684\u4e24\u6bb5\u6838\u5fc3\u5b9a\u4e49\u5f1b"
535             "\u5b8c\u6574\u53ef\u8fd0\u884c\u6e90\u7801\u968f\u7535\u5b50\u652f\u6491\u6750\u6599\u63d0\u4ea4\u3002")
536     BUILD.mkdir(exist_ok=True)
537     for n, (module, names) in enumerate(CORE_EXTRACTS, start=1):
538         snippet = BUILD / ("code_excerpt_%d.py" % n)
539         snippet.write_text(_extract(module, names), encoding="utf-8")
540         out += [r"\subsection*{E.%d\quad \texttt{%s}\u5f1a%s}"
541                % (n, esc(module), "\u3001".join(esc(x) for x in names)),
542                r"\lstinputlisting[style=pycode]{%s}" % snippet.as_posix()]
543     return "\n".join(out)
544
545
546 def main() -> None:
547     include_code = "--no-code" not in sys.argv
548     full_code = "--core-code" not in sys.argv
549     md = MANUSCRIPT.read_text(encoding="utf-8")
550     title, abstract, body = convert(md)
551     tex = build_tex(title, abstract, body, code_appendix(full_code) if include_code else "")
552
553     BUILD.mkdir(exist_ok=True)
554     tex_path = BUILD / "paper.tex"
555     tex_path.write_text(tex, encoding="utf-8")
556
557     for run in range(2):
558         proc = subprocess.run(
559             ["xelatex", "-interaction=nonstopmode", "-halt-on-error",
560              "-output-directory", str(BUILD), str(tex_path)],
561             cwd=ROOT, capture_output=True, text=True)
562         if proc.returncode != 0:
563             log = (BUILD / "paper.log").read_text(encoding="utf-8", errors="replace")
564             errs = [l for l in log.split("\n") if l.startswith("!")][12:]
565             print("XeLaTeX 失败: \n" + "\n".join(errs))
566             sys.exit(1)
567         print("已生成", BUILD / "paper.pdf")
568
569
570 if __name__ == "__main__":
571     main()

```